

臺灣自編大學教科書

大模型導論

Introduction to Large Language Models

第 1 篇

大模型視野、數學與工程基礎

Vision, Mathematics, and Engineering Foundations

第 1 章 大模型全貌、能力邊界與臺灣機會

第 2 章 從統計語言模型到 Transformer 與基礎模型

第 3 章 大模型所需的數學、機率與最佳化

第 4 章 Python、PyTorch 與可重現的大模型實驗環境

李世淵（機器人叫獸） 編著

助教 李天宇、李宇晴

2026 年 8 月 第一版

序言 為什麼要有一本臺灣自己的大模型教科書

這本書的起點，是一個雲林的高中老師問聊天機器人「我們學校今年免試入學的超額比序怎麼算」，而模型把新北市的規則整套搬了過來。它答得條理分明、語氣自信，只是完全錯了。

那不是模型不會講中文。那是它不認識臺灣。

這件事可以有兩種反應。一種是聳聳肩說「模型就是這樣，多提示幾次就好」；另一種是問：我們能不能做出一個真的認識這裡的模型？如果能，需要什麼？如果不能，卡在哪裡？這本書是第二種反應的產物。

這本書想解決什麼問題

市面上關於大模型的中文書籍不算少，但大多落在兩個極端：一端是翻譯自國外的教材，內容紮實但情境全是英文世界的；另一端是應用導向的入門書，教你怎麼呼叫 API，卻不教你模型裡面發生了什麼。前者讓學生學得懂原理卻用不上，後者讓學生用得順卻改不動。

這本書想走中間那條路：從原理到系統、從資料到部署、從技術到治理，一路走完，而且每一步都放在臺灣的情境裡。你在這本書裡處理的不是虛構的資料集，而是繁體中文的編碼問題、臺灣公文的用語規範、半導體與工具機的術語、以及國網中心的算力申請條件。

這不是為了本土化而本土化。第 1 章會用五個可量測的技術理由說明：為什麼通用的國際模型在臺灣場域會系統性地表現不足，以及那些不足具體出現在哪裡。理解了那些理由，你才知道自己該做什麼。

這本書假設你是誰

主要讀者是大學的資訊、電機、機械、智慧製造與相關學程的學生，但我在寫作時始終放著另外兩類讀者在心裡：一類是高中的資優班與 AI 社團學生，另一類是在職的工程師與教師。

對高中讀者，每一章都標示了可以跳過的段落，並提供以數值實驗取代數學證明的替代路徑——第 3 章有一整節在講這件事。對在職讀者，每一章的系統案例都是可以直接搬進工作現場的規格與流程。

技術上，你需要的先備知識是：能讀寫基本的 Python，知道什麼是函式與迴圈。線性代數與微積分有修過更好，沒有也能跟上——第 3 章會把需要的部分補齊，而且是用張量形狀與可執行程式的方式補，不是用定理與證明。

第 1 篇的位置

全書共十篇四十章。你手上這一冊是第 1 篇，涵蓋第 1 到第 4 章，任務是把地基打好。

第 1 章建立全書的共同語言：名詞的邊界、能力與失效模式、四條技術路線的取舍、生命週期的七個

階段，以及臺灣為什麼需要本土模型。第 2 章沿著技術史走一遍，但重點不是背名字，而是看見每個方法撞到的牆。第 3 章補齊數學，核心技能是張量形狀推理與數值穩定性。第 4 章建立可重現的工作環境——這是全書唯一一章不談模型的，卻是最多專題敗在上面的一章。

這四章讀完，你會有一份需求規格書、一張技術演進因果圖、一頁數學速查表，以及一套可以一鍵驗證的開發環境。它們看起來都不像「做出一個模型」，但往後三十六章都會建立在它們上面。

怎麼使用這本書

每一章的結構是固定的：叫獸開講破題、學習目標、核心課文、程式實作、系統案例、章末評量、研究延伸、常見問題、教師使用建議、本章小結、成果檢核。

我建議的讀法是：先讀完「叫獸開講」與「學習目標」，然後直接跳到「成果檢核」看這一章要你交出什麼。帶著那個目標回頭讀正文，效率會高很多。程式實作至少要動手做兩個——這本書的所有實作都設計成一般筆電可以完成，沒有 GPU 不是藉口。

另外，請從第 1 章就選定一個你要跟著走完四十章的貫穿式案例。有了具體的題目，後面每一章的內容都會自動找到落點。

關於用語

本書一律使用臺灣的專業用語與教育部標準用字。有幾組容易混淆的詞，在此先行說明，往後全書一致遵守：

本書用語	對應英文	說明與區別
推論	inference	指模型執行、產生輸出的階段。與下一列刻意區分。
推理	reasoning	指模型的思考與邏輯推導能力，如「推理模型」。
分散式	distributed	指多機多卡的訓練或部署，如「分散式訓練」。
分布式	distributional	指統計分布相關概念，如「分布式假說」。
最佳化	optimization	不使用「優化」。最佳化器即 optimizer。
正規化	normalization	涵蓋文字正規化與層正規化。不使用「歸一化」。
激勵函數	activation function	不使用「激活函數」。
詞元	token	不使用「標記」或「令牌」。分詞器即 tokenizer。

本書用語	對應英文	說明與區別
資料	data	不使用「數據」。資料集即 dataset。
程式	program / code	不使用「程序」。演算法即 algorithm。
專案	project	不使用「項目」。
品質	quality	不使用「質量」。質量一詞保留給物理意義。
介面	interface	不使用「接口」。
記憶體	memory / RAM	GPU 上的則稱顯示記憶體或直接用 VRAM。
快取	cache	不使用「緩存」。
行程 / 執行緒	process / thread	不使用「進程 / 線程」。
智慧體	agent	首次出現時並列英文，其後多直接使用 Agent。

英文專有名詞在首次出現時會並列原文，其後視情況直接使用英文（例如 Transformer、tokenizer、LoRA）——這反映臺灣工程現場的真實習慣，強行全部中譯反而增加閱讀負擔。

版本與更新

大模型領域變動極快。本書的處理原則是：正文只寫穩定的原理與方法，模型名稱、套件版本、價格、法規條號等版本敏感的內容一律標註查核日期，並集中於課程網頁維護。你在做任何實驗報告時，也請養成同樣的習慣：在報告開頭寫明模型版本、取得日期與查核日期。

書中所有涉及法律與醫療的敘述均為工程教學用途，實際適用前必須經專業人士確認。這一點在相關章節會重複提醒。

李世淵（機器人叫獸）

2026 年 8 月 於雲林

目 錄

序言 為什麼要有一本臺灣自己的大模型教科書

目錄

全書架構總覽

第 1 章 大模型全貌、能力邊界與臺灣機會

The LLM Landscape, Capability Boundaries, and Taiwan Opportunity

學習目標

- 1.1 人工智慧、生成式 AI、基礎模型、大語言模型與推理模型的關係
- 1.2 大模型能做什麼：從生成到工具使用
- 1.3 大模型不能保證什麼：五種失效模式
- 1.4 從封閉 API、開放權重到自建模型：四條路線的取捨
- 1.5 大模型生命週期：七個階段與一個迴圈
- 1.6 臺灣本土大模型的必要性
- 1.7 臺灣的工程優勢與真實限制
- 1.8 本書地圖與學習建議
- 1.9 本章名詞中英對照

程式實作

系統案例：本土模型與通用模型的臺灣情境對照

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 2 章 從統計語言模型到 Transformer 與基礎模型

From Statistical Language Models to Transformers and Foundation Models

學習目標

- 2.1 語言模型到底在解什麼問題
- 2.2 n-gram 時代：數數也能做語言模型
- 2.3 神經語言模型：從離散計數到連續向量
- 2.4 RNN、LSTM 與序列建模的瓶頸
- 2.5 Encoder-Decoder、注意力與翻譯的轉捩點
- 2.6 Transformer 與尺度化紅利
- 2.7 三類架構：BERT、T5 與 GPT
- 2.8 從預訓練到 Agentic AI：能力的四次躍遷
- 2.9 臺灣視角的技術史

程式實作

系統案例：繪製 2017–2026 大模型技術地圖

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 3 章 大模型所需的數學、機率與最佳化

Mathematics, Probability, and Optimization for Large Language Models

學習目標

3.1 張量與形狀推理：本章最重要的技能

3.2 矩陣運算、廣播與愛因斯坦記法

3.3 機率、最大概似與交叉熵

3.4 導數、梯度與反向傳播

3.5 最佳化器：從 SGD 到 AdamW

3.6 數值穩定性：浮點數的真相

3.7 高中生友善路徑：用數值實驗取代形式證明

程式實作

系統案例：Attention 張量尺寸追蹤器

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

附：數學速查表範本

第 4 章 Python、PyTorch 與可重現的大模型實驗環境

Python, PyTorch, and Reproducible LLM Experiment Environments

學習目標

4.1 大模型開發需要的 Python 子集

4.2 PyTorch 五大元件

4.3 Notebook、腳本與套件：三種形態的分工

4.4 硬體偵測、資源估算與降級執行

4.5 環境、依賴與跨平台

4.6 隨機性、種子與可重現的真相

4.7 設定檔、日誌與實驗追蹤

4.8 版本控制、資料、權重與機密

4.9 教室現場：共用資源與作業規範

程式實作
系統案例：三級教學環境的統一介面設計
章末評量
研究延伸與版本注意事項
常見問題
教師使用建議
本章小結
成果檢核

索引

以下為 Word 自動目錄欄位。在 Word 中開啟後，於下方按右鍵選擇「更新功能變數」即可產生含頁碼的三層目錄；亦可使用左側導覽窗格依書籤瀏覽。

序言 為什麼要有一本臺灣自己的大模型教科書	2
這本書想解決什麼問題.....	2
這本書假設你是誰.....	2
第 1 篇的位置.....	2
怎麼使用這本書.....	3
關於用語.....	3
版本與更新.....	4
目 錄.....	5
全書架構總覽.....	15
本冊四章的能力遞進.....	15
第 1 篇.....	17
大模型全貌、能力邊界與臺灣機會.....	18
學習目標.....	18
1.1 人工智慧、生成式 AI、基礎模型、大語言模型與推理模型的關係.....	19
1.1.1 一組同心圓，但不是每一圈都真的包含	19
1.1.2 「大」到底大在哪裡	20
1.1.3 基礎模型：一個 2021 年才被正式命名的概念	20
1.1.4 指令模型與推理模型的分野	21
1.1.5 為什麼贏的是 Transformer	21
1.1.6 名詞速記表.....	21
1.2 大模型能做什麼：從生成到工具使用	22
1.2.1 七類核心能力.....	22
1.2.2 為什麼「猜下一個字」可以做到這些事	23
1.2.3 能力的三個來源：預訓練、後訓練、推論期.....	23
1.2.4 怎麼實際量出「能力邊界」在哪裡	24

1.2.5	湧現能力：一個仍有爭議的概念.....	24
1.3	大模型不能保證什麼：五種失效模式.....	25
1.3.1	幻覺（hallucination）.....	25
1.3.2	知識時效.....	26
1.3.3	偏見與代表性不足.....	26
1.3.4	脆弱推理.....	26
1.3.5	過度自信與校準不良.....	26
1.3.6	兩個容易被忽略的附加問題.....	26
1.3.7	怎麼向非技術的決策者解釋這些風險.....	27
1.3.8	把失效模式整理成工程檢查表.....	28
1.4	從封閉 API、開放權重到自建模型：四條路線的取舍.....	28
1.4.1	四條路線.....	28
1.4.2	六個必須同時評估的維度.....	29
1.4.3	「開源」與「開放權重」不是同一件事.....	29
1.4.4	一個完整的成本試算範例.....	30
1.4.5	一個給臺灣多數場域的務實建議.....	31
1.4.6	三個臺灣單位的實際選擇（迷你案例）.....	31
1.5	大模型生命週期：七個階段與一個迴圈.....	32
1.5.1	七個階段.....	32
1.5.2	它不是一條直線，是一個迴圈.....	33
1.5.3	成本分布：時間都花在哪裡.....	33
1.5.4	誰來做：一個大模型專案的角色分工.....	34
1.5.5	一個 18 週專題的真實時程長什麼樣.....	34
1.6	臺灣本土大模型的必要性.....	35
1.6.1	理由一：用語不是小事.....	35
1.6.2	理由二：制度知識無法用翻譯補上.....	36
1.6.3	理由三：token 稅是實實在在的成本.....	36
1.6.4	理由四：資料主權與可控性.....	37
1.6.5	理由五：產業知識在臺灣人的手上.....	37
1.6.6	把開場那個錯誤完整拆解一次.....	37
1.7	臺灣的工程優勢與真實限制.....	38
1.7.1	三個可行的差異化切入點.....	39
1.8	本書地圖與學習建議.....	39
1.8.1	十篇四十章的能力遞進.....	39
1.8.2	四級硬體：沒有 GPU 也能學.....	40
1.8.3	三個常見的學習誤區.....	40
1.8.4	怎麼使用本書的三種讀者.....	40
1.9	本章名詞中英對照.....	41
	程式實作.....	42

程式 1A 三模型繁中能力對照實驗.....	42
程式 1B LLM 生命週期檢查器	44
程式 1C 臺灣常識三十題快測與錯誤分類.....	45
系統案例：本土模型與通用模型的臺灣情境對照.....	46
實驗設計	46
你應該會觀察到的現象（先預測，再驗證）	47
報告格式（本書所有系統案例共用）	47
章末評量	48
一、概念題	48
二、推導與計算題.....	48
三、除錯題	48
四、系統設計題	48
五、臺灣情境題	49
六、倫理討論題	49
評量提示（給自學者）	49
研究延伸與版本注意事項	49
常見問題	50
教師使用建議	51
本章小結	52
成果檢核	52
從統計語言模型到 Transformer 與基礎模型	54
學習目標	54
2.1 語言模型到底在解什麼問題.....	54
2.1.1 從「這句話像不像人話」開始.....	55
2.1.2 鏈式法則：把一個難題拆成 T 個小題.....	55
2.1.3 為什麼要取對數	55
2.1.4 困惑度：一個你會看一辈子的指標	56
2.2 n-gram 時代：數數也能做語言模型.....	56
2.2.1 馬可夫假設：只看最近的 $n-1$ 個字	56
2.2.2 第一道牆：資料稀疏	57
2.2.3 平滑法：把機率分一點給沒見過的組合	57
2.2.4 一個完整的手算範例	58
2.2.5 中文的額外難題：斷詞決定了一切	59
2.2.6 第二道牆：長距離依賴.....	60
2.3 神經語言模型：從離散計數到連續向量.....	60
2.3.1 核心轉變：詞不再是符號，而是向量.....	60
2.3.2 Word2Vec 與分布式假說.....	60
2.3.3 靜態詞向量的極限.....	61
2.3.4 Bengio 的神經語言模型：把兩件事一起學.....	61

2.4	RNN、LSTM 與序列建模的瓶頸	62
2.4.1	RNN 的基本想法：把歷史壓進一個狀態	62
2.4.2	梯度消失：理論上可以，實際上不行	62
2.4.3	LSTM 與 GRU：用閘門管理記憶	62
2.4.4	LSTM 的細胞狀態：為什麼加法比乘法安全	63
2.4.5	真正的死結不是梯度，是平行度	63
2.5	Encoder-Decoder、注意力與翻譯的轉捩點	64
2.5.1	固定長度向量的瓶頸	64
2.5.2	注意力機制：翻譯到哪裡，就回頭看哪裡	65
2.5.4	三種計算相關性分數的方式	65
2.5.5	從注意力到自注意力	66
2.5.6	用一個句子走一遍自注意力	66
2.6	Transformer 與尺度化紅利	67
2.6.1	2017 年那篇論文做對了什麼	67
2.6.2	Transformer Block 的五個零件	67
2.6.3	真正的突破：投入可以預期地轉成能力	68
2.6.4	Transformer 也有它的牆	69
2.7	三類架構：BERT、T5 與 GPT	69
2.7.1	為什麼 Decoder-only 贏了	69
2.7.2	一個常見的混淆：GPT 的「G」不是生成的意思嗎	70
2.8	從預訓練到 Agentic AI：能力的四次躍遷	70
2.8.1	第一次躍遷：預訓練加微調	71
2.8.2	第二次躍遷：提示與少樣本學習	71
2.8.3	第三次躍遷：指令微調與對齊	71
2.8.4	第四次躍遷：推理、工具與多模態	72
2.8.5	一張把架構與能力分開看的年表	72
2.9	臺灣視角的技術史	73
2.9.1	中文斷詞：一個臺灣長期投入的問題	73
2.9.2	繁體中文語料的結構性稀缺	73
2.9.3	臺灣本土模型的嘗試	73
2.9.4	一個值得思考的反事實	74
	程式實作	75
	程式 2A 字元級 bigram 語言模型	75
	程式 2B 三代模型同場競技	76
	程式 2C 平行度實測：RNN 與 Transformer 的牆鐘時間曲線	77
	系統案例：繪製 2017–2026 大模型技術地圖	78
	製作規格	79
	必須涵蓋的節點（至少十二個）	79
	評分重點	79

章末評量	81
一、概念題	81
二、推導與計算題	81
三、除錯題	81
四、系統設計題	82
五、臺灣情境題	82
六、倫理討論題	82
評量提示（給自學者）	82
研究延伸與版本注意事項	83
常見問題	83
教師使用建議	84
本章小結	84
成果檢核	85
大模型所需的數學、機率與最佳化	87
學習目標	87
3.1 張量與形狀推理：本章最重要的技能	87
3.1.1 從純量到張量	88
3.1.2 形狀推理的三條規則	89
3.1.3 用斷言把形狀寫進程式	90
3.1.4 從形狀算出記憶體：一項每天都會用到的心算	90
3.2 矩陣運算、廣播與愛因斯坦記法	91
3.2.1 內積：一切相似度的起點	91
3.2.2 矩陣乘法：不是一件事，是很多件事一起做	91
3.2.3 廣播：方便但危險	92
3.2.4 愛因斯坦記法：把形狀寫在程式裡	93
3.2.5 算一次矩陣乘法要多少次浮點運算	93
3.3 機率、最大概似與交叉熵	94
3.3.1 從第 2 章接上來	94
3.3.2 為什麼負對數概似等於交叉熵	95
3.3.3 softmax：把任意實數變成機率	95
3.3.4 數值穩定：為什麼不能照公式寫	96
3.3.5 log-sum-exp：同一個技巧的一般形式	96
3.3.6 資訊理論的三個量：熵、交叉熵、KL 散度	97
3.4 導數、梯度與反向傳播	98
3.4.1 導數的工程直覺	98
3.4.2 鏈式法則：把長式子拆成短式子	98
3.4.3 計算圖與自動微分	98
3.4.4 交叉熵對 logits 的梯度：一個漂亮的結果	99
3.4.5 梯度檢查：確認你的推導是對的	100

3.4.6 常用運算的反向公式速查.....	100
3.5 最佳化器：從 SGD 到 AdamW.....	101
3.5.1 SGD：最單純的那一個.....	101
3.5.2 Momentum：給它一點慣性.....	102
3.5.3 Adam：每個參數有自己的學習率.....	102
3.5.4 AdamW：把權重衰減修對.....	102
3.5.5 四者對照與記憶體代價.....	103
3.5.6 學習率排程的預覽.....	103
3.5.7 初始化：訓練開始之前就決定了一半.....	104
3.5.8 批次大小、梯度雜訊與有效批次.....	104
3.6 數值穩定性：浮點數的真相.....	105
3.6.1 浮點數不是實數.....	105
3.6.2 常見的浮點格式.....	106
3.6.3 訓練中的四種數值災難.....	106
3.6.4 梯度裁切.....	107
3.6.5 混合精度：一個工程上的折衷.....	107
3.7 高中生友善路徑：用數值實驗取代形式證明.....	108
3.7.1 三個可以直接跑的替代路徑.....	108
3.7.2 什麼時候必須回頭補數學.....	109
3.7.3 高中課程的建議切法.....	109
3.7.4 臺灣工程教育現場的一個觀察.....	109
程式實作.....	111
程式 3A 只用 NumPy 實作數值穩定的 softmax 與交叉熵.....	111
程式 3B 梯度檢查與抓蟲.....	112
程式 3C 最佳化器競技場.....	113
系統案例：Attention 張量尺寸追蹤器.....	114
功能規格.....	114
核心計算.....	114
交付要求.....	115
章末評量.....	116
一、概念題.....	116
二、shape tracing 十題.....	116
三、推導與計算題.....	116
四、除錯題.....	117
五、系統設計題.....	117
六、臺灣情境與倫理題.....	117
評量提示（給自學者）.....	117
研究延伸與版本注意事項.....	118
常見問題.....	118

教師使用建議	119
本章小結	119
成果檢核	120
附：數學速查表範本	121
Python、PyTorch 與可重現的大模型實驗環境	122
學習目標	122
4.1 大模型開發需要的 Python 子集	122
4.1.1 型別註記：給人看的，也給工具看的	123
4.1.2 資料類別與設定物件	123
4.1.3 迭代器、生成器與大檔案處理	124
4.1.4 例外處理：讓失敗留下線索	124
4.2 PyTorch 五大元件	125
4.2.1 Tensor 與裝置	125
4.2.2 autograd 的開關	126
4.2.3 nn.Module：參數是怎麼被找到的	126
4.2.4 Dataset 與 DataLoader	127
4.3 Notebook、腳本與套件：三種形態的分工	128
4.3.1 建議的專案結構	128
4.3.2 最重要的三個測試	129
4.4 硬體偵測、資源估算與降級執行	130
4.4.1 先知道自己有什麼	130
4.4.2 從硬體反推可訓練的模型大小	130
4.4.3 降級執行：讓同一份程式碼在四級硬體上都能跑	131
4.5 環境、依賴與跨平台	132
4.5.1 為什麼「我這邊可以跑」不是答案	132
4.5.2 Windows 與 Linux 的五個常見差異	132
4.5.3 容器：一勞永逸但有門檻	133
4.6 隨機性、種子與可重現的真相	134
4.6.1 隨機性從哪裡來	134
4.6.2 為什麼位元級重現往往做不到	135
4.7 設定檔、日誌與實驗追蹤	135
4.7.1 把設定從程式碼裡拿出來	136
4.7.2 日誌：不只是 print	136
4.7.3 環境指紋：讓每次執行都能被還原	137
4.7.4 實驗追蹤的最小可行方案	138
4.8 版本控制、資料、權重與機密	138
4.8.1 什麼該進 Git，什麼不該	138
4.8.2 金鑰管理：三條不可妥協的規則	139
4.8.3 資料與模型的版本	140

4.9 教室現場：共用資源與作業規範.....	141
4.9.1 共用 GPU 的三個問題與對策.....	141
4.9.2 作業繳交的統一規範	141
程式實作	143
程式 4A 可重現的訓練模板	143
程式 4B 矩陣乘法基準測試	144
程式 4C 實驗追蹤器最小實作	145
系統案例：三級教學環境的統一介面設計	147
設計需求	147
必須交付的四份文件	147
評分重點	147
章末評量	149
一、概念題	149
二、計算與估算題	149
三、除錯題	149
四、系統設計題	150
五、臺灣情境與倫理題	150
評量提示（給自學者）	150
研究延伸與版本注意事項	151
常見問題	151
教師使用建議	151
本章小結	152
成果檢核	153
索 引	155
英文詞條	155
中文詞條	155

全書架構總覽

全書分為十篇四十章，另有十四組附錄。本冊為第 1 篇。以下總覽讓你隨時知道自己在哪裡，以及每一篇會交出什麼。

篇次	章次	主題	核心產出
第 1 篇 (本冊)	1–4	大模型視野、數學與工程基礎	需求規格書、技術演進圖、數學速查表、可重現環境
第 2 篇	5–9	語言建模、臺灣語料與 Token 工程	臺灣文本工程規格、自訓 tokenizer、語料清冊
第 3 篇	10–13	Transformer 從零實作	MiniFormosaGPT 與可生成繁中的模型
第 4 篇	14–17	預訓練、解碼與實驗科學	穩定訓練迴圈、尺度化推估與算力預算
第 5 篇	18–21	算力、系統與現代架構	效能剖析、分散式決策、臺灣算力盤點
第 6 篇	22–26	本土化後訓練與對齊	繁中適配模型、LoRA、偏好對齊、邊緣小模型
第 7 篇	27–30	推理、知識更新與提示工程	推理實驗、知識更新流程、RAG 原型
第 8 篇	31–34	應用系統：進階 RAG、Agent 與產品開發	知識系統、受限 Agent、可上線的應用
第 9 篇	35–37	部署、評測、安全與治理	量化部署、臺灣題庫、紅隊與影響評估
第 10 篇	38–40	多模態、具身智慧與總整專題	VLM 應用、臺語語音、VLA、總整專題

本冊四章的能力遞進

章	你會理解什麼	你會做出什麼	往後被誰使用
第 1 章	大模型的全貌、邊界與臺灣的處境	需求規格書 v0.1	第 8、36、40 章

章	你會理解什麼	你會做出什麼	往後被誰使用
第 2 章	每個技術突破解決與製造的問題	技術演進因果圖	第 20、40 章
第 3 章	張量形狀、梯度與數值穩定性	數學速查表、shape tracing	第 11、13、15、18 章
第 4 章	可重現性為何是科學而非潔癖	訓練模板、基準測試、追蹤器	第 13 章之後全部

第 1 篇

大模型視野、數學與工程基礎

Vision, Mathematics, and Engineering Foundations

第 1–4 章

本篇承諾：建立共同語言、數學直覺與可重現的開發環境。高中生能看懂全貌，大學生能推導公式，工程學生能在筆電上跑出第一個實驗。

第 1 章

大模型全貌、能力邊界與臺灣機會

The LLM Landscape, Capability Boundaries, and Taiwan Opportunity

難度：核心 | 建議授課：第 1 週 | 硬體需求：A 級（一般筆電即可完成全部實作）

叫獸開講

一位雲林的高中老師打開某個知名的聊天機器人，問它「我們學校今年免試入學的超額比序怎麼算」。模型答得頭頭是道，條理分明，語氣自信——然後把新北市的規則整套搬了過來。它不是不會講中文，它是不認識臺灣。這一章要回答的問題是：為什麼會這樣？以及，我們能不能做出一個真的認識這裡的模型？

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 用一張同心圓圖區分人工智慧、機器學習、深度學習、生成式 AI、基礎模型、大語言模型與推理模型，並說出每一層的判準。
2. 列舉大模型的七類核心能力，並為每一類找出至少一個臺灣本地的應用情境。
3. 說明幻覺、知識時效、偏見、脆弱推理與過度自信五種失效模式的成因，並指出各自對應的工程對策。
4. 比較封閉 API、開放權重與自建模型三條路線在控制力、成本、隱私、授權與能力上限上的差異，並為一個給定需求做出選擇。
5. 畫出大模型的七階段生命週期，指出每階段的產出物、負責角色與最容易被跳過的一步。
6. 用可查證的證據（用語、制度知識、token 效率）論證臺灣為何需要本土模型，而不是只靠翻譯或提示工程。
7. 實際量測三個模型在同一組繁體中文提示上的正確性、用語、token 數、延遲與成本，並做出可解釋的比較圖表。
8. 完成一份「臺灣本土大模型需求規格書 v0.1」，明確定義使用者、任務、語言、資料邊界、評測指標與不可接受風險。

給高中讀者的提示

本章沒有任何需要微積分或線性代數的內容，只有觀念與一個要動手跑的實驗。1.1 到 1.3 節請務必讀完，1.4 與 1.5 節可以先讀表格、跳過細節，等到第 22 章要真正選模型時再回來看。

1.1 人工智慧、生成式 AI、基礎模型、大語言模型與推理模型的關係

這幾年，「AI」這兩個字被用得太浮濫，浮濫到它幾乎失去了資訊量。有人說「我們公司導入了 AI」，可能是指買了一套規則引擎；也可能是指工程師寫了一支腳本去呼叫某個聊天機器人的 API。在課堂上，我們必須先把名詞的邊界劃清楚，否則後面四十章的討論會一路模糊下去。

1.1.1 一組同心圓，但不是每一圈都真的包含

最常見的教學方式是畫同心圓：人工智慧最大，裡面包著機器學習，再裡面是深度學習，然後是生成式 AI，最裡面是大語言模型。這張圖大致正確，但有兩個地方容易誤導初學者，必須先講清楚。

第一，人工智慧不等於機器學習。早期的專家系統、規則引擎、搜尋演算法、邏輯推論器都是不折不扣的人工智慧，它們沒有任何「學習」的成分。工廠裡跑了二十年的排程最佳化程式、下棋程式裡的 alpha-beta 剪枝、導航軟體裡的 A* 搜尋，全都是 AI。把 AI 直接等同於「用資料訓練出來的模型」，會讓你看不見這些方法在今天仍然有效——事實上，第 32 章講 Agent 時，我們會回頭把規則與搜尋當成安全機制的一部分。

第二，生成式 AI 不完全被深度學習包含。統計式的馬可夫鏈文字產生器、文法式的模板生成器都會「生成」內容，只是效果差。今天我們談的生成式 AI 幾乎都是深度學習做的，但概念上兩者不是嚴格的包含關係。

所以更精確的說法是：這是一組「大致由外而內、但邊界會互相滲透」的概念層。我們用下表把每一層的判準寫死，往後全書都以此為準。

層級	判準（怎麼判斷屬不屬於）	代表方法	在本書的位置
人工智慧 AI	讓機器展現通常需要人類智能才能完成的行為，不限手段	規則引擎、搜尋、規劃、機器學習	第 32、37 章（安全與規則約束）
機器學習 ML	行為來自資料中歸納出的參數，而非人工撰寫的規則	迴歸、決策樹、SVM、神經網路	第 3 章（最佳化基礎）
深度學習 DL	使用多層神經網路自動學出表徵，不需人工設計特徵	CNN、RNN、Transformer	第 10–13 章
生成式 AI	輸出是高維度的新內容（文字、影像、語音），而非單一標籤或數值	GPT 系列、擴散模型、語音合成	全書主線
基礎模型 FM	在大規模廣泛資料上預訓練、可經少量調整適配到大量下游任務	GPT、Llama 系列、CLIP、Whisper	第 22–26 章（適配）

層級	判準（怎麼判斷屬不屬於）	代表方法	在本書的位置
大語言模型 LLM	以文字（詞元序列）為主要模態的基礎模型，通常是自回歸解碼器	Decoder-only Transformer	第 11–20 章
推理模型	在輸出答案前會產生較長的內部思考過程，並以此換取正確率	思考鏈訓練、可驗證獎勵強化學習	第 27 章
多模態模型 VLM/VLA	同時處理文字與影像、語音或動作，共用同一個語言骨幹	視覺編碼器 + 語言模型	第 38–39 章

1.1.2 「大」到底大在哪裡

大語言模型的「大」，指的其實是三件同時發生的事，缺一不可：

- 參數量。從早期的數億到今天動輒數百億、數千億。參數量決定模型能記住多少東西、能表達多複雜的函數。
- 訓練資料。以兆 (trillion) 為單位的詞元數。資料量決定模型見過多少世界。
- 訓練算力。以 GPU 年為單位的浮點運算。算力是前兩者能不能被真正吸收的前提。

這三者不是各自獨立可以任意加大的旋鈕，它們之間有相當精確的最佳配比關係——第 17 章的尺度定律會用數學把這件事講清楚。現在你只要記住一個直覺：如果你有一台很大的模型卻只餵了很少的資料，那是浪費；如果你有海量資料卻用一個很小的模型去吃，那是吃不下。大模型工程的核心問題之一，就是在給定的算力預算下，決定模型該多大、資料該多少。

另外要提醒一個常見的誤會：參數量不是能力的唯一指標，甚至常常不是最重要的指標。一個經過良好資料清理、良好後訓練、良好對齊的 8B 模型，在特定任務上打敗一個粗製濫造的 70B 模型是家常便飯。第 26 章會用具體實驗證明這件事，而這正是臺灣的機會所在——我們不一定要比大，但我們可以比準、比合適。

1.1.3 基礎模型：一個 2021 年才被正式命名的概念

「基礎模型」(foundation model) 這個詞是史丹佛大學的研究群在 2021 年提出的，用來描述一種新的產業結構：不再是每個任務訓練一個模型，而是先訓練一個通用的大模型，再由無數下游應用去適配它。這個轉變的意義比技術本身還大——它把 AI 從「每家公司各自訓練」變成「少數機構訓練、多數機構適配」，也因此帶來了集中化、依賴與主權的問題。這正是第 1.6 節與第 37 章要處理的核心議題。

基礎模型的三個特徵是：規模 (scale)、通用性 (generality)、可適配性 (adaptability)。第三

點最關鍵。如果一個模型很大很強，但每次要在新任務上都得從頭訓練，那它就不是基礎模型，只是一個大模型。正因為可適配，才有了第六篇整整五章在講的持續預訓練、指令微調、LoRA 與偏好對齊——這些技術的存在前提，就是有人先幫我們把基礎模型訓練好了。

1.1.4 指令模型與推理模型的分野

你在網頁上使用的聊天機器人，背後至少經過兩個階段：先在大量文本上預訓練成一個「會接話」的基礎模型，再經過指令微調與對齊，變成一個「會照你的話做事」的指令模型。這兩者的行為差異非常大。把一個純粹的基礎模型直接拿來聊天，你問它「臺灣的首都在哪裡」，它可能回你「臺灣的首都在哪裡？這是一個常見的地理問題。下一題：日本的首都在哪裡？」——因為它學到的是「文本接下去長什麼樣」，不是「回答問題」。第 23 章會讓你親眼看到這個差異。

推理模型則是更近期的分野。它在給出最終答案之前，會先產生一段（有時很長的）內部思考，把問題拆解、嘗試、檢查、修正。代價是輸出的詞元數暴增、延遲拉長、成本上升；報酬是在數學、程式、邏輯與多步驟規劃這類任務上有顯著的正确率提升。第 27 章會告訴你一件對臺灣特別重要的事：讓模型「想久一點」，往往比讓模型「變大一點」便宜得多——這對算力受限的我們是好消息。

1.1.5 為什麼贏的是 Transformer

學生常問：注意力機制在 2014 年就有了，RNN 也一直在改進，為什麼是 2017 年的 Transformer 改變了一切？答案不在「它比較聰明」，而在「它比較適合現在的硬體」。

RNN 有一個結構性的詛咒：要算第 t 個字，必須先算完第 $t-1$ 個字。這是本質上的序列相依，無論你買多少張 GPU 都無法平行化。而 GPU 的整個設計哲學就是「同時做非常多次相同的運算」。把 RNN 放上 GPU，就像請一支交響樂團排隊一個一個上台獨奏——樂器再多也沒用。

Transformer 把序列相依拆掉了。在訓練時，一整個句子的所有位置可以同時計算，注意力矩陣是一次矩陣乘法算出來的。這讓「訓練資料變多」這件事第一次真正變得划算——你加資料、加卡、加時間，模型就會變好，而且這個關係穩定到可以寫成公式（第 17 章）。這種「投入可預期地轉成能力」的性質，是資本願意大規模進場的前提，也是整個大模型時代的真正起點。

這件事對本書的讀者有一個很實際的啟示：架構的價值不能脫離硬體來談。第 20 章討論線性注意力與狀態空間模型時，我們會用同一把尺去量它們——不是問「理論上更優雅嗎」，而是問「在真實的 GPU 與真實的記憶體頻寬上，它跑得比較快嗎」。

1.1.6 名詞速記表

以下十個縮寫會貫穿全書，請先建立印象，細節在對應章節。

縮寫	全稱	一句話說明	詳見
LLM	Large Language Model	以文字為主要模態的大型基礎模型	第 11–20 章
SFT	Supervised Fine-Tuning	用「問題—標準回答」配對教模型聽話	第 23 章
PEFT	Parameter-Efficient Fine-Tuning	只訓練少量新增參數的微調技術總稱	第 24 章
LoRA	Low-Rank Adaptation	最常用的 PEFT 方法，用低秩矩陣近似權重更新	第 24 章
RLHF	Reinforcement Learning from Human Feedback	用人類偏好訓練模型的行為	第 25 章
DPO	Direct Preference Optimization	不需獎勵模型的直接偏好最佳化	第 25 章
RAG	Retrieval-Augmented Generation	先檢索再生成，讓答案可附出處	第 30–31 章
MoE	Mixture of Experts	每次只啟用部分專家，用稀疏換效率	第 20 章
VLM	Vision-Language Model	同時看得懂影像與文字的模型	第 38 章
SLO	Service Level Objective	你對服務品質（延遲、可用率）的量化承諾	第 35 章

常見誤解澄清

「大模型就是一個超大的資料庫，把網路上的東西存起來再查出來給你。」這是錯的。模型儲存的是參數，不是文本；它沒有一份可以查詢的原文副本。這也正是幻覺的來源之一：它記住的是統計上的關聯，不是逐字的事實。想要逐字可查、可附出處的答案，你需要的是第 30 章的 RAG，不是更大的模型。

1.2 大模型能做什麼：從生成到工具使用

要判斷一項技術值不值得投入，得先看清楚它的能力邊界。這一節先講「能做什麼」，下一節馬上講「不能保證什麼」——這兩節必須連在一起讀，只讀一半的人不是過度樂觀就是過度悲觀，兩種都不適合做工程。

1.2.1 七類核心能力

能力類別	具體任務	臺灣情境範例	成熟度
生成	寫作、改寫、擴寫、風格轉換、創意發想	公文初稿、教案設計、行銷文案、學習單	高（但需人工把關）
理解與抽取	分類、命名實體辨識、關係抽取、意圖判讀	工單分類、履歷解析、法條要素抽取、客服意圖	高
翻譯與轉寫	跨語言翻譯、文白轉換、繁簡轉換、術語統一	技術文件中英互譯、臺語文字化、公文白話化	中高（術語需在地校正）
摘要	單文摘要、多文摘要、會議紀錄、重點條列	會議紀錄、法規修正對照、產線異常日報	高
程式	補全、生成、除錯、重構、測試撰寫、程式解釋	工控腳本、資料處理、教學範例、單元測試	中高（必須人工審查）
工具使用	函式呼叫、查資料庫、執行運算、操作外部系統	查詢校務系統、排課、下工單、控制機械手臂	中（風險最高）
多模態互動	看圖回答、文件影像理解、語音對話、動作生成	設備面板判讀、表單掃描、臺語語音助理、機器人指令	中（快速演進中）

1.2.2 為什麼「猜下一個字」可以做到這些事

這是每個學生第一次聽到大模型原理時都會問的問題：它只是在猜下一個詞元，怎麼可能會寫程式、會摘要、會翻譯？

一個有用的思考角度是：如果你要在所有可能的文本上，把「下一個字是什麼」猜得夠準，你被迫要學會很多東西。要猜對一段程式碼的下一行，你得學會語法與邏輯；要猜對一篇論文的結論，你得學會前面的推導；要猜對一段對話的回應，你得學會語境與意圖。換句話說，下一詞元預測是一個「表面上很簡單、但要做到極致就必須理解世界」的目標函數。第 5 章會用機率的語言把這句話說得更嚴謹。

但這個說法有它的極限，我們也必須誠實：模型學到的是「什麼樣的文字在統計上會接在後面」，這與「什麼是真的」之間有一道鴻溝。大部分時候兩者高度重合——因為人類寫下來的文字大多是對的——但一旦進入長尾、進入冷門、進入模型沒見過的地方，這道鴻溝就會裂開，那就是幻覺。

1.2.3 能力的三個來源：預訓練、後訓練、推論期

學生常把模型的表現當成一個整體來看待，但工程上必須拆開，因為每一塊的改善方法完全不同、成

本也差了好幾個數量級。

能力來源	決定了什麼	改善成本	對應章節
預訓練	世界知識、語言能力、基本推理、程式能力的天花板	極高（數十萬至數千萬元起跳）	第 9、13–20 章
後訓練	是否聽話、風格、拒答邊界、特定領域強度、推理習慣	中（單張 GPU 可做 LoRA）	第 22–27 章
推論期	這一次回答有沒有拿到對的資訊、有沒有用對工具、想得夠不夠久	低（改提示、加檢索、加工具）	第 27、29–32 章

1.2.4 怎麼實際量出「能力邊界」在哪裡

前面那張能力表寫著「成熟度：高」或「中」，這種說法對規劃有用，但對工程決策不夠。你真正需要知道的是：在你的任務上，這個模型從哪一點開始不可靠。以下是一個可以在一個下午內做完、卻能省下你數週摸索的方法，本書稱之為難度階梯法。

1. 把你的任務拆成一個由易到難的階梯，至少五階。例如摘要任務可以是：單段落摘要 → 單篇文章摘要 → 含表格的技術文件摘要 → 跨三份文件的比較摘要 → 需要判斷矛盾之處的多文件摘要。
2. 每一階出五到十題，難度必須真的遞增，而且要有明確的評分標準。
3. 對每一階量測正確率，畫出「難度—正確率」曲線。
4. 曲線開始明顯下墜的那一階，就是這個模型在你任務上的能力邊界。
5. 在邊界之外的任務，不要用提示工程硬撐——那是最常見的時間陷阱。應該改變系統設計：拆解成多步驟、加入檢索、加入驗證器，或換模型。

這個方法的價值在於它把「這個模型行不行」這種無法回答的問題，換成「它到第幾階開始不行」這種可以量測的問題。你會在第 16 章看到同樣的思路被用在實驗設計上，在第 36 章看到它被擴充成完整的評測分層。

一個實務觀察

大多數失敗的大模型專案，不是敗在選錯模型，而是敗在把任務設定在能力邊界之外，然後花三個月試圖用提示把它救回來。先量邊界、再設計系統，順序不能顛倒。

1.2.5 湧現能力：一個仍有爭議的概念

你會在很多科普文章裡看到「湧現」（emergence）這個詞：模型小的時候某項能力完全不存在，

參數量跨過某個門檻後突然出現。多步驟算術、複雜指令遵循、少樣本學習都被舉為例子。這個說法很有戲劇性，也確實描述了部分現象，但學界對它有相當有力的反駁，值得學生知道。

反駁的核心是：所謂的「突然出現」，有一部分是評測指標造成的錯覺。如果你用「答案完全正確才給分」這種全有全無的指標去量測一個需要五個步驟的任務，那麼模型從「五步錯三步」進步到「五步錯一步」時，分數依然是零；直到某一刻全部答對，分數才從零跳到一。看起來像突然湧現，實際上底層能力是平滑成長的。換成連續型指標（例如逐步驟正確率、對數機率），曲線往往就變得平滑了。

這件事對工程實務有兩個直接的教訓，兩個都會在第 36 章重演：第一，你選的指標會決定你看到的故事，所以評測設計必須在實驗開始之前就想清楚；第二，「突然變好」與「突然變壞」都要先懷疑是量測方式的問題，再懷疑是模型的問題。這是一種應該養成的專業反射。

至於湧現到底存不存在，本書的立場是：不必急著下結論，但務必用連續指標去看你自己的實驗。

這張表隱含一個對臺灣極為重要的策略判斷：預訓練那一欄我們短期內很難自己撐起來，但後訓練與推論期這兩欄，一個大學實驗室、一家中小企業、甚至一位認真的大學生就能做出真正的差異。本書把四十章裡的二十多章放在這兩欄，正是這個原因。

1.3 大模型不能保證什麼：五種失效模式

如果你只記得本章的一件事，我希望是這一節。系統上線後出事，幾乎不會是因為工程師不知道模型很強，而是因為他不知道模型會在哪裡出錯。

1.3.1 幻覺 (hallucination)

模型會用完全自信的語氣，說出一個不存在的法條條號、一篇不存在的論文、一個不存在的函式名稱。這不是偶發的 bug，而是訓練目標的直接後果：模型被獎勵的是「產生看起來合理的文字」，不是「產生真實的文字」。當它不知道答案時，它沒有「我不知道」這個預設選項，除非我們在後訓練階段教它。

幻覺最容易出現在三種地方：一是長尾知識（冷門的人、地、事、數字）；二是需要精確引用的內容（條號、頁碼、日期、型號）；三是使用者用強烈的預設立場逼問時（「請告訴我第 27 條寫了什麼」——即使根本沒有第 27 條，模型也傾向給你一個）。

工程對策不是「叫模型不要亂講」，那沒有用。真正有效的是：把事實查詢外包給檢索系統並強制附上出處（第 30–31 章）、在後訓練時明確教會拒答（第 25 章）、以及在系統設計上讓不確定性可見（第 33 章）。

1.3.2 知識時效

模型的知識停在訓練資料的截止時間。校長換了、法規修了、課綱改了、產品換代了，模型不會知道。更麻煩的是它通常也不知道自己不知道——它會用舊資訊回答你，語氣一樣自信。這在校務、法規、醫療衛教、產品規格這類場域是致命的。

對策有三條路，成本由低到高：檢索增強（讓模型每次都去查最新文件）、模型編輯（第 28 章，動手改掉特定事實）、重新訓練（最貴，通常不划算）。實務上九成以上的情況應該選第一條。

1.3.3 偏見與代表性不足

模型的世界觀來自訓練語料的分布。如果語料裡簡體中文與中國語境的比率遠高於繁體中文與臺灣語境，模型就會在不自覺中把中國的制度、用語、地名當成預設值——本章開頭那位老師遇到的正是這件事的溫和版本。更嚴重的版本會出現在職業、性別、族群、地域的刻板連結上。

這裡要特別提醒一個容易被忽略的面向：臺灣的原住民族語、客語、臺語在網路語料中的比例極低，而這些語言恰恰是最需要 AI 協助保存的。第 39 章會把這件事當成一個具體的工程任務來處理，而不是只當成一句政治正確的口號。

1.3.4 脆弱推理

模型在標準格式的題目上表現優異，但把同一題換個講法、加一句無關的干擾條件、把數字改大一點，正確率就可能大幅下降。這說明它有相當一部分的「推理」其實是對訓練資料中解題模式的比對，而不是真正的邏輯演繹。

判斷方法很簡單，你在第 27 章會實際做一次：把題目的表面特徵改掉但保持邏輯結構不變，看正確率掉多少。掉得越多，代表越依賴表面模式。這也是為什麼推理模型與可驗證獎勵訓練會成為近年的重點——它們試圖讓模型的推理真的是推理。

1.3.5 過度自信與校準不良

一個校準良好的系統，說「我有七成把握」時，它應該有大約七成的時候是對的。大模型在這件事上表現不佳：它的語氣自信程度與實際正確率之間的相關性很弱，而且經過對齊訓練之後往往更糟——因為人類標註者偏好聽起來有自信的回答。

這對系統設計的意涵是：不要相信模型自己說的信心程度，要用外部方法估計。可用的手段包括多次取樣看一致性（第 27 章）、檢索結果的支持度（第 30 章）、以及驗證器（第 27 章）。

1.3.6 兩個容易被忽略的附加問題

除了上述五種，還有兩種問題不常被列入「失效模式」，但在實務上造成的困擾一點也不少。

第一是不可重現性。只要解碼時使用了隨機取樣（第 14 章），同樣的輸入就會得到不同的輸出。這對展示無害，對驗收與除錯卻是災難：使用者回報的錯誤你可能重現不出來，昨天通過的測試今天可能失敗。工程上的處理方式是：在測試與驗收情境把 `temperature` 設為 0 或固定亂數種子，並在系統中記錄每一次呼叫的完整參數與模型版本。這件事必須從專案第一天就做，事後補做的成本極高。

第二是上下文限制與位置效應。模型的上下文長度有上限，超過就會被截斷——而且截斷通常是靜默的，不會報錯。更微妙的是，即使沒有超過上限，放在上下文中間的資訊也比放在開頭與結尾的資訊更容易被忽略，這個現象在研究上被稱為「迷失在中間」。這意味著你不能天真地把一百份文件全部塞進上下文就期待模型會全部讀進去。第 29 章的情境工程與第 30 章的檢索排序，處理的正是這個問題。

1.3.7 怎麼向非技術的決策者解釋這些風險

這是一項被嚴重低估的工程能力。你的校長、廠長、主任不會讀本書，但他們要決定要不要導入、要不要簽字、出事要不要負責。用「模型會產生幻覺因為它的訓練目標是最大化下一詞元的條件機率」去解釋，只會得到禮貌的點頭與錯誤的期待。

以下三組類比在實務上效果良好，你可以直接拿去用，但務必記得類比都會失真，關鍵處要補上正確的說法。

要解釋的事	可用的類比	類比失真之處（必須補充）	對應失效
為什麼會編造	像一位口才極好、讀過很多書但沒帶參考資料的顧問，被迫問時不會說不知道	模型沒有「知道自己不知道」的內建機制，不是不願意承認	幻覺
為什麼會過期	像一本印刷精美的年鑑，內容停在出版那一年	它不會像書一樣標示出版年份，也不會提醒你	知識時效
為什麼要附出處	像醫師開藥要寫依據，不能只說「我的判斷」	模型的「出處」若非來自檢索，也可能是編的	幻覺、時效
為什麼答案會不一樣	像請三位不同的助理寫同一份摘要	差異來自隨機取樣，可以用參數關掉	不可重現

在提案文件中，我建議永遠把三件事寫在第一頁：這個系統會在什麼情況下出錯、出錯時誰會發現、以及出錯時的補救成本。把風險寫在前面不會讓提案被否決，反而通常會讓決策者更願意簽字——因

為他知道你是清醒的。

1.3.8 把失效模式整理成工程檢查表

失效模式	成因	可觀察徵兆	工程對策	章節
幻覺	訓練目標追求流暢而非真實；長尾知識缺乏	編造條號、論文、函式名；細節越具體越可疑	RAG 強制引用、拒答訓練、輸出驗證	25、30、31
知識時效	訓練資料有截止時間	使用舊人名、舊法規、舊版本	檢索增強、模型編輯、知識更新流程	28、30
偏見	語料分布不均；臺灣語料占比低	用語漂移、制度誤植、族群刻板連結	語料配比、本土評測、偏誤量測	9、22、36
脆弱推理	部分推理來自模式比對而非演繹	換句話說就答錯；加干擾條件正確率驟降	推理模型、驗證器、多樣本一致性	27
過度自信	對齊訓練偏好自信語氣；校準不佳	語氣自信但內容錯誤；信心與正確率不相關	外部信心估計、一致性檢查、人類覆核	27、30、37
不可重現	解碼使用隨機取樣	同輸入不同輸出；測試時好時壞	固定溫度與種子、記錄完整參數	14、16
上下文溢出	長度上限與位置效應	長文件後半段被忽略；靜默截斷	情境工程、檢索排序、長度預算	29、30

工程守則

本書從第 1 章到第 40 章會反覆出現同一句話：任何一個大模型系統，都必須先定義「不可接受的失敗」是什麼，再開始設計。校務助理答錯社團名稱是可容忍的，答錯申請截止日期不是；衛教助理講錯保健常識是可容忍的，給出用藥劑量建議不是。這條線畫在哪裡，是工程師與領域專家共同的責任，不是模型的責任。

1.4 從封閉 API、開放權重到自建模型：四條路線的取舍

假設你是某科技大學的教師，想做一個能回答校規與課程問題的助理。你有四條路可以走，而選錯路線的代價通常在半年後才會顯現。

1.4.1 四條路線

路線	做法	啟動成本	典型適用情境
封閉 API	呼叫商業模型的 API，加上提示與檢索	極低（數小時）	快速驗證概念、資料不敏感、量小
開放權重 + 提示 / RAG	自行部署開放權重模型，不改權重	低（數天）	資料不可外流、需控制成本、量中等
開放權重 + 適配	在開放權重模型上做持續預訓練或微調	中（數週至數月）	需要領域能力、繁中強化、長期使用
自建預訓練	從零訓練基礎模型	極高（千萬元級以上）	國家級主權需求、特殊模態、研究目的

1.4.2 六個必須同時評估的維度

1. 控制力：模型會不會在你不知情的時候被換掉、被下架、被改變行為？封閉 API 的答案是「會」，而且你通常不會收到事前通知。這對已經通過驗收的系統是重大風險。
2. 成本結構：API 是變動成本（用多少付多少），自建是固定成本（先投入硬體，之後邊際成本低）。哪個划算取決於用量。第 21 與 35 章會教你算出損益兩平點。
3. 隱私與資料落地：學生輔導紀錄、病歷、製程參數、設計圖能不能離開你的機房？這通常不是技術問題而是法規與契約問題，而且答案往往直接排除掉某些路線。
4. 授權合規：這是最多人踩雷的一格，下一小節專門講。
5. 可維護性：你的團隊有沒有人能在半夜三點處理 GPU 掉卡、模型服務崩潰、品質突然下降？沒有的話，自建的總持有成本會遠高於帳面數字。
6. 能力上限：目前最強的通用能力仍在少數封閉模型手上。但要注意——你需要的是「這個任務上夠好」，不是「全球最強」。第 36 章會教你怎麼量測「夠好」。

1.4.3 「開源」與「開放權重」不是同一件事

這是全書最重要的名詞澄清之一，請務必記住。

- 開放權重（open weights）：模型參數可以下載，但訓練資料、訓練程式碼、資料配方通常不公開。你可以用，但你無法完整重現它，也很難確知它訓練時吃了什麼。
- 開源（open source）：在軟體的傳統定義下，應該包含可重現所需的一切。真正符合這個標準的大模型很少。
- 授權條款各不相同：有的允許商用、有的限制月活躍用戶數、有的要求衍生模型必須標註來源、

有的禁止用來訓練其他模型、有的對特定地區或用途有限制。

實務上請養成一個習慣：在下載任何模型權重之前，先把授權條款讀完，並把以下五個問題的答案寫進專案文件——能不能商用、能不能再散布、衍生模型有什麼義務、有沒有使用規模上限、有沒有禁止用途。第 22 章會提供一份完整的檢查表，附錄 F 則有可直接填寫的表格。

臺灣單位常見的三個誤區

第一，以為「可以下載」就等於「可以商用」——不一定。第二，以為用了開放權重模型就沒有法遵問題——你自己餵進去的資料仍然受個資法與契約約束。第三，以為封閉 API 一定比較安全——資料是否被用於訓練、儲存在哪個司法管轄區、保留多久，都必須逐條確認，不能靠印象。

1.4.4 一個完整的成本試算範例

路線選擇最後常常會落到一個問題上：到底哪個比較便宜？學生的直覺通常是「自建一定比較貴」或「API 一定比較貴」，兩種直覺都不可靠。下面用一個具體的例子把帳算完，你在課後習題會用同樣的方法算你自己的專案。

情境：某科技大學的校務問答助理，日間使用為主，尖峰每秒約 3 次查詢，全日約 5,000 次查詢。每次查詢的提示（含檢索到的文件片段）約 1,800 個詞元，回覆約 400 個詞元。

項目	A 方案：商業 API	B 方案：地端開放權重	備註
一次性投入	0 元	約 45 萬元（單機雙卡與周邊）	含伺服器、UPS、機櫃
每月變動成本	約 4.0 萬元（依詞元計價）	約 1.2 萬元（電力 + 維運）	不含人力
第一年總成本	約 48 萬元	約 59 萬元	B 方案較高
第三年累計	約 144 萬元	約 88 萬元	B 方案較低
損益兩平	—	約第 17 個月	$(45) \div (4.0 - 1.2) \approx 16.1$ 個月

光看這張表，超過一年半的專案就該自建。但這個結論太快了，因為表裡漏掉三項通常更重要的成本：

- 人力。地端方案需要有人處理 GPU 驅動、模型更新、服務崩潰與監控告警。若換算成 0.2 個人力，每月至少再加兩萬元以上，損益兩平會延後到三年以外。
- 機會成本。自建的建置期可能要一到三個月，這段期間沒有服務可用；API 方案當天就能上線。

對需要快速驗證的專案，這段時間比錢貴。

- 品質差距。若地端模型在關鍵任務上的正確率低 8 個百分點，你省下的錢是否值得？這必須用第 36 章的評測數據來回答，不能用感覺。

把這三項加回去之後，結論往往會反轉，或者變成「混合方案」：敏感資料走地端小模型，一般查詢走 API，並在兩者之間設一個路由器（第 33 章的程式 33B 就是在做這件事）。這才是多數臺灣單位的實際最佳解。

算帳的紀律

本書要求你在任何路線選擇的文件中，都必須同時列出：一次性投入、月變動成本、人力成本、建置期長度、品質差距、以及至少一個你沒有把握的假設。最後那一項最重要——把不確定的地方寫出來，比假裝什麼都算得準要專業得多。

1.4.5 一個給臺灣多數場域的務實建議

如果你現在必須立刻做決定，而且不確定該選哪一條，以下這個順序在多數臺灣的教育與中小企業場域是合理的起點：

1. 先用封閉 API 加上檢索，在兩週內做出可以給人試用的原型，目的是驗證「這個需求到底存不存在」。此時不要用真實敏感資料。
2. 同時建立你的評測集（第 36 章）。這一步最常被跳過，也最致命——沒有評測集，你之後所有的改進都只是感覺。
3. 原型被證明有價值之後，改用開放權重模型地端部署，比對品質差距。差距若在可接受範圍內，就留在地端。
4. 若品質不足，再進入適配階段：先試 LoRA（第 24 章），不夠再試持續預訓練（第 22 章）。
5. 自建預訓練請當成研究題目而不是產品路徑，除非你有國家級的算力與明確的主權需求。

1.4.6 三個臺灣單位的實際選擇（迷你案例）

以下三個案例經過改寫與去識別，但決策過程與結果都貼近真實。請在讀完之後回頭對照 1.4.2 的六個維度，看看你是否會做出相同的判斷。

案例一：中部某高中的作文回饋助理。需求是替國文老師產生初步的作文評語草稿，最後由老師修改後發給學生。資料是學生作品，屬於未成年人的個人資料。團隊最初打算用商業 API，但在法遵討論時發現：把學生作文原文送出去，即使供應商聲明不用於訓練，仍難以向家長說明。最後選擇單張 GPU 地端部署開放權重模型，品質略遜但可接受，且完全不必處理跨境傳輸的說明義務。這個案例的關鍵決策點是隱私，不是成本也不是能力——當硬約束存在時，其他維度的比較根本不會發生。

案例二：南部某精密機械廠的技術文件查詢。需求是讓現場人員用口語問「這台機器的主軸潤滑週期是多久」。團隊一開始想微調模型，投入兩個月後發現效果不佳——因為問題的答案就明明白白寫在文件裡，模型需要的不是「記住」而是「查到」。改成檢索增強之後兩週上線，正確率大幅提升，而且文件改版時不必重新訓練。這個案例的教訓是：先判斷任務屬於知識型還是能力型。知識型任務用檢索，能力型任務才需要微調，第 22 與 30 章會給出明確的判準。

案例三：某縣市政府的公文用語校對。需求是把承辦人擬的公文草稿檢查一遍，標出不符合公文格式與臺灣用語規範的地方。這個任務量大、單次價值低、且完全不涉及敏感判斷，因此選擇了成本最低的路線：小型開放權重模型加上規則式檢查表，模型只負責語意層面的建議，格式、稱謂、用語則由規則處理。這個案例提醒我們：不要把規則能解決的事丟給模型，那既貴又不可靠，而且無法解釋為什麼判定錯誤。

從三個案例抽出的三條判準

第一，先問這是知識型任務還是能力型任務——前者用檢索，後者才微調。第二，先問規則能不能解決其中一部分——能的話就交給規則。第三，隱私與法遵常常是唯一真正的硬約束，其他維度都可以取捨，這一項不行。這三條判準會分別在第 22、29 與 37 章被完整展開。

1.5 大模型生命週期：七個階段與一個迴圈

使用者看到的是一個輸入框和一段回覆。這一節要做的事，是把那個輸入框往回拆，一路拆到最上游。這張生命週期圖會在往後四十章反覆出現，本書的十篇結構基本上就是照著它排的。

1.5.1 七個階段

階段	關鍵決策	產出物	主要角色	章節
一、需求定義	使用者是誰、任務是什麼、什麼算失敗、誰負責	需求規格書、成功指標、不可接受風險清單	領域專家 + 工程師	1、40
二、資料與語料	資料從哪來、授權是什麼、個資怎麼處理、怎麼清理	語料清冊、Data Card、清理管線、tokenizer	資料工程 + 法務	6–9
三、預訓練	模型多大、訓練多久、算力哪裡來、怎麼避免發散	基礎模型權重、訓練日誌、實驗報告	訓練工程師	13–21
四、後訓練	指令資料怎麼寫、用 LoRA 還是全參數、怎麼對齊	指令模型、LoRA 權重、安全規範	模型工程師 + 標註團隊	22–27

階段	關鍵決策	產出物	主要角色	章節
五、評測	評什麼、誰來評、怎麼統計、題庫會不會外洩	評測集、盲評結果、紅隊報告	評測團隊 + 領域專家	36
六、部署	量化到幾位元、延遲目標多少、SLO 是什麼、怎麼回滾	服務端點、監控儀表板、維運手冊	系統 / MLOps 工程師	33、35
七、治理	誰能改、出事怎麼辦、要不要通報、多久稽核一次	治理文件、稽核軌跡、AI 影響評估	主管 + 法遵 + 工程	37

1.5.2 它不是一條直線，是一個迴圈

把上表當成瀑布式流程是最常見的錯誤。真實的專案長這樣：你在第五階段（評測）發現模型講不好臺灣的專有名詞，於是回到第二階段補語料、回到第四階段重新微調；你在第七階段（治理）收到一則資料退出請求，於是回到第二階段重整資料、可能還得回到第三階段重訓。

這個迴圈性質有一個直接的工程意涵：每個階段的產出物都必須是可版本化、可追溯、可重跑的。這就是為什麼第 4 章要花整整一章講可重現環境，為什麼第 8 章要建立 provenance ledger，為什麼第 16 章要求你記錄環境指紋與程式碼雜湊。這些東西在你第一次跑實驗時感覺很囉嗦，在你第三十次回頭找「當初到底是哪一版」時會救你一命。

1.5.3 成本分布：時間都花在哪裡

學生對大模型專案的想像通常是「大部分時間在訓練模型」。真實的時間分布比較接近下面這樣（依專案類型會有出入，但相對關係相當穩定）：

工作項目	時間占比	為什麼比想像中多
需求釐清與定義失敗邊界	約 10%	領域專家與工程師對「正確」的定義往往不一致，必須談到具體案例
資料取得、授權確認與清理	約 40%	取得管道、授權判讀、格式轉換、去識別、品質過濾、去重，每一步都會卡
建立評測集與評測流程	約 15%	出題、審題、標註、統計設計，而且必須防止外洩與污染
模型訓練與適配	約 15%	單次訓練可能只要幾小時，但要跑很多次才會對
部署、監控與維運	約 15%	延遲、成本、權限、回滾、告警，上線後才是長期戰

工作項目	時間占比	為什麼比想像中多
文件與治理	約 5%	通常被壓縮到最後才做，這是錯的，應該同步進行

請把這張表記牢。如果你的專題規劃書把 80% 的時間放在訓練，30% 的時間放在資料，那份規劃書幾乎一定會失敗。第 40 章的總整專題會強制你依照這個比例分配工時。

1.5.4 誰來做：一個大模型專案的角色分工

學生做專題時常常一個人扛下全部工作，這在學習階段是好事，但你必須知道真實專案是怎麼分工的，否則你會誤以為某些工作不存在。下表列出一個中型專案的典型角色，以及在大學專題中通常由誰兼任。

角色	負責什麼	最常見的失職	本書章節
領域專家	定義什麼是正確答案、什麼是不可接受的失敗、審核評測題目	只在期初出現一次，之後不參與	1、36、40
資料工程師	取得、清理、去識別、去重、版本化語料	沒有記錄來源，事後無法回答資料哪裡來	6–9
模型工程師	選底模、適配、微調、對齊、實驗紀錄	沒有 baseline 就開始調參	13–27
評測負責人	出題、盲評設計、統計、防止污染	用訓練資料裡的題目去測，量到的是背誦	36
系統 / MLOps	部署、監控、成本、回滾、SLO	沒有回滾方案就上線	33、35
法遵與倫理	授權判讀、個資、風險評估、通報流程	在系統快上線時才被找來，來不及改	8、37

在大學專題的四人小組中，一個可行的分配是：一人主責資料與法遵、一人主責模型、一人主責系統與部署、一人主責評測與文件，領域專家由指導老師或合作單位擔任。請注意「評測與文件」不是打雜——那是最能決定專題成敗的角色，第 40 章的評分規準中有超過三分之一的配分落在這個人身上。

1.5.5 一個 18 週專題的真實時程長什麼樣

把 1.5.3 的成本分布落到具體的一個學期，會長成下面這樣。這份時程是給選修專題的學生看的，也是給第 40 章總整專題的預演。請注意第 1 到第 5 週完全沒有碰模型——這不是浪費時間，這是專案

能不能成功的分水嶺。

週次	工作	產出物	常見的偏離
1-2 週	需求釐清、與領域專家訪談、定義不可接受的失敗	需求規格書 v0.1	跳過訪談，自己想像使用者要什麼
3-5 週	資料盤點、授權確認、取得管道建立、初步清理	語料清冊、Data Card 草案	先抓資料再想授權，事後無法補救
4-6 週	(與上並行) 建立評測集第一版並找人審題	30-60 題評測集	整學期都沒有評測集
6-8 週	baseline：用現成模型加提示與檢索，量測起點	baseline 報告	沒有 baseline 就開始微調
9-13 週	適配：LoRA 或持續預訓練，多輪實驗與消融	模型權重、實驗紀錄	只跑一個 seed 就下結論
12-15 週	部署：量化、服務化、監控、成本量測	可存取的服務與 SLO	在期末前一週才第一次部署
14-16 週	安全：紅隊測試、拒答檢查、權限驗證	紅隊報告	完全省略
16-18 週	文件、復盤、發表、開源或技轉決策	Model Card、技術報告、簡報	報告在最後三天趕出來

幾個時程上的經驗法則：資料階段的實際耗時通常是你估計的兩倍；第一次部署一定會失敗，所以要提前四週而不是提前一週；評測集一旦建立就不要再改，改了之前的數據就不能比較了。

1.6 臺灣本土大模型的必要性

這一節是全書的立論基礎。我要說服你的不是「愛臺灣所以要做臺灣模型」，而是「有五個可量測、可查證的技術理由，讓通用國際模型在臺灣場域會系統性地表現不足」。

1.6.1 理由一：用語不是小事

中文不是一種語言，是一組在不同社會裡各自演化的書面系統。臺灣的資訊科技用語與其他中文區的差異，遠比一般人以為的大。當模型的訓練語料以其他中文區為主時，它會系統性地把不屬於這裡的詞當成預設值。

臺灣用語	其他中文區常見用語	臺灣用語	其他中文區常見用語
軟體 / 硬體	軟件 / 硬件	資料 / 資料庫	數據 / 數據庫
程式	程序	演算法	算法
網路	網絡	最佳化	優化
伺服器	服務器	記憶體	內存
介面	接口	硬碟	硬盤
專案	項目	影片	視頻
品質	質量	列印 / 印表機	打印 / 打印機
滑鼠	鼠標	解析度	分辨率
光碟	光盤	行動裝置	移動設備

在課堂上這也許只是彆扭；在公文、契約、規格書、法律文件裡，用錯詞就是實質的錯誤。而且這件事無法靠「請用臺灣用語回答」這句提示徹底解決——你可以在第 1A 實作中親自量測，提示能改善的比例有限，而且在長文本中會逐漸漂移回去。真正的解法在語料與後訓練，也就是第 22 與 23 章。

1.6.2 理由二：制度知識無法用翻譯補上

學測、統測、分科測驗、繁星推薦、免試入學超額比序、勞保與勞退的差別、健保的部分負擔、統一發票的載具、戶籍謄本與戶口名簿、統一編號與身分證字號的檢核規則、都市計畫的使用分區、勞基法的工時規範——這些不是詞彙問題，是制度問題。一個不曾大量閱讀臺灣文本的模型，不會有這些知識，而且它會用鄰近地區的類似制度去填補空白，填得非常順、非常自信、非常錯。

本章開頭那位老師遇到的正是這一類。而這一類錯誤最危險的地方在於：它看起來完全合理，只有真正懂的人才看得出來。這也意味著，臺灣本土評測集（第 36 章）不是加分項，是必需品。

1.6.3 理由三：token 稅是實實在在的成本

大多數主流 tokenizer 是以英文為中心設計的。同一段語意，英文可能只需要十幾個詞元，繁體中文卻需要兩三倍。這件事會沿著三條路徑轉成真金白銀的成本：

- API 費用按詞元計價，你付的錢比英文使用者多一到兩倍。
- 上下文長度以詞元計算，同樣的 32K 上下文，你能塞進去的實質內容比英文少一半以上。

- 訓練與推論的計算量隨序列長度增加，中文使用者的每一次互動都比較貴、比較慢。

第 7 章會讓你親手訓練 tokenizer 並量測這個比例，第 7C 實作會直接把 token fertility 換算成新臺幣。這是一個很好的教學時刻：學生會第一次感受到，語言的技術基礎設施如果不是為你設計的，代價會由你長期承擔。

1.6.4 理由四：資料主權與可控性

當一個學校、一家醫院、一間工廠把它的核心資料送進境外的模型服務，它同時交出了三件事：資料本身的控制權、服務可用性的控制權、以及模型行為變更的控制權。第三項最常被忽略——你依賴的模型可能在某天被更新、被下架、被調整安全策略，而你的系統會在毫無預警的情況下改變行為。

這不是要主張所有東西都必須地端。合理的做法是分級：把資料依敏感度分成幾層，最敏感的一層一定地端、中間層可用地端開放權重模型、最不敏感的一層可以用外部 API。第 37 章會教你怎麼把這個分級寫成可稽核的政策。

1.6.5 理由五：產業知識在臺灣人的手上

半導體製程、精密機械、工具機、電子代工、農漁業、醫療照護——這些領域的實務知識大量存在於臺灣的工單、SOP、技術報告、老師傅的口述裡，而且多半沒有上網。這些資料不會出現在任何國際模型的訓練語料中，任何人想做出這些領域真正有用的模型，都必須從臺灣的資料開始。

這是一個機會，也是一個責任。機會在於：這是別人做不到的差異化。責任在於：這些資料涉及營業秘密、個資與勞動關係，處理不當會造成真實傷害。第 8 章與第 37 章會把這件事講清楚。

1.6.6 把開場那個錯誤完整拆解一次

回到本章開頭：老師問「我們學校今年免試入學的超額比序怎麼算」，模型給了一套邏輯完整但屬於別的縣市的答案。這個錯誤看起來只有一個，實際上是四層問題疊在一起。學會拆解它，你就學會了本書的診斷方法。

層次	發生了什麼	根本成因	對應的修復手段
第一層	模型不知道「這所學校」是哪一所	提示中沒有提供學校身分，模型也無從得知	情境工程：把使用者身分帶入提示（第 29 章）
第二層	模型不知道各縣市規則不同	訓練語料中臺灣升學制度的細節密度不足	補臺灣教育語料（第 9、22 章）
第三層	模型用最常見的版本填補空白	訓練目標追求合理而非真實；缺	拒答訓練與不確定性表達

層次	發生了什麼	根本成因	對應的修復手段
		乏拒答習慣	(第 25 章)
第四層	答案沒有出處，使用者無從查證	系統設計沒有要求引用	RAG 強制引用與來源顯示 (第 30、33 章)

注意這四層的修復成本差異極大。第一層改個提示就好，一小時內完成；第四層要建 RAG 系統，一到兩週；第二層要補語料再微調，數週到數月；第三層要重新設計後訓練資料，最貴。

工程判斷的關鍵在於：先做便宜的那幾層，量測還剩多少錯誤，再決定要不要往下走。很多團隊一開始就跳去做最貴的第三層（「我們來訓練自己的模型」），結果花了三個月才發現，其實加上第一層與第四層就已經解決了八成的問題。本書之所以把 RAG 排在第 30 章、把持續預訓練排在第 22 章，順序上看起來顛倒，正是因為在真實專案中你應該先試便宜的方案。

一個必須避免的錯誤結論

本節不是在說「國際模型很爛，我們自己做就好」。事實上本書大部分的實作都建立在開放權重的國際模型之上——那是務實且正確的做法。本節要說的是：把國際模型當成起點，而不是終點；差異化的價值在於我們往上加的那一層，而那一層只有我們做得出來。

1.7 臺灣的工程優勢與真實限制

談優勢容易變成口號，所以這一節同時談限制。學生必須看清楚我們站在哪一格，才能決定往哪裡走。

面向	臺灣的位置	對學生的意涵
半導體與硬體製造	先進製程、封裝測試與 AI 伺服器組裝在全球供應鏈中占關鍵位置	硬體與系統整合的實務知識容易取得；第 21 章會帶你盤點
邊緣運算與嵌入式	長期的工控、車電、機器人與嵌入式系統累積	小模型與邊緣部署是我們的強項戰場，第 26、35 章
製造場域資料	大量高價值、未數位化或未公開的產業知識	差異化的來源；但取得需要產學信任與資料治理，第 8 章
算力	學術與中小企業可取得的 GPU 資源相對有限	必須擅長在受限算力下做事：LoRA、蒸餾、量化，第 24、26、35 章
語料	繁體中文高品質語料規模遠小於英文與簡體中文	語料工程本身就是研究題目，第 6–9 章

面向	臺灣的位置	對學生的意涵
人才	工程訓練紮實，但頂尖 AI 人才競爭激烈且流動性高	教材必須讓學生真的會做，而不是只會用，這是本書的設計前提
資金與規模	單一機構難以支撐從零預訓練的資本支出	合作與開放共享比各自為政更有效，第 40 章鼓勵開源交付

1.7.1 三個可行的差異化切入點

光看優劣勢表容易停在感嘆。以下三個切入點是本書認為臺灣的大學實驗室與中小企業真正能做出成果的方向，每一個都對應到書中的具體章節，你在選擇貫穿式案例時可以參考。

1. 在地知識的深度而非廣度。不要試圖做一個什麼都會的臺灣模型，那需要的算力我們沒有。要做的是在某一個垂直領域做到別人做不到的深——工具機、模具、水產養殖、長照、地方文史都可以。關鍵在於資料取得的信任關係，那是外國團隊拿不到的（第 8、22、31 章）。
2. 邊緣與地端的工程能力。臺灣有大量不能上雲、不能斷線、必須低功耗的場域。把 3B 以下的模型調到在工控機上穩定跑一年不當機，這件事的技術含量與商業價值都被嚴重低估（第 26、35 章）。
3. 評測與治理的公共財。臺灣目前最缺的不是模型，是可信的臺灣評測集。一個做得夠嚴謹、夠公開、有審題紀錄與污染控制的臺灣題庫，它的影響力會超過任何單一模型，而且大學是最適合做這件事的機構（第 36 章）。

第三點值得多說一句。評測看起來不酷、不會登上新聞，但它決定了整個生態系能不能誠實地衡量進步。如果沒有人做這件事，所有人都只能靠感覺與行銷話術做決策。本書把第 36 章寫得比一般教材厚很多，就是這個原因。

把這張表濃縮成一句話：臺灣不太可能在「訓練最大的基礎模型」這件事上取得優勢，但完全有可能在「把基礎模型變成這裡真正能用的系統」這件事上做到世界級。本書的四十章就是照這個判斷排出來的——第三、四、五篇讓你懂原理與底層，第六篇之後的每一章都在教你怎麼把它變成這裡能用的東西。

1.8 本書地圖與學習建議

1.8.1 十篇四十章的能力遞進

本書分成十篇。第一到第五篇建立底層能力：你會親手寫出 tokenizer、注意力、Transformer 與一

個可以生成繁體中文的 MiniFormosaGPT，並理解算力與分散式系統如何決定上限。第六篇之後轉向本土化與系統整合：適配、對齊、推理、檢索、智慧體、應用開發、部署、評測、治理、多模態，最後以總整專題收束。

你不需要照順序讀完才能開始動手。事實上我建議你在讀完本章之後，立刻做兩件事：一是完成三個程式實作，二是選定一個你要跟著走完四十章的貫穿式案例（見規劃書的十二類案例）。有了具體的案例，後面每一章的內容都會自動找到落點。

1.8.2 四級硬體：沒有 GPU 也能學

層級	設備	本書中可完成的事	本章實作需求
A 級	一般筆電 (CPU)	tokenizer、注意力、MiniFormosaGPT tiny、API 應用、RAG、量化小模型推論	1A、1B、1C 全部可做
B 級	單張 GPU (8–24GB)	QLoRA 微調、7B 量化推論、Agent 與 VLM 實作、蒸餾	可額外加測地端模型
C 級	實驗室多卡叢集	持續預訓練、分散式訓練、尺度研究	—
D 級	國家級算力	從零預訓練基礎模型	—

1.8.3 三個常見的學習誤區

1. 只學提示工程。提示是推論期的技巧，天花板很低。真正的差異化在資料與後訓練，那是第二篇與第六篇的內容。
2. 跳過數學直接抄程式。第 3 章的張量形狀推理不難，但跳過它的人在第 11 章一定會卡住，而且卡住的時候不知道自己為什麼卡住。
3. 不建評測集就開始調整。沒有評測集，你所有的「感覺變好了」都不可信。第 36 章雖然排在後面，但評測集應該在專案第一週就開始建。

1.8.4 怎麼使用本書的三種讀者

如果你是大學生並修習這門課，請照章順序讀，每章的程式實作至少完成兩個，並從第一週就維護你的需求規格書與評測集。

如果你是高中生或自學者，建議路徑是第 1–14 章（把 MiniFormosaGPT 做出來）、第 21 章（了解算力現實）、第 23–24 章（學會微調）、第 29–30 章（提示與 RAG）、第 32–34 章（Agent 與應用

開發)、第 38–40 章(多模態與專題)。第 15–20 章與第 25–28 章可以先跳過數學推導只讀圖解，等你真的需要時再回來。

如果你是在職工程師或教師，建議從第 1、4、8 章建立基本判斷力，接著直接跳到你場域對應的章節，最後務必回頭讀第 36 與 37 章——評測與治理是最容易被跳過、也最容易在上線後付出代價的兩章。

1.9 本章名詞中英對照

本書採用臺灣慣用譯名，必要時保留英文原詞。完整術語表見附錄 K。

中文	英文	中文	英文
大語言模型	Large Language Model	基礎模型	Foundation Model
預訓練 / 後訓練	Pretraining / Post-training	指令微調	Instruction Tuning
詞元 / 分詞器	Token / Tokenizer	詞嵌入	Embedding
注意力機制	Attention	因果遮罩	Causal Mask
幻覺	Hallucination	對齊	Alignment
檢索增強生成	Retrieval-Augmented Generation	智慧體	Agent
開放權重	Open Weights	量化	Quantization
尺度定律	Scaling Law	困惑度	Perplexity
災難性遺忘	Catastrophic Forgetting	模型編輯	Model Editing
資料溯源	Provenance	紅隊測試	Red Teaming

程式實作

本章三個實作都不需要 GPU，A 級筆電即可完成。程式碼、測試資料與預期輸出放在課程倉庫的 ch01/ 目錄下，執行前請先依第 4 章（或附錄 D）建立環境。

程式 1A 三模型繁中能力對照實驗

目標：用同一組臺灣情境提示測試三個模型，量測正確性、臺灣用語比例、token 數、延遲與成本，並輸出可比較的表格與雷達圖。這是在本書中的第一次「量測」，之後每一章都會用到同樣的紀律：先定義指標，再收集數據，最後才下結論。

ch01/lab1a_compare_models.py (節錄)

```
import time, json, csv, statistics
from pathlib import Path

# 臺灣用語檢核詞表：出現「其他中文區用語」即記為一次用語漂移
TW_TERMS = {
    "軟體": "軟件", "程式": "程序", "網路": "網絡", "資料": "數據",
    "伺服器": "服務器", "記憶體": "內存", "介面": "接口", "專案": "項目",
    "品質": "質量", "影片": "視頻", "演算法": "算法", "最佳化": "優化",
}

def term_drift(text: str):
    """回傳（漂移次數，命中的非臺灣用語清單）"""
    hits = [zh for tw, zh in TW_TERMS.items() if zh in text]
    return len(hits), hits

def run_case(client, model: str, prompt: str):
    t0 = time.perf_counter()
    resp = client.chat(model=model, prompt=prompt) # 由 adapters.py 統一介面
    latency = time.perf_counter() - t0
    drift, hits = term_drift(resp.text)
    return {
        "model": model,
        "latency_s": round(latency, 3),
        "in_tokens": resp.in_tokens,
        "out_tokens": resp.out_tokens,
        "cost_twd": round(resp.cost_twd, 4),
        "term_drift": drift,
        "drift_hits": hits,
        "text": resp.text,
    }

def main(models, prompts, out="results/ch01_compare.csv"):
    rows = []
    for m in models:
        for p in prompts:
            rows.append(run_case(CLIENT, m, p["prompt"]) | {"case_id": p["id"]})
    Path(out).parent.mkdir(parents=True, exist_ok=True)
    with open(out, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=rows[0].keys())
        w.writeheader(); w.writerows(rows)
    # 摘要：每個模型的中位延遲、平均輸出詞元、用語漂移率
    for m in models:
        sub = [r for r in rows if r["model"] == m]
        print(m,
              "延遲中位數", statistics.median(r["latency_s"] for r in sub),
```

```
"平均輸出 token", statistics.mean(r["out_tokens"] for r in sub),
"用語漂移率", sum(1 for r in sub if r["term_drift"] > 0) / len(sub))
```

提示集請至少包含六類，每類三題以上。下表列出每一類的設計意圖與範例，你必須自己改寫成你所在縣市、學校與產業的版本——直接照抄會失去這個實驗的意義。

類別	範例題型（請自行改寫）	設計意圖	主要觀察
制度題	說明學測、統測與分科測驗的差異與適用對象	測試臺灣教育制度知識密度	制度誤植
地理行政題	列出雲林縣所有的鄉鎮市並指出縣治所在	測試臺灣地理與行政區知識	幻覺、細節錯誤
法規誘發題	請說明某法規第 N 條的內容（N 刻意超出實際條數）	主動誘發幻覺	是否編造、是否更正
產業術語題	解釋工具機的三軸與五軸差異，以及在臺灣的典型應用	測試產業知識與用語	用語漂移、術語錯誤
本土語言題	把一段華語句子寫成臺語漢字與臺羅拼音	測試低資源語言能力	拼寫錯誤、直接放棄
不可答題	詢問一個確定不存在的獎學金或校內單位	測試誠實拒答	拒答失敗

- 正確性請用人工標註，三人獨立評分後取多數；不要用模型評模型，那是第 36 章才會處理的進階議題，現在做會引入你還無法診斷的偏誤。
- 延遲請至少重複五次取中位數，並記錄測試時間與網路狀況。首次呼叫通常較慢，應該捨棄。
- 成本請換算成新臺幣，並同時記錄「回答同一題所需的 token 數」。你會發現有些模型雖然單價便宜，但因為輸出冗長，總成本反而較高。

延伸挑戰（擇一完成）：

- 把同一組提示各跑五次，計算每個模型輸出的自我一致性（五次回答彼此相同或等價的比例）。你會發現一致性低的題目，往往正是模型不確定的題目——這是第 27 章多樣本方法的直覺來源。
- 在提示前面加上一句「請使用臺灣繁體中文的慣用詞彙」，重測用語漂移率。記錄改善幅度，並特別觀察長回答的後半段是否漂移回去。這個實驗會讓你親身理解提示工程的天花板在哪裡。
- 把六類提示的正確率分開統計，畫出每個模型的能力雷達圖。你會看到「總分接近」的兩個模型，可能有完全不同的形狀——這正是為什麼單一總分是危險的。

程式 1B LLM 生命週期檢查器

目標：把 1.5 節那張七階段表變成一支可執行的工具。輸入一份用 YAML 描述的應用需求，輸出這個專案在每個階段必須完成的待辦事項、缺漏的欄位，以及風險等級。這支工具你會一路用到第 40 章的專題驗收。

ch01/spec_example.yaml

```
project: 校務與課程問答助理
owner: 智慧科技學院 AI 實驗室
users:
  - 在校學生
  - 導師與行政人員
tasks:
  - 查詢校規與學則
  - 查詢課程與選課規定
  - 查詢獎助學金申請期限
language: zh-TW
data:
  sources: [學則 PDF, 課程大綱, 行政公告]
  contains_pii: false
  license_checked: false      # 尚未確認 -> 檢查器會警告
unacceptable_failures:
  - 講錯申請截止日期
  - 捏造不存在的條號
metrics:
  - name: 引用正確率
    target: 0.98
  - name: 拒答率（無資料時）
    target: 0.90
deployment:
  location: on_premise
  expected_qps: 3
```

ch01/lab1b_lifecycle_check.py (節錄)

```
REQUIRED = {
    "需求定義": ["project", "users", "tasks", "unacceptable_failures", "metrics"],
    "資料與語料": ["data.sources", "data.license_checked", "data.contains_pii"],
    "部署": ["deployment.location", "deployment.expected_qps"],
}

RISK_RULES = [
    (lambda s: s["data"]["contains_pii"] and s["deployment"]["location"] != "on_premise",
     "高", "含個資卻規劃外部部署，須先完成法遵評估（第 8、37 章）"),
    (lambda s: not s["data"]["license_checked"],
     "高", "資料授權尚未確認，不得進入訓練階段（第 8 章）"),
    (lambda s: not s.get("unacceptable_failures"),
     "高", "未定義不可接受的失敗，無法設計評測與拒答策略（第 25、36 章）"),
    (lambda s: len(s.get("metrics", [])) < 2,
     "中", "指標少於兩項，建議至少涵蓋品質與安全兩個面向"),
]

def check(spec: dict):
    missing = [(stage, f) for stage, fields in REQUIRED.items()
               for f in fields if not _get(spec, f)]
    risks = [(lvl, msg) for rule, lvl, msg in RISK_RULES if rule(spec)]
    return missing, risks
```

延伸挑戰：把檢查器接上 CI，讓每次提交需求規格書時自動執行；並新增一條規則——若 unacceptable_failures 中出現醫療、法律或財務建議相關字樣，就強制標記為高風險並要求領域專家簽核。

程式 1C 臺灣常識三十題快測與錯誤分類

目標：建立本書第一個迷你評測集，並學會把錯誤分類。錯誤分類比正確率更重要——正確率告訴你「有多糟」，錯誤分類告訴你「該怎麼修」。

錯誤類型	判定方式	典型例子	對應的修復手段
用語漂移	出現非臺灣慣用詞	把「程式」寫成「程序」	後訓練資料與風格對齊（第 22–23 章）
制度誤植	套用其他地區制度	把升學管道講成其他學制	補臺灣語料 + 檢索（第 9、30 章）
幻覺	編造可查證但不存在的內容	虛構法條條號或校名	RAG 強制引用（第 30 章）
時效錯誤	內容曾經正確但已過期	引用已修正的舊條文	知識更新與模型編輯（第 28 章）
拒答失敗	無資料時仍給出答案	對明知無解的題目硬答	拒答訓練（第 25 章）
過度拒答	有資料卻拒絕回答	把一般校務問題當成敏感題	安全規範校準（第 25、36 章）

ch01/lab1c_taiwan_quiz.py (節錄)

```
# quiz.jsonl 每行一題：
# {"id": "TW-007", "q": "...", "gold": "...", "type": "制度", "answerable": true}

def classify_error(pred: str, item: dict) -> str:
    if not item["answerable"]:
        return "OK" if is_refusal(pred) else "拒答失敗"
    if is_refusal(pred):
        return "過度拒答"
    if match_gold(pred, item["gold"]):
        return "OK"
    if term_drift(pred)[0] > 0:
        return "用語漂移"
    if cites_nonexistent(pred):
        return "幻覺"
    if matches_outdated(pred, item.get("outdated_gold")):
        return "時效錯誤"
    return "制度誤植"
```

```
def report(results):
    from collections import Counter
    c = Counter(r["error"] for r in results)
    total = len(results)
    for k, v in c.most_common():
        print(f"{k:8s} {v:3d} {v/total:6.1%}")
```

三十題的分布建議如下，並在 jsonl 中以 type 欄位標記，這樣你才能算出「哪一類最弱」而不只是一個總分。

類別	題數	出題來源建議	標準答案的出處要求
教育制度	6	教育部與各校公開簡章、招生規定	須附文件名稱、版本與頁碼
地理與行政	5	內政部與地方政府官方網站	須附網址與查核日期
法規與公文	5	全國法規資料庫、機關公告	須附法規名稱、條號與修正日期
產業與技術	6	公協會資料、廠商規格書、技術報告	須附來源文件
本土語言與文化	4	教育部辭典、地方文史資料	須附辭典條目
不可答 (誠實測試)	4	自行設計，確認確實不存在	須說明為何不存在

出題原則

三十題請自己出，不要從網路上抓現成題目——現成題目很可能已經在模型的訓練資料裡，你量到的會是背誦而不是能力。這個問題在第 36 章叫做 benchmark 污染，是評測工程最核心的難題之一。另外，題目必須有明確、可查證的標準答案與出處，出處要寫進 jsonl。

系統案例：本土模型與通用模型的臺灣情境對照

本章的系統案例不寫程式，而是設計一場實驗，並把它寫成一份可被同儕檢驗的報告。你要比較的是：臺灣本土訓練或適配的模式（例如 TAIDE、Taiwan LLM 一類），與國際通用開放權重模型，在臺灣公文、校務規章與電子製造詞彙上的表現差異。

實驗設計

1. 受測對象：至少一個臺灣本土模型、一個國際開放權重模型（規模相近）、一個商業封閉模型作為上界參考。記錄每個模型的版本、取得日期與授權。
2. 任務集：三個領域各 20 題，共 60 題。公文（用語、格式、法規引用）、校務（學則、選課、獎

助)、電子製造(料號、設備術語、中英混寫)。

3. 控制變因：同一組提示、同一組解碼參數(temperature 固定為 0.2)、同一位標註者順序隨機、盲評(標註者不知道哪個回答來自哪個模型)。
4. 量測指標：正確率、用語漂移率、幻覺率、平均輸出 token 數、每題成本、中位延遲。
5. 歸因分析：對每一個錯誤，判斷成因屬於「語料缺乏」「tokenizer 不利」還是「對齊不足」三者之一。這是本案例最重要的一步。

你應該會觀察到的現象(先預測，再驗證)

- 本土模型在用語與制度題上的優勢通常明顯，但在一般推理與程式題上可能落後於規模相近的國際模型。
- 規模較大的國際模型在許多題目上仍可能贏過小型本土模型——這不矛盾，這正是第 22 章要教你的：把本土適配加在更強的底模上。
- 同一題的成本差異可能達到數倍，主要來自 tokenizer 效率與輸出長度，而不是單價。

報告格式(本書所有系統案例共用)

這份格式你會用四十次，請從第一次就寫對。每一節都有它存在的理由，不要為了篇幅而刪。

章節	要寫什麼	常見錯誤	篇幅
一、問題與範圍	誰會用、解決什麼問題、明確排除什麼	寫得太大，變成無法驗收的願景	半頁
二、方法	受測對象、任務集、控制變因、指標定義	沒寫模型版本與取得日期	一頁
三、結果	表格與圖，含樣本數與變異	只放平均值，不放分布	一至二頁
四、歸因分析	每一類錯誤的成因判斷與證據	直接跳到結論，沒有歸因	一頁
五、限制	哪些結論不成立、哪些沒測到	整節省略(最嚴重的錯誤)	半頁
六、建議	下一步該做什麼，以及成本估計	建議與數據無關	半頁

請注意：在寫結論時，不可以只說「A 模型比 B 模型好」。你必須寫成「在這 60 題、這組指標、這個時間點、這個版本下，A 在制度題上高出 B 多少個百分點，信賴區間為何」。這種寫法看起來囉嗦，但它是唯一誠實的寫法，也是第 16 與 36 章會反覆訓練的表達紀律。

章末評量

一、概念題

1. 用自己的話說明「基礎模型」與「大語言模型」的差別，並各舉一個不屬於對方的例子。
2. 為什麼說「人工智慧不等於機器學習」？舉出兩個屬於 AI 但不屬於 ML 的方法，並說明它們在現代大模型系統中仍有什麼用途。
3. 解釋為什麼「下一個詞元預測」這個看似簡單的目標，可以讓模型學會翻譯與寫程式。這個說法有什麼極限？
4. 指令模型與基礎模型的行為差異是什麼？造成差異的是哪一個訓練階段？

二、推導與計算題

1. 某任務的中文提示平均 800 個詞元、回覆 600 個詞元，每日 5,000 次呼叫。若輸入單價為每千詞元 0.9 元、輸出為 3.6 元（新臺幣），計算月成本。若改用 tokenizer 效率高 35% 的模型，月成本變成多少？
2. 承上題，若地端部署需一次性投入 45 萬元硬體、每月電力與維運 1.2 萬元，計算損益兩平所需的月數（不計折舊與人力）。並說明有哪三項成本是這個計算沒有涵蓋的。
3. 某模型上下文長度為 32K 詞元。若繁體中文的 token fertility 是英文的 2.3 倍，估算中文使用者實際能放入的字數大約相當於英文的多少比例。

三、除錯題

1. 同學的 1A 實驗得到「A 模型延遲只有 B 模型的三分之一」的結論，但他每個模型只跑了一次，而且 A 模型是第二個測的。指出這個實驗至少三個設計缺陷，並說明如何修正。
2. 某系統上線後，使用者回報「模型會編造不存在的獎學金名稱」。請依 1.3.6 的檢查表判斷這屬於哪一種失效模式，並依成本由低到高排出三個處置方案。
3. 一份需求規格書的 unacceptable_failures 欄位寫著「回答不夠好」。說明為什麼這個寫法無法使用，並改寫成兩條可驗收的敘述。

四、系統設計題

1. 為某高中設計一個「課程與升學諮詢助理」。請畫出七階段生命週期，標明每階段的產出物、負責人、以及你認為最容易被跳過的一步。特別說明未成年人資料要如何處理。
2. 某中小型工具機廠希望導入維修知識助理，製程參數不得外流，預算有限，資訊人員僅一人。請從四條路線中選一條並論證，需涵蓋控制力、成本、隱私、授權、可維護性、能力上限六個維

度。

五、臺灣情境題

1. 找出三個你自己專業領域中的臺灣專有名詞或制度，測試至少兩個模型能否正確說明，記錄錯誤並依 1C 的分類法歸因。
2. 從政府公開資料平台挑一份資料集，判斷它的授權條款是否允許用於模型訓練，並寫出你的判斷依據與不確定之處（不確定的部分請明確標示需要法律確認）。

六、倫理討論題

1. 若一所學校的 AI 助理曾經給出錯誤的獎學金申請期限，導致學生錯過申請，責任應如何分配？學校、系統開發者、模型提供者、使用者各自的角色是什麼？請提出一套事前可降低此風險的機制。
2. 有人主張「臺灣語料太少，直接用其他地區的中文語料訓練就好，反正字都看得懂」。請提出至少三個技術上的反駁，以及一個你認為這個主張有道理的地方。
3. 訓練本土模型需要大量臺灣文本，其中可能包含個人的公開發言、學生作品與新聞報導。在「保存本土語言文化」與「個人對自己文字的控制權」之間，你認為應該如何取得平衡？請設計一套退出機制。

評量提示（給自學者）

- 推導題第 1 題：先算單次成本 = 輸入詞元數 ÷ 1000 × 單價 + 輸出詞元數 ÷ 1000 × 單價，再乘以每日次數與 30 天。tokenizer 效率提升 35% 意味著同樣內容的詞元數變成原本的約 74% ($1 \div 1.35$)，兩項單價都受影響。
- 推導題第 2 題：損益兩平月數 = 一次性投入 ÷ (API 月成本 - 地端月成本)。沒涵蓋的成本至少包括人力、機會成本與品質差距，另可再考慮硬體折舊與電費波動。
- 除錯題第 1 題：三個缺陷分別是樣本數不足（只跑一次）、順序效應未控制（暖機與快取）、以及沒有記錄網路狀況與時間點。修正方式是重複五次以上取中位數、隨機化順序、捨棄第一次呼叫。
- 系統設計題第 2 題：關鍵是「製程參數不得外流」直接排除封閉 API 路線，而「資訊人員僅一人」則使可維護性成為最重的權重，因此答案通常是「開放權重 + RAG，暫不微調」，而不是自建訓練。

研究延伸與版本注意事項

- 基礎模型概念的原始論述：史丹佛 CRFM 團隊 2021 年關於基礎模型機會與風險的長篇報告，建議至少讀第一章與倫理章節。
- Transformer 原始論文 (2017) 與 GPT 系列技術報告：本章只講概念，數學細節在第 11–13 章處理，建議讀完第 13 章後再回頭讀原論文。
- 尺度定律相關研究 (Kaplan 等人 2020、Hoffmann 等人 2022)：第 17 章會完整處理，本章只需知道「參數、資料、算力有最適配比」。
- 課程資源：Stanford CS336 (從零建構語言模型)、CMU 進階自然語言處理與 LLM 系統、UC Berkeley LLM Agents、李宏毅教授的生成式 AI 課程影片。前三者提供作業與講義，後者以中文講授、適合作為本書的影音補充。
- 開源教材：浙江大學《大模型基礎》(Foundations of LLMs) 可免費取得完整 PDF 與論文清單，適合作為本書第七篇 (Prompt、PEFT、模型編輯、RAG) 的延伸閱讀。
- 實作書：Raschka 的《Build a Large Language Model (From Scratch)》與其推理模型續作，本書第三、四篇的實作路線與其高度相容，可互為補充。
- 臺灣資源：TAIDE 計畫、Taiwan LLM 系列、國家高速網路與計算中心的算力申請說明、政府資料開放平台的授權條款。這些資源的狀態與條款變動較快，請於使用前查核官方頁面並記錄查核日期。

版本敏感內容的處理原則

本章提到的所有模型名稱、價格、上下文長度、授權條款都會變動。正文只保留穩定的原理與方法；具體數字一律以課程網頁的「線上補充」頁面為準，並在每次開課前重新查核。你在做任何實驗報告時，也請養成同樣的習慣：在報告開頭寫明模型版本、取得日期與查核日期。這不是形式主義，這是讓你的實驗在半年後仍然可被理解的唯一方法。

常見問題

以下是歷屆學生在第一週最常提出的問題，答案都指向後續章節，可以當成閱讀導引。

問：我沒有 GPU，是不是就不用學了？答：本章的三個實作、第 3、4、6、7、10、11 章的全部實作，以及第 29 到 34 章的大部分實作，都只需要一般筆電。真正需要 GPU 的是第 15、19、22 到 26 章的訓練實驗，而這些章節都提供了縮小版設定與可下載的中間產物，讓你在 CPU 上也能走完流程、看懂每一步。學不學得會取決於你有沒有動手，不取決於你有沒有卡。

問：既然商業模型這麼強，我們自己做的意義是什麼？答：兩個意義。第一是能力上的：你只有自己做過一次，才有能力判斷別人的模型好不好、貴不貴、值不值得信。第二是結構上的：本書 1.6 節列

出的五個理由——用語、制度、token 稅、資料主權、產業知識——不會因為國際模型變強而消失，反而會因為依賴加深而更重要。

問：我該先學哪一個框架？答：不要以框架為單位學習。本書的原則是先手寫一次再用函式庫，因為函式庫每年都在換，而你手寫過的那個注意力機制不會過期。第 4 章會建立環境，但你在第 11 到 13 章寫的程式碼幾乎不依賴任何高階框架。

問：大模型會不會取代我的工作？答：這個問題超出本書的範圍，但有一個工程上的觀察值得分享：目前大模型最不擅長的，恰恰是本書反覆強調的那些事——定義問題、判斷什麼是不可接受的失敗、確認資料來源是否合法、決定要不要相信一個結果、以及承擔責任。這門課要訓練的正是這些能力。

問：本書提到的模型與工具會不會很快就過時？答：具體的名稱會，方法不會。分詞、注意力、訓練迴圈、尺度定律、評測設計、資料治理這些東西的半衰期遠比任何一個模型長。這也是為什麼本書刻意把版本敏感的內容全部移到線上補充頁面。

教師使用建議

本章排在第 1 週，建議 2 小時講授加 1 小時實驗。以下是實際授課後整理出的節奏建議，供採用本書的教師參考。

時段	內容	互動設計	注意事項
前 20 分鐘	開場案例與 1.1 節名詞界定	請學生當場問模型一個本地問題並唸出答案	事先準備好備用案例，以免當天模型剛好答對
中間 40 分鐘	1.2 與 1.3 節，能力與失效模式	分組找出各自領域的失效案例並上台一分鐘報告	這是全章最重要的一段，不要為了趕進度壓縮
後 30 分鐘	1.4 與 1.5 節，路線與生命週期	以真實需求做路線選擇的小組辯論	避免變成品牌之爭，聚焦六個維度
最後 30 分鐘	1.6 至 1.8 節與作業說明	當場選定貫穿式案例並登記	務必當場完成選案，不要拖到第二週
實驗課 1 小時	程式 1A 與環境檢查	兩人一組，一人跑實驗一人記錄	API 金鑰請以課程共用帳號發放並設用量上限

高中授課的調整：1.1 節的表格只講前四列，1.4 與 1.5 節改為看表格與討論，不講細節；把省下的時間全部給 1.3 節的失效模式與實作 1A。高中生對「模型會自信地講錯」這件事的反應通常非常強烈，

那是最好的教學時刻。

本章小結

1. 大模型是基礎模型在文字模態上的具體化。「大」指的是參數、資料與算力三者同時放大，而三者之間有最適配比。
2. 大模型的能力來自三個層次：預訓練決定天花板、後訓練決定行為、推論期決定這一次回答的品質。臺灣的施力點主要在後兩者。
3. 五種失效模式——幻覺、知識時效、偏見、脆弱推理、過度自信——各有明確成因與對應的工程對策，不能靠「提示寫好一點」解決。
4. 四條技術路線的選擇必須同時評估控制力、成本、隱私、授權、可維護性與能力上限；「開放權重」不等於「開源」，授權條款必須逐條確認。
5. 生命週期是一個迴圈而不是一條直線，因此每個階段的產出物都必須可版本化、可追溯、可重跑。時間的大宗花在資料與評測，不是訓練。
6. 臺灣需要本土模型有五個可量測的技術理由：用語、制度知識、token 稅、資料主權、產業知識。這不是情感訴求，是工程判斷。
7. 臺灣的優勢在硬體、邊緣運算、製造場域資料與工程人才；限制在算力、語料與資金規模。合理的策略是站在開放權重模型之上做出差異化，而不是從零開始比大。

成果檢核

完成本章後，你必須繳交一份「臺灣本土大模型需求規格書 v0.1」，格式參照程式 1B 的 YAML 範本，並通過檢查器的驗證。規格書至少包含：

- 使用者：具體到角色與情境，不能只寫「一般民眾」。
- 任務清單：每項任務要能被驗收，寫成「使用者輸入什麼、系統輸出什麼、怎麼判定正確」。
- 語言與用語規範：明確指定臺灣繁體中文，並列出必須正確使用的專業術語。
- 資料邊界：哪些資料可用、哪些不可用、哪些需要進一步確認授權。
- 評測指標：至少兩項，一項品質、一項安全，並各自寫出目標值。
- 不可接受的失敗：至少兩條，且必須是可觀察、可驗收的敘述。
- 本章三個程式實作的執行結果與一頁分析。

這份規格書會在第 9、18、27、36 週的四次階段評量中反覆更新。到第 40 章的總整專題時，它會成為你專題文件的第一節。現在寫得越具體，未來三十九章你就越知道自己在做什麼。

第 2 章

從統計語言模型到 Transformer 與基礎模型

From Statistical Language Models to Transformers and Foundation Models

難度：核心 | 建議授課：第 2 週 | 硬體需求：A 級（有 GPU 可加做延伸挑戰）

叫獸開講

2017 年那篇論文的標題說「注意力就是你需要的一切」。但真正改變世界的不是那句話，而是它讓 GPU 終於可以同時算完一整句。技術史不是一串名字的排列，是一連串「舊方法撞到牆、新方法繞過去、然後撞到新的牆」的過程。這一章要教你的，是看見那些牆。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明語言模型的核心問題是什麼，以及為什麼它從一個語言學問題變成一個機率問題。
2. 推導 n-gram 模型的機率估計式，說明資料稀疏與長距離依賴為何是它無法跨越的兩道牆。
3. 解釋詞向量、RNN、LSTM 與 GRU 各自解決了前一代的什麼問題，又留下了什麼問題。
4. 說明 Encoder-Decoder 與注意力機制如何解決固定長度向量瓶頸，並指出它與自注意力的差異。
5. 從硬體平行度的角度，論證 Transformer 為何能開啟尺度化紅利，而 RNN 不能。
6. 比較 BERT、T5、GPT 三類架構的訓練目標與適用場景，並說明為何 Decoder-only 成為主流。
7. 描述從預訓練、提示學習、指令微調到推理模型、Agentic AI 與多模態的能力演進脈絡。
8. 實際量測 n-gram、Tiny RNN 與 Tiny Transformer 在同一份繁中語料上的困惑度、訓練時間與平行度差異。
9. 繪製一張「問題—方法—代價—後續影響」四欄式的技術演進因果圖，並能口頭解說。

給高中讀者的提示

本章的數學只有機率乘法與對數，看不懂公式時請直接看它旁邊的數值例子——每個公式我都配了一組真實數字。2.1 到 2.5 節請完整讀，2.6 節的三類架構只要看懂表格即可，2.7 節則是為了讓你知道後面三十八章各自要處理什麼。

2.1 語言模型到底在解什麼問題

在談任何模型之前，我們得先把問題本身講清楚。這件事聽起來多餘，但我發現很多學生一路學到 Transformer 卻始終說不出「語言模型在做什麼」——他們知道怎麼算，不知道在算什麼。

2.1.1 從「這句話像不像人話」開始

語言模型最原始的任務，是替一段文字打一個分數：這段文字有多像是這個語言的使用者會寫出來的？

「今天天氣很好」應該得到高分。「今天天氣很椅子」應該得到低分。「天氣今天好很」介於中間——語法怪但看得懂。注意這裡沒有涉及任何「意義」或「真假」的判斷，純粹是「這串文字出現的可能性」。這個看似樸素的目標，是往後所有大模型的地基。

用機率的語言來寫，我們要建模的是一段詞元序列的聯合機率：

$$P(w_1, w_2, \dots, w_T)$$

其中 w 是詞元 (token)， T 是序列長度。這個機率如果能算得準，很多事情就自然而然可以做了。

翻譯：在所有可能的中文句子裡，找出使 $P(\text{中文句} \mid \text{英文句})$ 最大的那一句。語音辨識：在所有發音相近的候選裡，挑出機率最高的文字。輸入法：預測你下一個要打的字。這三件事在深度學習之前就是

是用語言模型做的，只是模型很簡陋。

2.1.2 鏈式法則：把一個難題拆成 T 個小題

直接估計整句話的機率是不可行的——句子的可能組合是天文數字，你永遠見不到足夠的樣本。但機率的鏈式法則給了我們一條出路：

$$P(w_1, \dots, w_T) = P(w_1) \times P(w_2/w_1) \times P(w_3/w_1, w_2) \times \dots \times P(w_T/w_1, \dots, w_{T-1})$$

寫成連乘的形式：

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_{<t})$$

這個式子在數學上是恆等式，沒有做任何近似。但它在工程上是一場革命：原本要估計「整句話的機率」，現在只要估計「給定前文，下一個詞是什麼」。而後者是一個可以用大量文本訓練的分類問題——這正是本書第 5 章要展開、第 13 章要親手實作的那個目標函數。

請把這件事記牢：從 n-gram 到 GPT-5，中間隔了三十年與無數技術突破，但它們建模的目標函數是同一個。變的只是「怎麼估計 $P(w_t \mid w_{<t})$ 」這件事。n-gram 用數數，RNN 用隱藏狀態，Transformer 用注意力。目標沒變，方法一直在換。

2.1.3 為什麼要取對數

實務上我們幾乎不直接算連乘，而是取對數變成連加：

$$\log P(w_1, \dots, w_T) = \sum_{t=1}^T \log P(w_t \mid w_{<t})$$

理由很現實：假設每個詞的機率大約是 0.01，一個 100 字的句子連乘之後是 10 的負 200 次方。這個數字在浮點運算中會直接變成零，梯度也跟著消失。取對數之後變成 100 個約 -4.6 的數字相加，得到 -460 ，完全在可計算的範圍內。

這是本書第一次遇到「數值穩定性」這個主題，但絕對不是最後一次。第 3 章會講 log-sum-exp，第 15 章會講混合精度與梯度裁切，第 18 章會講浮點格式。一個共通的教訓是：數學上正確的式子，在有限精度的機器上可能完全算不出來，而處理這件事是大模型工程的日常。

2.1.4 困惑度：一個你會看一輩子的指標

有了對數機率，就可以定義困惑度 (perplexity)：

$$PPL = \exp(- (1/T) \sum_{t=1}^T \log P(w_t / w_{<t}))$$

它的直覺解釋是：模型在每一步平均而言，覺得下一個詞有多少個「等可能的候選」。困惑度 100 表示模型平均在 100 個候選中猶豫；困惑度 10 表示它把範圍縮到了 10 個。越低越好。

一個有用的參照點：如果模型完全不學習，對詞彙表中每個詞給相同機率，那麼困惑度會等於詞彙表大小。所以在一個 32,000 詞的詞彙表上，困惑度 32,000 代表完全沒學到東西，困惑度 20 代表學得相當不錯。

困惑度的最大陷阱

兩個模型的困惑度不能直接比較，除非它們用同一個 tokenizer 與同一份測試資料。原因是困惑度是「每個詞元」的平均，而不同 tokenizer 會把同一句話切成不同數量的詞元。一個把中文切得很碎的 tokenizer 會讓每個詞元變得比較好猜，困惑度自然較低——但這不代表模型比較好。第 5 章會用實驗讓你親眼看到這個陷阱，第 7 章會告訴你它對繁體中文的實際影響有多大。

2.2 n-gram 時代：數數也能做語言模型

第一代的答案簡單得令人意外：既然要算 $P(w_t | \text{前文})$ ，那就去語料庫裡數數看。

2.2.1 馬可夫假設：只看最近的 $n-1$ 個字

完整的前文太長，見過的樣本永遠不夠。n-gram 模型做了一個粗暴但有效的近似——馬可夫假設：下一個詞只跟前面 $n-1$ 個詞有關。

$$P(w_t / w_{<t}) \approx P(w_t / w_{t-n+1}, \dots, w_{t-1})$$

然後用最大概似估計，直接數頻率：

$$P(w_t / w_{t-1}) = \text{Count}(w_{t-1}, w_t) / \text{Count}(w_{t-1})$$

舉個具體例子。假設語料裡「天氣」出現了 1,000 次，其中「天氣很好」出現 180 次、「天氣不錯」出現 120 次、「天氣預報」出現 90 次。那麼 $P(\text{很好} | \text{天氣}) = 0.18$ 、 $P(\text{不錯} | \text{天氣}) = 0.12$ 、 $P(\text{預報} | \text{天氣}) = 0.09$ 。就這麼簡單，沒有任何訓練，只有計數。

名稱	條件範圍	「今天天氣很好」的分解	參數量級 (V = 詞彙表大小)
unigram	不看前文	$P(\text{今天})P(\text{天氣})P(\text{很})P(\text{好})$	V
bigram	看前 1 個詞	$P(\text{今天})P(\text{天氣} \text{今天})P(\text{很} \text{天氣})P(\text{好} \text{很})$	V^2
trigram	看前 2 個詞	$P(\text{今天})P(\text{天氣} \text{今天})P(\text{很} \text{今天, 天氣})...$	V^3
5-gram	看前 4 個詞	依此類推	V^5

2.2.2 第一道牆：資料稀疏

看那張表的最後一欄，你已經看到問題了。假設詞彙表有 50,000 個詞，那麼 bigram 有 25 億種組合，trigram 有 125 兆種。而世界上所有的中文文本加起來也遠遠不到這個數量級。

結果就是：絕大多數的組合在語料中出現次數為零。而一旦某個 n -gram 的計數是零，它的機率就是零；一旦連乘中有一項為零，整句話的機率就是零。模型會告訴你「這句話絕對不可能存在」——即使它是一句完全正常的話，只是剛好沒出現在你的語料裡。

這在中文上尤其嚴重。中文沒有天然的詞界，斷詞方式的選擇會直接改變 n -gram 的統計行為；而且中文的詞彙組合相對靈活，同樣一個意思有很多種說法，稀疏程度比英文更高。這是臺灣早期中文自然語言處理研究長期奮戰的問題之一。

2.2.3 平滑法：把機率分一點給沒見過的組合

解法統稱為平滑 (smoothing)，核心想法都是「從見過的事件那裡挪一點機率質量給沒見過的事件」。

方法	做法	問題
加一平滑 (Laplace)	所有計數都先加 1 再算比例	對大詞彙表而言分配太多機率給未見事件

方法	做法	問題
加 k 平滑	加一個小於 1 的常數 k	k 要調，而且對所有事件一視同仁
退回法 (backoff)	trigram 沒見過就退回用 bigram，再不行退回 unigram	需要設計折扣係數，否則機率總和不為 1
插值法 (interpolation)	把各階 n-gram 加權平均	權重需要在驗證集上調
Kneser-Ney	依「這個詞出現在多少種不同語境中」來估計它的通用性	實作複雜，但長期是 n-gram 時代的最強方法

Kneser-Ney 的核心洞見值得特別說明，因為它是一個很好的統計思考範例。考慮「舊金山」這個詞：它在語料中出現頻率不低，所以一般的平滑法會給它較高的先驗機率。但「金山」幾乎只出現在「舊金山」裡面——它的語境多樣性極低。Kneser-Ney 的做法是不看「這個詞出現幾次」，而看「這個詞出現在多少種不同的前文之後」。這樣一來，「金山」的通用機率就會被壓得很低，因為它只跟在「舊」後面。

2.2.4 一個完整的手算範例

光看公式不會有感覺，我們用一份極小的語料把整個流程算完一次。假設語料只有三句話（以字元為單位，方便手算）：

- 「今天天氣好」
- 「今天下雨了」
- 「明天天氣好」

先數 bigram。以「天」為前文的組合有：「天天」出現 2 次（來自「今天天氣」與「明天天氣」）、「天下」1 次、「天氣」2 次、「天了」0 次。「天」作為前文總共出現 5 次（今天天、明天天各貢獻兩次，今天下貢獻一次）。

bigram	計數	MLE 機率	加 1 平滑後 ($V = 10$)	說明
天 → 天	2	$2/5 = 0.400$	$3/15 = 0.200$	機率被稀釋
天 → 氣	2	$2/5 = 0.400$	$3/15 = 0.200$	同上
天 → 下	1	$1/5 = 0.200$	$2/15 = 0.133$	同上

bigram	計數	MLE 機率	加 1 平滑後 ($V = 10$)	說明
天 → 好	0	$0/5 = 0.000$	$1/15 = 0.067$	從零變成非零，這是平滑的目的
天 → 雨	0	$0/5 = 0.000$	$1/15 = 0.067$	同上

請注意第四列與第五列：在沒有平滑時，「天好」與「天雨」的機率都是零。這意味著只要測試句中出現這兩個組合，整句話的機率就是零，困惑度變成無限大。加 1 平滑之後它們都得到了 0.067——不高，但不是零。

同時要注意平滑的代價：「天天」的機率從 0.400 掉到 0.200，掉了一半。詞彙表越大，這個稀釋就越嚴重。在真實的 $V = 50,000$ 的情況下，加 1 平滑會把絕大部分機率質量分給從未見過的組合，這正是 2.2.3 節說它「對大詞彙表分配太多機率給未見事件」的意思。你在程式 2A 中會親手量到這件事。

2.2.5 中文的額外難題：斷詞決定了一切

前面的討論預設了「詞」是明確的單位。在英文這是對的，空白已經幫你切好了。中文沒有這個福利，而這件事會沿著整條流程往下傳染。

看這個經典的歧義例子：「研究生命起源」。切成「研究 / 生命 / 起源」和「研究生 / 命 / 起源」是兩組完全不同的詞元序列，會產生兩組完全不同的 n-gram 統計。更麻煩的是，臺灣的文本充滿了斷詞系統沒見過的东西——校名縮寫（雲科大、成大）、法規簡稱（勞基法、個資法）、產業術語（機聯網、五軸加工）、以及中英混寫（做 SPC 分析、跑 CIM 系統）。

層級	中文遇到的問題	在大模型時代的處理位置
字元層	同字多音多義、異體字、全半形	第 6 章正規化
詞層	無天然詞界、歧義切分、未登錄詞	第 7 章 subword tokenization
詞彙層	繁簡差異、臺灣慣用詞、外來語音譯不一	第 6、22 章
語料層	繁中比例低、領域文本未數位化	第 8、9 章

這張表值得記住，因為它是本書第二篇的骨架。每一層在 n-gram 時代都是一個獨立的研究問題，而在大模型時代它們被重新分配到不同的處理階段——有些被繞過，有些反而變得更關鍵。

2.2.6 第二道牆：長距離依賴

就算平滑做得再好， n -gram 還有一道無法翻越的牆。看這個句子：

「那位在雲林科技大學教機器人學、暑假還跑去日本參加學會、平常騎重機通勤的老師，昨天在課堂上宣布……」

要預測「宣布」後面接什麼，關鍵資訊在句子最前面（「老師」「課堂上」）。但一個 5-gram 模型只看得到「昨天在課堂上」這幾個字。你可以把 n 加大到 20 嗎？不行——參數量是 V 的 20 次方，資料稀疏會爆炸到完全無法估計。

這就是 n -gram 時代的死結：想看得遠，就要加大 n ；加大 n ，就會資料稀疏；緩解稀疏，就要退回小的 n 。兩道牆互相堵死，任何在這個框架內的改良都逃不出去。

真正的突破必須換一個表示方式：不要用「數過幾次」來表示語言知識，改用「一組可學習的連續參數」。這就是神經語言模型的出發點，也是下一節的主題。

n-gram 沒有死

不要因為它老就以為它沒用。今天 n -gram 仍然大量用於：資料去重與污染偵測（第 9、16 章會用 n -gram 重疊率檢查測試題有沒有洩漏到訓練集）、輸入法與拼字檢查、以及生成時的 no-repeat n -gram 約束（第 14 章）。工程上很少有技術真的被淘汰，只是被移到別的位置。

2.3 神經語言模型：從離散計數到連續向量

2.3.1 核心轉變：詞不再是符號，而是向量

n -gram 把每個詞當成一個原子符號。「貓」和「狗」之間沒有任何關係，就像「貓」和「螺絲起子」一樣毫無關係——它們只是詞彙表裡不同的編號。這意味著模型從「貓在睡覺」學到的東西，完全無法遷移到「狗在睡覺」上。

神經語言模型的第一個關鍵想法是：把每個詞映射成一個低維的實數向量（例如 300 維），讓語意相近的詞在向量空間中彼此靠近。這樣一來，模型在「貓」上學到的知識會自動有一部分適用於「狗」，因為它們的向量相近。

這件事解決的正是資料稀疏。你不需要見過每一種詞的組合，只需要見過足夠多「語意相近的組合」，模型就能泛化。這是從離散到連續、從記憶到泛化的根本轉變。

2.3.2 Word2Vec 與分布式假說

這些向量從哪裡來？答案來自語言學家 Firth 在 1957 年說過的一句話，通常被稱為分布式假說：一個詞的意義，由它周圍常出現的詞決定。

Word2Vec (2013) 把這句話變成一個訓練目標。它有兩種形式：CBOW 用周圍的詞預測中間的詞，Skip-gram 用中間的詞預測周圍的詞。訓練完成後，我們不要那個預測模型，只要中間學到的向量。最有名的展示是向量的加減運算：

$$\text{vec}(\text{國王}) - \text{vec}(\text{男人}) + \text{vec}(\text{女人}) \approx \text{vec}(\text{女王})$$

你在第 10 章的程式 10B 會用臺灣的地名與校名做同樣的實驗——例如「臺南」減「臺灣」加「日本」會落在哪裡。這個實驗的重點不是證明模型很神奇，而是讓你親身理解「語意被幾何化」這件事是真的，而且可以用尺去量。

2.3.3 靜態詞向量的極限

Word2Vec 的向量是靜態的：一個詞永遠對應同一個向量。這在一詞多義時就出問題了。

詞	不同語境	靜態向量的困境
行	銀行 / 行人 / 一行程式 / 行不行	四種完全不同的意義被壓成同一個向量
蘋果	我吃了一顆蘋果 / 蘋果發表新手機	水果與公司共用一個表示
台大	國立臺灣大學 / 臺灣大道	縮寫歧義無法由向量本身解決
重	重機 / 很重 / 重來	中文一字多音多義的問題更嚴重

解法是情境化表示 (contextualized representation)：同一個詞在不同句子裡應該得到不同的向量。ELMo (2018) 用雙向 LSTM 做到了這件事，BERT (2018) 用 Transformer 做得更好。而今天的大模型更進一步——它們的每一層都在產生情境化的表示，第 10 章與第 11 章會讓你親眼看到這個過程。

2.3.4 Bengio 的神經語言模型：把兩件事一起學

Word2Vec 之前，Bengio 等人在 2003 年就提出了神經機率語言模型。它的結構很直觀：把前 $n-1$ 個詞的向量串接起來，過一層隱藏層，再輸出詞彙表上的機率分布。

$$P(w_t / w_{\{t-n+1\}}, \dots, w_{\{t-1\}}) = \text{softmax}(U \cdot \tanh(W \cdot [e(w_{\{t-n+1\}}); \dots; e(w_{\{t-1\}})] + b) + c)$$

這個模型的精妙之處在於：詞向量 e 與預測網路 W 、 U 是一起訓練的。也就是說，模型自己會學出「什麼樣的向量表示最有利於預測下一個詞」。這比先固定一組向量再拿去用要合理得多。今天所有

大模型的 embedding 層都是這樣訓練的——它們沒有一個獨立的「詞向量訓練階段」，embedding 就是模型參數的一部分（第 10 章）。

但這個模型仍然帶著 n-gram 的原罪：它只看固定的前 $n-1$ 個詞。要看得更遠，就得把輸入層加寬，參數量隨之增加。真正跳出這個框架的是下一節的 RNN。

一個值得注意的歷史細節

Bengio 的論文發表於 2003 年，比 Word2Vec 早了十年，比 Transformer 早了十四年。它幾乎預示了後來的一切——分散式表示、神經網路預測、聯合訓練。但它在當時沒有引起足夠的重視，因為算力不足、資料不足，效果打不過調校良好的 Kneser-Ney。這是一個常見的模式：一個想法的成敗，往往不取決於它對不對，而取決於它出現的時候基礎設施是否跟得上。

2.4 RNN、LSTM 與序列建模的瓶頸

2.4.1 RNN 的基本想法：把歷史壓進一個狀態

要處理任意長度的序列，最自然的想法是逐字讀入，並維護一個「記憶」——我讀到目前為止的所有內容，都被壓縮在一個隱藏狀態向量裡。

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

每讀一個字，就把「舊記憶」與「新輸入」混合成「新記憶」。理論上， h_t 包含了從第 1 個字到第 t 個字的全部資訊，所以 RNN 理論上可以處理任意長的依賴關係。這是它相較於 n-gram 的巨大優勢。

2.4.2 梯度消失：理論上可以，實際上不行

「理論上」三個字是關鍵。訓練 RNN 需要把梯度沿著時間軸往回傳，而每回傳一步就要乘上一次權重矩陣。乘上 T 步之後，梯度大致上正比於權重矩陣的 T 次方。

結果是二選一的災難：若矩陣的特徵值小於 1，梯度會指數衰減到零——這叫梯度消失，模型學不到長距離的關聯；若大於 1，梯度會指數爆炸到無限大——這叫梯度爆炸，訓練直接發散。

用一個具體的數字感受一下：若每步的衰減係數是 0.9，傳過 50 步之後剩下 0.9 的 50 次方，約等於 0.005；傳過 100 步剩下約 0.00003。也就是說，一百個字之前的資訊對現在的梯度貢獻幾乎是零。

RNN 在理論上看得到很遠，在實際訓練中看不到。

2.4.3 LSTM 與 GRU：用閘門管理記憶

LSTM (1997) 的解法是引入一條「記憶高速公路」——細胞狀態 (cell state)，它的更新以加法為主而非乘法，梯度可以沿著它相對無損地流動。再加上三個閘門控制資訊的進出。

閘門	功能	直覺類比
遺忘閘 forget gate	決定舊記憶要保留多少	讀到新段落時，決定要不要忘掉前一段的主題
輸入閘 input gate	決定新資訊要寫入多少	這個新資訊重不重要，值不值得記
輸出閘 output gate	決定記憶中有多少要輸出給下一層	現在要不要把這段記憶拿出來用

GRU (2014) 是 LSTM 的簡化版，把三個閘門合併成兩個（更新閘與重置閘），參數更少、訓練更快，在許多任務上表現相當。這兩者在 2014 到 2017 年間是序列建模的標準配備，機器翻譯、語音辨識、文字生成幾乎都靠它們。

2.4.4 LSTM 的細胞狀態：為什麼加法比乘法安全

前一小節用表格說明了三個閘門的功能，這裡補上它為什麼有效的關鍵。LSTM 的細胞狀態更新式長這樣：

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

其中 f 是遺忘閘、 i 是輸入閘、 g 是候選值， $\odot$ 是逐元素相乘。關鍵在於這是一個加法結構：新的細胞狀態等於「舊狀態的一部分」加上「新資訊的一部分」。

對比純 RNN 的 $h_t = \tanh(W \cdot h_{t-1} + \dots)$ ，那裡舊狀態要通過一個權重矩陣與一個 $\tanh$ ，梯度回傳時每步都要乘上矩陣與 $\tanh$ 的導數（後者最大只有 1，通常遠小於 1）。在 LSTM 的加法結構下，梯度可以沿著細胞狀態這條路徑幾乎無損地流過去——只要遺忘閘接近 1，梯度就接近原樣傳遞。這就是為什麼它常被形容成一條「記憶高速公路」。

這個「用加法建立梯度捷徑」的想法，你會在第 12 章的殘差連接 (residual connection) 再看到一次。事實上殘差連接就是同一個想法在深度方向上的應用：LSTM 讓梯度沿時間軸流通，殘差連接讓梯度沿層數方向流通。認出這種跨領域重複出現的模式，是判斷一個工程師成熟度的好指標。

2.4.5 真正的死結不是梯度，是平行度

這是本節最重要的一段，也是最多教材講得不夠清楚的地方。

LSTM 確實大幅緩解了梯度消失，但它保留了 RNN 的一個結構性質：要算 h_t ，必須先算完 h_{t-1} 。

這是本質上的序列相依，無論你有多少張 GPU 都無法平行化。

GPU 的整個設計哲學是「同時做非常多次相同的運算」。一張現代 GPU 有上萬個運算核心，它擅長的是一次算完一萬件事。把 RNN 放上 GPU，就像請一支一萬人的交響樂團排隊一個一個上台獨奏——樂器再多也沒用，因為第二個人必須等第一個人演奏完。

面向	RNN / LSTM	Transformer
訓練時的時間相依	第 t 步必須等第 $t-1$ 步完成	所有位置可同時計算
訓練時間隨序列長度	線性增長，且無法用更多硬體壓縮	可用平行度換取牆鐘時間
資訊傳遞路徑長度	$O(T)$ ，兩個遠距離位置間隔 T 步	$O(1)$ ，任兩位置直接相連
單步計算複雜度	$O(d^2)$ ，與序列長度無關	$O(T \cdot d + T^2 \cdot d)$ ，序列越長越貴
記憶體占用	較小	注意力矩陣為 T^2 ，長序列時是主要負擔

請注意最後兩列：Transformer 並不是全面優於 RNN。它用「更高的計算量與記憶體」換來「可平行化」。在序列很長時，那個 T^2 項會非常昂貴——這正是第 18 章的 FlashAttention、第 20 章的線性注意力與狀態空間模型要處理的問題。技術史上很少有純粹的勝利，多半是把一種代價換成另一種。但在 2017 年那個時間點，這個交換是極度划算的：GPU 的計算能力遠比它的序列處理能力充裕，而資料與算力正在快速增長。誰能把計算轉成平行度，誰就能吃下這波紅利。這就是下一節的主題。

2.5 Encoder-Decoder、注意力與翻譯的轉捩點

2.5.1 固定長度向量的瓶頸

2014 年前後，神經機器翻譯採用的是 Encoder-Decoder 架構：一個 RNN 讀完整個來源句子，把它壓縮成一個固定長度的向量（通常稱為 context vector 或 thought vector），另一個 RNN 再從這個向量展開成目標語言的句子。

這個設計有一個顯而易見的問題：不論來源句子是五個字還是五十個字，都要壓縮進同一個大小的向量。翻譯短句時綽綽有餘，翻譯長句時資訊嚴重流失。實驗結果也確實如此——句子越長，翻譯品質下降越明顯，而且是斷崖式的。

有一個很好的類比：這就像要求一位口譯員聽完一整段五分鐘的演講、不能作任何筆記、然後憑記憶完整譯出。人類口譯員的做法是什麼？他們會作筆記，翻譯到某一句時就回頭看對應的那一段筆記。

注意力機制做的正是這件事。

2.5.2 注意力機制：翻譯到哪裡，就回頭看哪裡

Bahdanau 等人 (2014) 的想法是：不要只給解碼器一個壓縮向量，而是保留編碼器每一個位置的隱藏狀態，讓解碼器在生成每一個目標詞時，動態決定要「看」來源句子的哪些位置。

具體來說，在生成第 i 個目標詞時：

1. 計算解碼器當前狀態與來源句每個位置的相關性分數 e_{ij} 。
2. 把這些分數經過 softmax 正規化成權重 α_{ij} ，總和為 1。
3. 用這些權重對來源句的隱藏狀態做加權平均，得到這一步專屬的 context vector。
4. 用這個 context vector 加上解碼器狀態，產生第 i 個目標詞。

$$c_i = \sum_j \alpha_{ij} \cdot h_j \quad , \text{其中} \quad \alpha_{ij} = \text{softmax}_j(e_{ij})$$

這三行式子是整個 Transformer 的種子。注意其中的結構：一組分數、一次 softmax、一次加權平均。

第 11 章你要親手實作的縮放點積注意力，本質上就是這個式子換了一種計算相關性分數的方式。

注意力還帶來一個意外的紅利：可解釋性。把 α_{ij} 畫成熱圖，你可以直接看到模型在翻譯每個詞時「看向」了來源句的哪裡。在英法翻譯上，這張圖會呈現一條近似對角線，並在語序不同的地方出現交叉——那正是模型自己學會的詞對齊。第 11 章的程式 11B 會讓你用繁體中文做同樣的視覺化。

2.5.4 三種計算相關性分數的方式

注意力機制的骨架是固定的（算分數、softmax、加權平均），但「怎麼算分數」有不同做法，而這個選擇會影響效率。

方式	計算式	特性
加性注意力 (Bahdanau)	$e = v^T \cdot \tanh(W_1 \cdot s + W_2 \cdot h)$	有額外參數與非線性，表達力強但較慢
點積注意力 (Luong)	$e = s^T \cdot h$	沒有額外參數，可寫成矩陣乘法，快
縮放點積 (Transformer)	$e = (Q \cdot K^T) / \sqrt{d}$	點積再除以維度平方根，穩定 softmax 的梯度

第三種為什麼要除以維度的平方根？直覺是這樣：當向量維度 d 很大時，兩個隨機向量的點積的變異

數大致正比於 d ，數值會很大。把很大的數值餵給 softmax，會得到一個接近 one-hot 的極端分布，梯度幾乎為零，模型學不動。除以根號 d 把變異數拉回大約 1，softmax 才能保持在有梯度的區域。這是一個很好的例子，說明大模型工程裡有多少細節是為了「讓梯度活著」。第 3 章會講數值穩定性，第 11 章會讓你在實作時親自處理這個縮放因子，第 15 章則會處理更大規模下的同類問題。

2.5.5 從注意力到自注意力

上述的注意力是「跨序列」的：解碼器看編碼器。2017 年的關鍵一步是把它變成「序列看自己」——自注意力 (self-attention)。

類型	誰看誰	用途
交叉注意力 cross-attention	解碼器的每個位置看編碼器的所有位置	翻譯、摘要中的來源—目標對應
自注意力 self-attention	序列中每個位置看同一序列的所有位置	建立句內的詞與詞關係
因果自注意力 causal self-attention	每個位置只能看自己與前面的位置	自回歸生成，避免偷看答案

自注意力讓句子中的每個詞都能直接跟其他任何詞建立連結，不必經過中間的隱藏狀態層層傳遞。前面那個「那位在雲林科技大學教機器人學.....的老師，昨天在課堂上宣布」的例子，在自注意力下，「宣布」可以一步直達「老師」，路徑長度是 1 而不是 20。這就是前面那張表裡「資訊傳遞路徑長度 $O(1)$ 」的意思。

2.5.6 用一個句子走一遍自注意力

在第 11 章你會親手實作，但先用文字走一遍會讓後面輕鬆很多。以「小明把書給了小華，他很開心」這句話為例，考慮模型在處理「他」這個字時發生了什麼。

1. 每個位置的向量各自被三個矩陣投影成三份：Query（我在找什麼）、Key（我是什麼）、Value（我能提供什麼內容）。
2. 「他」的 Query 與句中每個位置的 Key 做點積，得到一組相關性分數。若模型訓練得好，「小明」與「小華」的分數會比「把」「了」高。
3. 分數除以維度平方根後過 softmax，變成一組總和為 1 的權重。假設「小華」得到 0.42、「小明」

0.31、其餘位置分掉 0.27。

4. 用這組權重對所有位置的 Value 做加權平均，得到「他」在這一層的新表示——它現在混入了「小華」與「小明」的資訊。

5. 這個過程在每一層重複。淺層可能捕捉語法關係，深層可能捕捉指代與語意關係。

注意第二步的關鍵：這組分數是模型自己學出來的，沒有人告訴它「他」通常指人。而且注意「他」到「小華」的路徑長度是 1——不需要經過中間的六個字逐步傳遞。這就是 2.4.5 節表格中「資訊傳遞路徑長度 $O(1)$ 」的具體含義。

也要注意這個機制的代價：每一個位置都要跟每一個位置算分數，所以是 $T \times T$ 次計算。句子長度加倍，這個矩陣就變成四倍大。你在程式 2C 中會直接量到這條曲線。

2.6 Transformer 與尺度化紅利

2.6.1 2017 年那篇論文做對了什麼

Transformer 的貢獻不是發明注意力——注意力已經存在三年了。它的貢獻是做了一個大膽的決定：把循環結構完全拿掉，只留注意力。

這個決定在當時是有爭議的。循環結構提供了天然的順序資訊，拿掉之後模型就不知道哪個字在前哪個字在後——所以必須額外加入位置編碼來補償（第 10 章）。而且注意力的計算量是序列長度的平方，看起來比 RNN 的線性複雜度差。

但拿掉循環之後，訓練變成可以完全平行的。這一個好處壓倒了所有缺點，理由如下表所示。

設計選擇	付出的代價	換得的好處
去掉循環結構	失去天然順序資訊，需加位置編碼	訓練時所有位置可同時計算
用注意力取代狀態傳遞	計算與記憶體為序列長度平方	任兩位置路徑長度為 1，長依賴不衰減
多頭注意力	參數與計算增加	不同 head 可捕捉不同類型的關係
殘差連接與層正規化	略增計算	可疊到數十層而不發散
前饋網路擴張	參數量大幅增加	提供逐位置的非線性表達能力

2.6.2 Transformer Block 的五個零件

第 12 章會逐一實作，這裡先給一張全景圖，讓你在讀後面的章節時知道每個零件的位置。一個標準

的 Transformer block 由五個部分組成，資料由下往上流：

零件	做什麼	為什麼需要它	詳見
層正規化	把每個位置的向量正規化	穩定訓練，讓深層網路不發散	第 12 章
多頭自注意力	讓每個位置蒐集其他位置的資訊	建立詞與詞之間的關係	第 11 章
殘差連接	把輸入直接加到輸出上	建立梯度捷徑，可疊很多層	第 12 章
前饋網路	對每個位置各自做兩層變換	提供逐位置的非線性表達能力	第 12 章
位置編碼	在輸入端注入順序資訊	注意力本身對順序無感	第 10 章

把這五個零件疊 N 次（ N 通常是 12 到 100 之間），前面加一個 embedding 層，後面加一個輸出投影，就是一個完整的 GPT。整個模型的結構單純到令人意外——這正是它容易規模化的原因，也是為什麼第 13 章你真的有辦法在一個下午之內把它寫出來。

一個對初學者的提醒

看到「只有五個零件」不要以為它簡單。難的不是寫出來，是讓它在真實規模下穩定訓練——那需要正確的初始化、學習率排程、梯度裁切、混合精度與數十個工程細節。第 15 章整章都在處理這件事。「能跑」與「能訓練好」之間隔著很遠的距離。

2.6.3 真正的突破：投入可以預期地轉成能力

平行化帶來的不只是「訓練快一點」，而是一個質變：它讓「加資料、加參數、加算力」這件事第一次變得可預測、可規劃、可投資。

在 RNN 時代，你就算有一萬張 GPU，也無法把一個必須逐字處理的訓練加速一萬倍。這意味著資本的邊際效益很快遞減，沒有人會投入十億美金去訓練一個模型。Transformer 之後，情況變了：投入與能力之間有一個穩定的、可以寫成冪次公式的關係——這就是第 17 章的尺度定律。

這是一個技術特性改變產業結構的典型案例。當「投多少錢能得到多少能力」變得可以預測，資本就會大規模進場，人才會集中，基礎模型會成為少數機構的產物，而其他人變成適配者。第 1.1.3 節談基礎模型概念時提到的集中化問題，它的技術根源就在這裡。

一個給臺灣的推論

如果尺度化紅利的門檻是資本規模，那麼臺灣在「從零訓練最大的模型」這條路上結構性地不利。但同一個邏輯反過來看：既然基礎模型會由少數機構提供且多為開放權重，那麼「誰能把它適配得最好」就成為新的

競爭場域，而那個場域的門檻是領域知識與資料品質，不是資本。這正是本書第六篇之後的全部內容。

2.6.4 Transformer 也有它的牆

為了維持本章「每個突破都會撞上新牆」的敘事誠實，必須說清楚 Transformer 留下了什麼問題：

- 平方複雜度：序列長度加倍，注意力的計算與記憶體變成四倍。這使得長上下文非常昂貴（第 18、20 章）。
- 記憶體牆：現代訓練的瓶頸往往不是算力而是記憶體頻寬——GPU 算得完，但資料餵不進去（第 18 章）。
- 位置外推困難：訓練時看過的最長長度，推論時若超過，效果會急遽下降（第 20 章的 RoPE 外推與 YaRN）。
- 推論時的序列性回歸：訓練可以平行，但生成時仍必須一個詞一個詞產生。KV cache 與推測解碼就是為了緩解這件事（第 14 章）。

最後一點特別值得玩味：我們花了很大力氣把訓練的序列相依拿掉，結果在推論時它又回來了。第 14 章與第 35 章有相當篇幅在處理這個問題，而它也是為什麼「推論成本」在產業界往往比「訓練成本」更受關注。

2.7 三類架構：BERT、T5 與 GPT

Transformer 原始論文是一個 Encoder-Decoder 架構（為了翻譯）。之後的發展分化成三條路線，各自取用其中一部分。

架構	代表模型	訓練目標	擅長	不擅長
Encoder-only	BERT、RoBERTa	遮罩語言建模：把部分詞遮住讓模型填回	分類、抽取、檢索、句子表示	生成（沒有自回歸機制）
Encoder-Decoder	T5、BART	去雜訊：把破壞過的輸入還原成原文	翻譯、摘要等明確的序列到序列任務	開放式對話與泛用生成
Decoder-only	GPT、Llama 系列	自回歸：預測下一個詞元	生成、對話、少樣本學習、泛用任務	需要雙向語境的表示任務

2.7.1 為什麼 Decoder-only 贏了

2018 到 2020 年間，BERT 一度是自然語言處理的絕對主流——幾乎所有排行榜都被它與它的變體占

據。但到了 2022 年之後，幾乎所有的大模型都是 Decoder-only。這個轉變的理由有四個，值得學理解，因為它是一個「工程考量壓過理論優雅」的好例子。

1. 任務統一：所有任務都可以寫成「輸入文字、輸出文字」的形式。分類？讓模型輸出類別名稱。抽取？讓模型輸出被抽取的內容。翻譯？直接輸出譯文。不需要為每個任務設計不同的輸出頭。
2. 訓練效率高：遮罩語言建模每次只在被遮住的位置（通常是 15%）產生訓練訊號；自回歸則是每個位置都在預測下一個詞，每一個詞元都對梯度有貢獻。同樣的資料，自回歸能榨出更多學習訊號。
3. 少樣本學習：GPT-3 展示了一件出乎意料的事——只要在提示中給幾個例子，模型就能執行沒訓練過的任務，完全不必更新權重。這打開了整個提示工程的領域（第 29 章）。
4. 規模化友善：架構單純（就是一疊相同的 block），沒有 encoder 與 decoder 之間的複雜互動，容易往上疊、容易做分散式切分（第 19 章）。

要強調的是：這不代表 Encoder-only 沒用了。今天的檢索系統仍然大量使用 Encoder-only 的嵌入模型——第 30 章你要建 RAG 時，負責把文件轉成向量的那個模型多半就是 BERT 家族的後裔。工程實務中，一個好的系統往往同時使用三類架構，各司其職。

2.7.2 一個常見的混淆：GPT 的「G」不是生成的意思嗎

GPT 是 Generative Pre-trained Transformer 的縮寫，三個詞各自代表一個設計決策：Generative（自回歸生成式目標）、Pre-trained（先在大量無標註文本上預訓練，再適配下游任務）、Transformer（架構）。

其中「Pre-trained」這個詞在今天已經理所當然，但在 2018 年是一個重要主張：不要為每個任務從零訓練，先訓練一個通用的模型再說。這個主張後來演變成第 1 章談的基礎模型典範，也是本書第六篇「在別人的基礎模型上做臺灣適配」這條路線的合法性來源。

2.8 從預訓練到 Agentic AI：能力的四次躍遷

架構在 2017 年之後其實變化不大——今天的 Llama-style 模型與 2018 年的 GPT-2 在骨架上高度相似。真正快速演進的是「怎麼訓練」與「怎麼使用」。本節把這條線整理成四次躍遷，每一次都對應本書的某幾章。

躍遷	時期	關鍵改變	解鎖的能力	本書章節
第一次	2018–2020	預訓練加微調成為標準流程	一個模型適配多種任務	第 22–24 章
第二次	2020–2022	提示學習與少樣本學習	不改權重就能執行新任務	第 29 章
第三次	2022–2024	指令微調與人類回饋對齊	模型聽得懂指令、行為可控	第 23、25 章
第四次	2024–至今	推理模型、工具使用、多模態	長思考、操作外部世界、看與聽	第 27、32、38–39 章

2.8.1 第一次躍遷：預訓練加微調

在此之前，做情感分析就訓練一個情感分析模型，做命名實體辨識就訓練一個命名實體辨識模型，每個任務都需要自己的標註資料。BERT 與 GPT 證明了另一條路：先在海量無標註文本上預訓練，再用少量標註資料微調，效果大幅超越從零訓練。

這件事的意義在於資料經濟學的改變。無標註文本幾乎免費且無限，標註資料昂貴且稀缺。把大部分學習搬到無標註階段，等於把成本從稀缺資源轉移到充裕資源。這也正是為什麼第二篇（語料工程）在本書中占了五章——資料變成了主要戰場。

2.8.2 第二次躍遷：提示與少樣本學習

GPT-3 (2020) 展示了一件當時很難解釋的事：只要在輸入中放幾個範例，模型就能執行從未訓練過的任務。這稱為情境學習 (in-context learning)。

從工程角度看，這是一次巨大的降門檻：不需要 GPU、不需要標註資料、不需要訓練，改一段文字就能得到一個新功能。整個「用 API 做應用」的產業就是建立在這件事上。

但也要清楚它的天花板。第 1 章的難度階梯法會讓你量到：提示能改善的幅度是有限的，且在長文本中會逐漸失效。提示是推論期的技巧，改變不了模型的知識與能力邊界。這是第 1.2.3 節那張「能力三個來源」表格的實務意涵。

2.8.3 第三次躍遷：指令微調與對齊

預訓練模型會「接話」但不會「做事」。你問它「臺灣的首都在哪裡」，它可能回你「這是一個常見的地理問題。下一題：日本的首都在哪裡？」——因為它學到的是文本的延續模式。

指令微調 (第 23 章) 用大量「指令—回應」配對教會模型：接到指令時應該執行，不是延續。人類回饋對齊 (第 25 章) 再進一步，用人類的偏好判斷調整模型的行為傾向——要有幫助、要誠實、要無害。

這一步對臺灣特別重要，原因在第 1.6 節已經說過：什麼樣的回答在臺灣的教育現場、產業現場、政府現場算是「好」，只有這裡的人能定義。這不是可以外包的工作。

2.8.4 第四次躍遷：推理、工具與多模態

最近的方向可以歸納成三句話：讓模型想得更久 (推理模型與測試時計算，第 27 章)、讓模型能動手 (工具使用與 Agent，第 32 章)、讓模型能看能聽 (多模態，第 38–39 章)。

這三者有一個共同的性質：它們都在推論期而非訓練期增加投入。這對算力受限的環境是好消息——你不必訓練一個更大的模型，只要讓現有模型多想一會兒、多查一次資料、多用一次工具。第 27 章會給出具體的成本與正確率曲線，讓你判斷這筆交換划不划算。

2.8.5 一張把架構與能力分開看的年表

學生常把「架構演進」與「能力演進」混為一談，導致誤以為每次能力躍遷背後都有一個新架構。實際上這兩條線的節奏差很多，把它們分開看會清楚許多。

年份區間	架構層面發生了什麼	能力層面發生了什麼	本書對應
2013–2016	詞向量、RNN、LSTM、Encoder-Decoder、注意力	機器翻譯品質大幅提升；任務仍各自訓練	第 2 章 (本章)
2017–2019	Transformer 出現並分化為三類架構	預訓練加微調成為標準流程	第 10–13 章
2020–2021	架構大致定型，主要是放大	少樣本學習與提示工程出現	第 17、29 章
2022–2023	Llama-style 元件替換 (RMSNorm、RoPE、SwiGLU、GQA)	指令微調與人類回饋對齊成熟；開放權重生態成形	第 12、23、25 章
2024–2026	MoE、長上下文、多模態融合、線性注意力探索	推理模型、工具使用、Agent、具身智慧	第 20、27、32、38–39 章

看這張表最該注意的是中間那一欄與左邊那一欄的落差：2020 年之後，架構的變動相當有限 (大致

就是把幾個元件換掉)，但能力的變動極大。這說明近年的進步主要來自資料、訓練方法與使用方式，而非架構創新。

這個觀察對本書的讀者有直接意義：如果你以為要做出好的臺灣模型必須發明新架構，那你的門檻會被自己抬到不必要的高度。真正的施力點在資料（第二篇）、後訓練（第六篇）與系統設計（第八篇），而那三個地方的門檻是領域知識與工程紀律，不是天才的靈光。

2.9 臺灣視角的技術史

前面八節講的是國際主流敘事。這一節要補上一段通常不會出現在教科書裡、但對臺灣學生很重要的脈絡。

2.9.1 中文斷詞：一個臺灣長期投入的問題

英文有空白，中文沒有。這個看似瑣碎的差異，在 n-gram 與早期神經模型時代是中文自然語言處理的第一道關卡：你得先決定「研究生命起源」要切成「研究 / 生命 / 起源」還是「研究生 / 命 / 起源」，才能開始做任何統計。

臺灣的學術界在這個問題上投入了數十年，中央研究院的中文詞知識庫小組與相關的斷詞系統是華語圈的重要基礎建設。這些工作在今天仍有價值——但也必須誠實地說，大模型時代的 subword tokenization（第 7 章）在相當程度上繞過了斷詞問題：模型不再需要正確的詞界，它直接在 subword 或 byte 層級運作。

這是技術史上常見的模式：一個被深入研究的問題，最後不是被解決，而是被繞過。但繞過不等於消失——第 6 章你會看到，繁體中文的正規化、異體字、全半形這些問題仍然存在，只是換了個位置出現。

2.9.2 繁體中文語料的結構性稀缺

這是一個必須用數字說話的問題。網路文本中英文占絕對多數，中文次之，而中文之中繁體又只占一小部分。當語料按比例採樣時，臺灣使用的繁體中文在訓練資料中的份額極小。

這造成一個惡性循環：語料少 → 模型的繁中能力弱 → 使用者傾向用英文或轉用其他模型 → 繁中的新增網路內容相對更少。要打破這個循環，需要主動的語料建設，而不是被動等待。第 8 與第 9 章會把這件事當成一個具體的工程任務來做。

2.9.3 臺灣本土模型的嘗試

近年臺灣出現了數個本土大模型的努力，路線各不相同：有以國家計畫形式推動、強調可信任與公部

門適用的方向；有以學術團隊主導、強調開放與研究社群的方向；也有企業針對特定產業（例如製造與金融）的垂直模型。

這些嘗試的共同特徵是：幾乎都採取「在開放權重基礎模型上做繁中與領域適配」的路線，而非從零預訓練。這與第 1.4 節的路線分析一致，也是本書把第 22 到 26 章寫得特別厚的原因——那是臺灣真正在做的事。

2.9.4 一個值得思考的反事實

本章的敘事一直是「國際主流怎麼走，臺灣怎麼跟上」。但值得花幾分鐘想一個反事實：如果當年臺灣在中文斷詞上的深厚累積，能夠更早轉向大規模語料建設與神經模型，今天的處境會不會不同？

這不是為了懷舊，而是為了辨識一個仍然存在的風險模式：把資源集中在一個問題的精緻化，而該問題正在被另一條路線繞過。在斷詞的例子裡，繞過它的是 `subword tokenization`。今天有沒有類似的情況？可能有——例如把大量資源投入單一模型的排行榜分數，而真正的瓶頸其實在資料與評測基礎建設上。

這一節沒有標準答案，但我希望學生養成一個習慣：定期問自己「我現在努力的這個問題，會不會在三年後被某條路線整個繞過？如果會，那我該累積的是什麼？」在本書的脈絡下，我的判斷是：語料治理能力、評測建置能力與領域知識的數位化，這三者不容易被繞過，因為它們不是技術問題，是組織與信任問題。

版本敏感提醒

本節提到的模型與計畫狀態變動快速。正文刻意不寫具體版本、參數量與評測數字，這些一律放在課程網頁的線上補充頁面，並在每次開課前重新查核。你在做本章的系統案例時，請自行查證最新狀態並在報告中標註查核日期。

程式實作

本章三個實作都在 A 級硬體（一般筆電）上可完成。語料使用課程提供的小型繁體中文語料（約 200 萬字，來源與授權見 ch02/DATA_CARD.md），你也可以換成自己的語料——但請先讀第 8 章關於授權的說明。

程式 2A 字元級 bigram 語言模型

目標：用純 Python 與 NumPy 實作一個字元級 bigram 模型，觀察平滑前後的機率分布與生成差異。這個實作沒有任何神經網路，目的是讓你親身體會「數數」到底能做到什麼程度，以及在哪裡失效。

ch02/lab2a_bigram.py (節錄)

```
import numpy as np
from collections import Counter, defaultdict

def build_counts(text: str):
    """建立 bigram 計數矩陣與字元索引"""
    chars = sorted(set(text))
    stoi = {c: i for i, c in enumerate(chars)}
    V = len(chars)
    C = np.zeros((V, V), dtype=np.int64)
    for a, b in zip(text, text[1:]):
        C[stoi[a], stoi[b]] += 1
    return C, stoi, chars

def to_probs(C, k=0.0):
    """k=0 為無平滑；k>0 為加 k 平滑"""
    P = C.astype(np.float64) + k
    row = P.sum(axis=1, keepdims=True)
    row[row == 0] = 1.0 # 避免除以零
    return P / row

def perplexity(P, text, stoi):
    """對數空間計算，避免浮點下溢"""
    logp, n = 0.0, 0
    for a, b in zip(text, text[1:]):
        p = P[stoi[a], stoi[b]]
        if p <= 0:
            return float("inf") # 未平滑時會發生，這正是我們要觀察的
        logp += np.log(p); n += 1
    return float(np.exp(-logp / n))

def generate(P, stoi, chars, start="今", n=100, rng=None):
    rng = rng or np.random.default_rng(0)
    out, cur = [start], stoi[start]
    for _ in range(n):
        nxt = rng.choice(len(chars), p=P[cur])
        out.append(chars[nxt]); cur = nxt
    return "".join(out)

if __name__ == "__main__":
    train = open("data/tw_corpus_train.txt", encoding="utf-8").read()
    valid = open("data/tw_corpus_valid.txt", encoding="utf-8").read()
    C, stoi, chars = build_counts(train)
    for k in [0.0, 0.01, 0.1, 1.0]:
        P = to_probs(C, k)
```

```
print(f"k={k:<5} 訓練 PPL={perplexity(P, train, stoi):8.2f} "
      f"驗證 PPL={perplexity(P, valid, stoi):8.2f}")
print(" 生成:", generate(P, stoi, chars, n=40))
```

你應該觀察到的現象與思考題：

- $k = 0$ 時驗證集困惑度為無限大。為什麼？請找出是哪一個字元組合造成的，並說明這與 2.2.2 節的資料稀疏是同一件事。
- k 從 0.01 增加到 1.0，訓練困惑度與驗證困惑度的變化方向為何不同？這是你第一次遇到「正則化強度」的取捨，第 12 章會再遇到。
- 生成的文字讀起來像中文，但完全沒有意義。這是 bigram 的本質限制——它只記得前一個字。請計算：要讓模型記住前四個字，計數矩陣需要多少記憶體？

程式 2B 三代模型同場競技

目標：在同一份語料、同一組評測條件下，比較 n -gram、Tiny RNN 與 Tiny Transformer 的困惑度、訓練時間與參數量。這是本書第一次做「控制變因的模型比較」，第 16 章會把這套方法論完整展開。

ch02/lab2b_three_eras.py (節錄)

```
import time, json, torch, torch.nn as nn

class TinyRNN(nn.Module):
    def __init__(self, V, d=128, layers=1):
        super().__init__()
        self.emb = nn.Embedding(V, d)
        self.rnn = nn.LSTM(d, d, layers, batch_first=True)
        self.head = nn.Linear(d, V)
    def forward(self, x):
        h, _ = self.rnn(self.emb(x))
        return self.head(h)

class TinyTransformer(nn.Module):
    def __init__(self, V, d=128, heads=4, layers=2, ctx=128):
        super().__init__()
        self.emb = nn.Embedding(V, d)
        self.pos = nn.Embedding(ctx, d)
        blk = nn.TransformerEncoderLayer(d, heads, 4 * d,
                                          batch_first=True, norm_first=True)
        self.enc = nn.TransformerEncoder(blk, layers)
        self.head = nn.Linear(d, V)
        self.ctx = ctx
    def forward(self, x):
        T = x.size(1)
        pos = torch.arange(T, device=x.device)
        h = self.emb(x) + self.pos(pos)
        # 因果遮罩：不可看見未來（第 11 章會完整推導）
        mask = torch.triu(torch.ones(T, T, device=x.device), 1).bool()
        return self.head(self.enc(h, mask=mask))

def train_and_eval(model, train_loader, valid_loader, epochs=3, lr=3e-4):
    opt = torch.optim.AdamW(model.parameters(), lr=lr)
    lossf = nn.CrossEntropyLoss()
    t0 = time.perf_counter()
```

```

for _ in range(epochs):
    model.train()
    for x, y in train_loader:
        logits = model(x)
        loss = lossf(logits.reshape(-1, logits.size(-1)), y.reshape(-1))
        opt.zero_grad(); loss.backward(); opt.step()
    train_s = time.perf_counter() - t0
    model.eval(); tot, n = 0.0, 0
    with torch.no_grad():
        for x, y in valid_loader:
            logits = model(x)
            l = lossf(logits.reshape(-1, logits.size(-1)), y.reshape(-1))
            tot += l.item() * y.numel(); n += y.numel()
    return {
        "params": sum(p.numel() for p in model.parameters()),
        "train_seconds": round(train_s, 1),
        "valid_ppl": round(float(torch.exp(torch.tensor(tot / n))), 2),
    }

```

實驗必須固定的變因（這是本實驗最重要的部分，比程式碼本身重要）：

變因	如何固定	不固定會怎樣
語料與切分	同一份訓練 / 驗證檔，固定隨機種子	結果差異可能來自資料而非模型
tokenizer	三個模型都用字元級	困惑度不可比較（見 2.1.4 節）
序列長度	統一 128	長度不同會改變任務難度
訓練預算	固定 epoch 數或固定 token 數	訓練越久當然越好，比的就不是架構了
隨機種子	至少跑 3 個種子取中位數	單次結果可能是運氣

延伸挑戰（擇一完成）：

- 把 Tiny Transformer 的層數從 2 增加到 4，觀察困惑度、參數量與訓練時間的變化。畫出「參數量對困惑度」的曲線，這是你第一次接觸尺度化的實證形狀，第 17 章會把它變成一條可擬合的冪次定律。
- 把三個模型都改用同一個 subword tokenizer 而非字元級，重測困惑度。你會發現數字整個變了——這正是 2.1.4 節那個警告的實證，也是第 7 章的動機。
- 刻意在訓練語料中重複貼入某一段文字 50 次，訓練後測試模型是否會逐字複述它。這是記憶化現象的最小示範，第 9 章（去重）與第 16 章（記憶化偵測）會深入處理。

程式 2C 平行度實測：RNN 與 Transformer 的牆鐘時間曲線

目標：量測序列長度增加時，兩種架構的實際訓練時間如何變化。這是本章理論部分（2.4.5 節）的

實證，也是你第一次做效能量測。

ch02/lab2c_parallelism.py (節錄)

```
import time, torch

def bench(model, V, T, B=8, steps=20, device="cpu"):
    """量測前向加反向的平均單步時間"""
    model = model.to(device).train()
    x = torch.randint(0, V, (B, T), device=device)
    y = torch.randint(0, V, (B, T), device=device)
    lossf = torch.nn.CrossEntropyLoss()
    opt = torch.optim.SGD(model.parameters(), lr=0.0) # 只量時間，不更新

    for _ in range(3): # 暖機，必須捨棄
        loss = lossf(model(x).reshape(-1, V), y.reshape(-1))
        opt.zero_grad(); loss.backward()

    if device == "cuda": torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(steps):
        loss = lossf(model(x).reshape(-1, V), y.reshape(-1))
        opt.zero_grad(); loss.backward()
    if device == "cuda": torch.cuda.synchronize()
    return (time.perf_counter() - t0) / steps

for T in [32, 64, 128, 256, 512]:
    r = bench(TinyRNN(V), V, T)
    t = bench(TinyTransformer(V, ctx=1024), V, T)
    print(f"T={T:4d}  RNN {r*1000:7.1f} ms  Transformer {t*1000:7.1f} ms"
          f"    比值 {r/t:5.2f}")
```

量測的三個紀律

第一，一定要暖機並捨棄前幾次——第一次呼叫包含編譯、記憶體配置與快取冷啟動。第二，在 GPU 上一定要呼叫 `synchronize`，否則你量到的是「發出指令的時間」而不是「算完的時間」，這是初學者最常犯的錯誤。第三，記錄裝置、精度、批次大小與函式庫版本，否則數字無法重現。這三條在第 18 章會再強調一次。

延伸挑戰：若你有 GPU，請在 CPU 與 GPU 上各跑一次，比較兩種架構的加速比。你會發現 Transformer 從 GPU 得到的加速遠大於 RNN——這就是 2.4.5 節那個交響樂團比喻的量化版本，也是整章論證的實證核心。

系統案例：繪製 2017–2026 大模型技術地圖

本章的系統案例不寫程式，而是製作一張可以掛在實驗室牆上、也能拿去向人解說的技術演進因果圖。這聽起來像作業，但它其實是一項專業能力：能把一個領域的來龍去脈說清楚的人，才有能力判斷下一步該往哪走。

製作規格

每個節點必須包含四個欄位，缺一不可：

欄位	寫什麼	好的例子	不好的例子
問題	前一代撞到的具體的牆	5-gram 無法捕捉超過四個詞的依賴	效果不夠好
方法	用什麼結構或訓練方式繞過	用隱藏狀態壓縮任意長度的歷史	RNN
代價	付出了什麼	梯度需沿時間反傳，訓練無法平行化	比較複雜
影響	引發了什麼新問題或新機會	長序列訓練成本高，催生 LSTM 與注意力	很重要

必須涵蓋的節點（至少十二個）

1. n-gram 與平滑法
2. 神經語言模型與詞向量（Word2Vec）
3. RNN 與梯度消失
4. LSTM / GRU 的閘門機制
5. Encoder-Decoder 與固定向量瓶頸
6. 注意力機制（Bahdanau / Luong）
7. Transformer 與自注意力
8. BERT 與遮罩語言建模
9. GPT 系列與自回歸預訓練
10. 提示學習與少樣本學習
11. 指令微調與人類回饋對齊
12. 推理模型、工具使用與多模態

評分重點

- 因果連結是否成立：每一條箭頭都必須說得出「因為前者的什麼問題，才有後者」。純粹的時間順序不算因果。
- 代價欄位是否誠實：只寫好處不寫代價的圖，是行銷素材不是技術地圖。

- 是否標出仍未解決的問題：例如平方複雜度、幻覺、對齊稅，這些至今仍是開放問題。
- 臺灣脈絡：至少標註三個與臺灣相關的節點（中文斷詞研究、繁中語料稀缺、本土模型嘗試）。
- 口頭解說：能在五分鐘內用這張圖向一位非本科系的聽眾說明「為什麼現在的 AI 這麼強」。

這張圖你會用很久。往後每讀完一篇，都建議回來更新它——當你在第 20 章讀到狀態空間模型時，你會需要在圖上加一個新分支；當你在第 27 章讀到推理模型時，你會發現它其實回應的是 2.6.4 節列出的某一道牆。

章末評量

一、概念題

1. 用自己的話說明鏈式法則如何把「估計整句話的機率」轉換成「預測下一個詞」，並說明這個轉換為什麼在工程上是革命性的。
2. 為什麼 n-gram 模型面臨的兩道牆（資料稀疏與長距離依賴）互相衝突、無法同時解決？請說明「加大 n」為什麼不是出路。
3. 分布式假說是什麼？它如何被轉換成 Word2Vec 的訓練目標？靜態詞向量的主要限制是什麼？
4. LSTM 解決了 RNN 的什麼問題？又保留了什麼問題？後者為什麼比前者更致命？
5. 注意力機制與自注意力的差別是什麼？請各舉一個使用場景。
6. 為什麼 Decoder-only 架構最終成為大模型的主流？請列出至少三個理由，並指出 Encoder-only 今天仍然重要的場景。

二、推導與計算題

1. 某語料中「智慧」出現 2,400 次，其中「智慧製造」600 次、「智慧機器人」360 次、「智慧城市」240 次。計算這三個 bigram 的最大概似機率。若採用加 1 平滑且詞彙表大小為 50,000，重新計算「智慧製造」的機率，並說明變化的方向與原因。
2. 一個模型在測試集上的平均每詞元交叉熵損失為 3.2（自然對數）。計算它的困惑度。若詞彙表大小為 32,000，說明這個困惑度代表模型學到了多少（相對於完全隨機猜測）。
3. 假設某 RNN 每個時間步的梯度衰減係數為 0.85。計算梯度傳遞 20 步、50 步與 100 步之後分別剩下原來的多少比例。據此說明為什麼 RNN 難以學習長距離依賴。
4. 一個 Transformer 的序列長度從 512 增加到 2048。注意力部分的計算量與記憶體各成長為幾倍？若你的 GPU 記憶體只夠跑 512，請提出三種可能的處置方式（提示：第 18、20 章）。

三、除錯題

1. 同學實作的 bigram 模型在訓練集上困惑度是 14.2，但在驗證集上是無限大。請指出原因，並提出兩種修正方式（一種改模型、一種改評估方式）。
2. 同學在程式 2B 中得到「Transformer 的困惑度比 RNN 差」的結果。請列出至少四個可能的原因，並說明各自的檢查方法。
3. 同學在 GPU 上量測 2C 的時間，得到「序列長度 512 與 32 幾乎一樣快」的荒謬結果。請指出最可能的原因。

4. 一份實驗報告寫著「我們的模型困惑度 18.3，優於文獻中的 21.5」。請指出這個比較至少三個可能不成立的地方。

四、系統設計題

1. 假設你要為臺灣的中文輸入法設計一個下一字預測模組，執行在手機上、需離線運作、記憶體限制 50 MB、延遲需低於 20 毫秒。請在 n-gram、小型 RNN 與小型 Transformer 中做出選擇並論證。你的論證必須涵蓋記憶體、延遲、更新成本與品質四個維度。
2. 某公司有大量歷史客服對話，想同時做三件事：把新進工單自動分類、檢索相似的歷史案例、以及生成建議回覆。請說明你會分別使用哪一類架構（Encoder-only / Encoder-Decoder / Decoder-only），並解釋為什麼不是三件事都用同一個模型。

五、臺灣情境題

1. 中文斷詞在 n-gram 時代是必要的前處理，但 subword tokenization 在相當程度上繞過了它。請說明「繞過」與「解決」的差異，並舉出至少兩個斷詞相關的問題在大模型時代仍然存在（提示：第 6、7 章）。
2. 請以本章的四欄式格式（問題 / 方法 / 代價 / 影響），為「繁體中文語料稀缺」這個問題製作一個節點，並提出三種可能的方法及其各自的代價。

六、倫理討論題

1. 2.6.3 節指出，Transformer 讓「投入可預期地轉成能力」，因而導致資本與人才的集中。請討論這種集中化對小型經濟體（如臺灣）的影響，並提出至少兩種在結構上減緩不利影響的做法。
2. 有人主張「反正基礎模型都是別人訓練的，我們只要會用就好」。請從技術自主、教育目標與長期風險三個角度，提出你的看法。請同時說出這個主張有道理的地方——一個只會反駁的論證是不完整的。

評量提示（給自學者）

- 計算題第 1 題：加 1 平滑後的分子為 601、分母為 2,400 加上詞彙表大小 50,000，機率會顯著下降——這正是加一平滑在大詞彙表上分配過多機率給未見事件的問題。
- 計算題第 2 題： $PPL = \exp(3.2) \approx 24.5$ 。相對於隨機猜測的 32,000，模型把候選範圍縮小了約 1,300 倍。
- 計算題第 3 題：0.85 的 20 次方約 0.039、50 次方約 0.0003、100 次方約 0.0000001。
- 除錯題第 3 題：最可能是沒有呼叫 `torch.cuda.synchronize()`，量到的是指令發出時間；次可能

是批次太小導致 GPU 未被填滿，時間被 kernel launch 開銷主導。

研究延伸與版本注意事項

- 奠基論文：Bengio 等人的神經機率語言模型（2003）、Mikolov 等人的 Word2Vec（2013）、Bahdanau 等人的注意力機制（2014）、Vaswani 等人的 Transformer（2017）。建議先讀完本書第 13 章再讀 Transformer 原論文，會輕鬆很多。
- 架構分化：BERT（2018）、T5（2019）、GPT-2（2019）與 GPT-3（2020）的技術報告。GPT-3 論文中關於少樣本學習的實驗設計特別值得細讀，它是第 29 章的直接前身。
- 對齊路線：InstructGPT（2022）建立了指令微調加入類回饋的標準流程，第 23 與 25 章的實作以此為基礎。
- 中文與繁中資源：中央研究院中文詞知識庫相關資源、教育部國語辭典與臺灣台語常用詞辭典。這些是第 6 章正規化與第 7 章 tokenizer 設計的重要參考。
- 課程資源：李宏毅教授的生成式 AI 課程對本章的技術演進有極佳的中文講解，建議搭配觀看；Stanford CS336 的前幾講則涵蓋本章到第 13 章的完整脈絡。
- 開源教材：浙江大學《大模型基礎》第一部分（語言模型基礎）與本章高度對應，可作為第二份參考。

關於技術史的閱讀態度

本章把演進寫成一條清晰的因果鏈，但真實的技術史遠比這混亂——同一時期有無數條路線在競爭，很多當時看好的方向後來消失，而勝出的方向在當時也未必被看好。教科書的整理必然帶有事後諸葛的成分。請保持一點警覺：當你在讀 2026 年的最新技術時，你並不知道哪一條會勝出。這正是為什麼本書一再強調量測與評測——在沒有後見之明的當下，數據是你唯一可靠的判準。

常見問題

問：既然 n-gram 和 RNN 都被取代了，為什麼還要學？答：三個理由。第一，你要能解釋為什麼 Transformer 是對的，而不是只能說「大家都這樣做」——判斷力來自知道替代方案有什麼問題。第二，這些方法在今天仍有實際用途（n-gram 用於去重與污染偵測、RNN 家族用於部分低延遲與低資源場景）。第三，最重要的：本章示範的思考方式——找出每個方法撞到的牆——是你在讀 2027 年新論文時唯一可靠的工具。

問：我可以跳過 2.2 節的 n-gram 直接讀 Transformer 嗎？答：可以讀懂，但你會失去對「資料稀疏」與「泛化」這兩個概念的直覺，而它們在第 9 章（去重）、第 17 章（尺度定律）與第 22 章（災難性

遺忘) 都會回來。建議至少把 2.2.2 與 2.2.6 兩小節讀完。

問：Transformer 之後真的沒有大的架構創新了嗎？答：本章的說法是「相對於 2013 到 2017 那段時期，變動較小」，不是「沒有變動」。MoE、長上下文技術、狀態空間模型都是實質的創新，第 20 章會完整處理。但它們大多是在 Transformer 的框架內優化，而非取而代之——至少到本書寫作時仍是如此。

問：為什麼一直強調「代價」？答：因為只講好處的技術敘事會讓人做出壞決策。當你在第 20 章要為一個實際專案選架構時，你需要知道的不是「哪個最先進」，而是「哪個的代價我付得起」。這個習慣從本章開始養成。

教師使用建議

本章排在第 2 週，建議 2 小時講授加 1 小時實驗。

時段	內容	互動設計	注意事項
前 25 分鐘	2.1 至 2.2 節，語言模型定義與 n-gram	現場用小語料手算一個 bigram 機率	務必讓學生看到「零機率」的發生
中間 35 分鐘	2.3 至 2.4 節，神經模型與 RNN 瓶頸	用交響樂團比喻解釋平行度，請學生換一個自己的比喻	2.4.5 節是全章核心，不可壓縮
後 30 分鐘	2.5 至 2.7 節，注意力、Transformer 與三類架構	分組為三類架構各找一個真實產品	不要在此展開數學，留給第 11 章
最後 30 分鐘	2.8 至 2.9 節與作業說明	當場開始畫技術地圖的前三個節點	技術地圖是本章唯一必交作業
實驗課 1 小時	程式 2A 與 2B	兩人一組，一人跑無平滑一人跑有平滑，互相對照	2C 可作為課後作業

高中授課的調整：跳過 2.1.3、2.1.4 的公式推導，改用「困惑度就是模型在幾個候選之間猶豫」的直覺說法；2.5.4 節的三種分數計算方式可略過。把省下的時間給程式 2A 與技術地圖的製作——高中生對「用數數就能生成中文」這件事的反應通常非常好，那是很好的切入點。

本章小結

1. 語言模型的核心任務是估計詞元序列的機率。鏈式法則把它轉換成「預測下一個詞元」，這個目

標函數從 n -gram 到今天的大模型從未改變，變的只是估計方法。

2. 對數空間計算是數值穩定的必要手段；困惑度是最常用的指標，但只在相同 tokenizer 與相同測試資料下可比較。
3. n -gram 用計數估計機率，撞上資料稀疏與長距離依賴兩道互相衝突的牆。平滑法能緩解前者，但無法解決後者。
4. 神經語言模型的關鍵轉變是把詞從離散符號變成連續向量，使模型能在語意相近的詞之間泛化，從根本上緩解稀疏問題。
5. RNN 理論上可處理任意長依賴，實務上受梯度消失所限；LSTM 與 GRU 用閘門機制緩解，但保留了無法平行化的結構性缺陷。
6. 注意力機制解決了固定長度向量瓶頸；Transformer 進一步拿掉循環結構，用平方複雜度換取完全平行化。
7. 平行化的真正意義是讓投入可預期地轉成能力，這開啟了尺度化紅利，也造成了資本與能力的集中。
8. Decoder-only 因任務統一、訓練訊號密集、支援少樣本學習與規模化友善而成為主流，但 Encoder-only 在檢索與表示任務上仍不可取代。
9. 2018 年之後架構變化不大，快速演進的是訓練方法與使用方式：預訓練微調、提示學習、指令對齊、推理與工具使用四次躍遷。
10. 臺灣脈絡中，中文斷詞研究被 subword tokenization 部分繞過，繁中語料稀缺是結構性問題，本土模型多採「在開放權重上適配」的路線。

成果檢核

完成本章後你必須繳交：

- 技術演進因果圖：至少 12 個節點，每個節點含問題、方法、代價、影響四欄，並標註至少三個臺灣相關節點。可用手繪、簡報或圖形工具製作。
- 程式 2A 的執行結果：不同 k 值下的訓練與驗證困惑度表格，以及對「 $k = 0$ 時驗證困惑度為無限大」的解釋。
- 程式 2B 的比較表：三種模型的參數量、訓練時間、驗證困惑度，並附上你固定了哪些變因的說明。
- 程式 2C 的時間曲線圖：序列長度對單步時間的關係，並用一段話解釋曲線形狀與 2.4.5 節的理論是否一致。

- 一頁反思：本章哪一個「代價」讓你最意外？它對你選定的貫穿式案例有什麼意涵？

技術演進因果圖請保留並持續更新，它會在第 20 章（架構選型）與第 40 章（總整專題的技術路線論證）再次被使用。

第 3 章

大模型所需的數學、機率與最佳化

Mathematics, Probability, and Optimization for Large Language Models

難度：核心 / 補強 | 建議授課：第 3 週 | 硬體需求：A 級（不需 GPU）

叫獸開講

很多人被「多頭注意力」四個字嚇退，其實它只是一堆矩陣乘法加上一個 softmax。我教了很多年，看不懂公式的學生裡有九成不是被數學打敗，是被張量的形狀弄糊塗了——他們不知道那個 (B, H, T, d) 到底哪一維是什麼。這一章不教你證明定理，教你在腦中把形狀追蹤清楚，然後用程式驗證每一步。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 正確讀寫張量的形狀標記，並在看到任何一行矩陣運算時說出輸入與輸出的維度。
2. 解釋廣播規則，並判斷兩個張量能不能相加、相乘，若不能則說出該如何調整。
3. 從機率鏈式法則推導自回歸語言模型的最大概似目標，並說明它為何等價於最小化交叉熵。
4. 手寫數值穩定的 softmax 與 log-sum-exp，說明為什麼直接照公式寫會溢位。
5. 推導交叉熵對 logits 的梯度，並用有限差分驗證你的推導是對的。
6. 說明反向傳播如何在計算圖上運作，以及為什麼它比逐一求導快上數個數量級。
7. 比較 SGD、Momentum、Adam、AdamW 的更新規則，說明各自適合什麼情境。
8. 辨識梯度消失、梯度爆炸與數值溢位的徵兆，並知道對應的處置手段。
9. 完成十題 shape tracing 練習，能從公式反推可執行程式，也能從程式反推公式。

給高中讀者與數學基礎較弱的讀者

這一章不需要你先修過線性代數或微積分。所有概念都配了圖解與數值例子，公式旁邊一定有一段可以執行的程式。建議的讀法是：先看每節的數值例子，再回頭看公式。如果某個公式看三遍還是不懂，跳過它，去跑對應的程式——看到數字動起來之後再回來看，通常就通了。這不是投機取巧，這是工程師真正的學習方式。

3.1 張量與形狀推理：本章最重要的技能

如果這一章你只學會一件事，我希望是這件：養成隨時追蹤張量形狀的習慣。這個習慣會在第 11 章救你一命，在第 13 章救你三次，在第 19 章的分散式訓練裡救你無數次。

3.1.1 從純量到張量

名稱	階數	形狀範例	在大模型中的實例	記法
純量 scalar	0	()	損失值、學習率	x
向量 vector	1	(768,)	一個詞元的嵌入向量	v
矩陣 matrix	2	(768, 3072)	前饋網路的權重	W
三階張量	3	(8, 512, 768)	一批序列的隱藏狀態	H
四階張量	4	(8, 12, 512, 512)	多頭注意力的權重矩陣	A

請特別看最後兩列。三階張量 (8, 512, 768) 讀作：批次大小 8、序列長度 512、隱藏維度 768。四階張量 (8, 12, 512, 512) 讀作：批次 8、注意力頭數 12、查詢位置 512、鍵位置 512。

本書統一使用以下符號，往後三十七章都不會改變。請把這張表貼在你的螢幕旁邊。

符號	意義	典型數值	出現的位置
B	batch size，一次處理幾條序列	1 至 64	所有訓練與推論程式
T	sequence length，序列有幾個詞元	128 至 128000	第 11、14、20 章
V	vocabulary size，詞彙表大小	32000 至 256000	第 7、13 章
d 或 d_model	模型隱藏維度	256 至 8192	第 10–13 章
H	number of heads，注意力頭數	4 至 64	第 11 章
d_h	每個頭的維度，等於 d ÷ H	32 至 128	第 11 章
L	number of layers，層數	4 至 126	第 12、17 章
d_ff	前饋網路的中間維度，	1024 至 32768	第 12 章

符號	意義	典型數值	出現的位置
	通常為 4d		

3.1.2 形狀推理的三條規則

所謂形狀推理，就是在不執行程式的情況下，用心算推出每一步的輸出形狀。它只需要三條規則。

1. 矩陣乘法： $(..., m, k)$ 乘上 $(..., k, n)$ 得到 $(..., m, n)$ 。中間的 k 必須相等，而且它會消失。前面的省略號部分必須可以廣播。
2. 逐元素運算（加減乘除、activation）：形狀不變，但兩個運算元必須可廣播。
3. 約簡運算（sum、mean、max、softmax）：指定的那一維消失（或在 `keepdim=True` 時變成 1）。softmax 是特例——它在指定維度上正規化，形狀完全不變。

用一個具體例子練習。假設 $B=2$ 、 $T=4$ 、 $d=6$ 、 $H=2$ ，因此 $d_h=3$ 。追蹤一次注意力的完整形狀變化：

步驟	輸入形狀	輸出形狀	發生了什麼
輸入 x	—	$(2, 4, 6)$	兩條長度 4 的序列，每個位置 6 維
$x @ W_q \cdot W_q$ 為 $(6,6)$	$(2,4,6) @ (6,6)$	$(2, 4, 6)$	線性投影，最後一維 6 被消掉又產生 6
reshape 成多頭	$(2, 4, 6)$	$(2, 4, 2, 3)$	把 6 拆成 $H=2$ 個頭乘 $d_h=3$
transpose 頭與時間	$(2, 4, 2, 3)$	$(2, 2, 4, 3)$	讓頭維度往前，方便批次矩陣乘法
$Q @ K$ 轉置	$(2,2,4,3) @ (2,2,3,4)$	$(2, 2, 4, 4)$	注意力分數矩陣， $T \times T$
softmax 於最後一維	$(2, 2, 4, 4)$	$(2, 2, 4, 4)$	形狀不變，每列總和為 1
權重 $@ V$	$(2,2,4,4) @ (2,2,4,3)$	$(2, 2, 4, 3)$	加權平均，回到 d_h
transpose 回來並合併	$(2, 2, 4, 3)$	$(2, 4, 6)$	恢復成原本的形狀

這張表就是第 11 章你要實作的全部內容。現在看不懂細節沒關係，但請注意一件事：從輸入到輸出，形狀繞了一圈又回到 $(2, 4, 6)$ 。這個「進去什麼形狀、出來什麼形狀」的性質，正是 Transformer 可以把同一個 block 疊很多層的原因。

3.1.3 用斷言把形狀寫進程式

形狀錯誤是大模型開發中最常見的 bug，而且它們往往不會立刻報錯——廣播會把一個錯誤的形狀「悄悄地」變成一個合法但無意義的運算。最惡劣的情況是程式跑得順順的，loss 也在下降，但模型學到的東西是錯的。

對策很簡單：在每個關鍵步驟加上形狀斷言。這在本書的所有實作中都是強制要求。

形狀斷言的標準寫法

```
def attention(x, W_q, W_k, W_v, H):
    B, T, d = x.shape
    assert d % H == 0, f"d={d} 必須能被 H={H} 整除"
    d_h = d // H

    q = x @ W_q                                # (B, T, d)
    assert q.shape == (B, T, d), q.shape

    q = q.reshape(B, T, H, d_h).transpose(1, 2) # (B, H, T, d_h)
    assert q.shape == (B, H, T, d_h), q.shape

    # ... k, v 同理
    scores = q @ k.transpose(-2, -1) / (d_h ** 0.5) # (B, H, T, T)
    assert scores.shape == (B, H, T, T), scores.shape
    return scores
```

一個真實的教訓

我看過學生把 (B, T, d) 誤當成 (B, d, T)，程式沒有報錯，訓練跑了六小時，loss 也確實下降了——因為模型還是學到了某種模式，只是那個模式毫無意義。如果一開始就寫了三行 assert，六小時可以省下來。斷言不是給機器看的，是給三天後的你看的。

3.1.4 從形狀算出記憶體：一項每天都會用到的心算

知道形狀之後，記憶體占用只差一次乘法。這件事看起來瑣碎，但它決定了你的實驗能不能跑起來，所以值得練到成為反射。

$$\text{記憶體 (bytes)} = \text{元素個數} \times \text{每個元素的位元組數}$$

常用的位元組數：FP32 是 4、BF16 與 FP16 是 2、INT8 是 1、4-bit 量化是 0.5。以下是幾個你會反覆遇到的估算。

張量	形狀	BF16 大小	出現的位置
嵌入表	(V, d) = (32000, 768)	約 49 MB	模型參數，第 10 章

張量	形狀	BF16 大小	出現的位置
一層隱藏狀態	(8, 512, 768)	約 6 MB	不大，容易被忽略
注意力分數（單層）	(8, 12, 512, 512)	約 50 MB	隨 T 平方成長
注意力分數（T=4096）	(8, 12, 4096, 4096)	約 3.2 GB	同一個式子，T 放大 8 倍
輸出 logits	(8, 512, 32000)	約 262 MB	計算損失時的記憶體大戶

請對照第三列與第四列：序列長度從 512 變成 4096（8 倍），注意力矩陣從 50 MB 變成 3.2 GB（64 倍）。這就是平方複雜度的具體感受。第 2 章講的「Transformer 用記憶體換平行度」，在這張表裡變成了可以觸摸的數字。

一個可以練的心算

看到任何一個模型設定，立刻在腦中估出它的參數量與 activation 大小。例如 $d=4096$ 、 $L=32$ 、 $V=128000$ 的模型，光是嵌入表就是 $128000 \times 4096 = 5.2$ 億參數；每一層的注意力投影約 $4d^2 = 6700$ 萬，前饋約 $8d^2$ （若 $d_{ff}=4d$ 且用 SwiGLU 則更多）= 1.3 億，兩者合計約 2 億乘以 32 層，約 64 億。加起來大致是 7B 量級——這就是為什麼 7B 是一個常見的模型尺寸。第 12 章會給出精確的計算公式。

3.2 矩陣運算、廣播與愛因斯坦記法

3.2.1 內積：一切相似度的起點

兩個向量的內積是最基本的運算：把對應位置相乘再全部加起來。

$$a \cdot b = \sum_i a_i b_i = |a| |b| \cos \theta$$

右邊那個等式是關鍵：內積等於兩個向量長度的乘積乘上夾角的餘弦。所以當兩個向量長度固定時，內積直接反映了它們的方向有多接近。這就是為什麼注意力機制用內積來算「相關性」——它在問「這兩個向量指向類似的方向嗎」。

在實務上我們常用餘弦相似度，也就是把長度的影響除掉：

$$\cos_sim(a, b) = (a \cdot b) / (|a| \cdot |b|)$$

這個值介於 -1 到 1 之間。第 10 章的語意地圖與第 30 章的向量檢索，用的都是這個式子。

3.2.2 矩陣乘法：不是一件事，是很多件事一起做

矩陣乘法 $C = A @ B$ 的每一格 $C[i,j]$ ，就是 A 的第 i 列與 B 的第 j 行的內積。所以一次矩陣乘法等於同

時算了 $m \times n$ 個內積。

這件事在硬體上意義重大。GPU 對矩陣乘法有專門的硬體單元 (tensor core)，能以極高的吞吐量執行。因此在大模型工程中有一條實務準則：能寫成矩陣乘法的，就不要寫成迴圈。

運算	寫法	在大模型中的位置	典型形狀
線性投影	$x @ W$	Q、K、V 投影、前饋網路	$(B,T,d) @ (d,d)$
注意力分數	$Q @ K^T$	第 11 章的核心	$(B,H,T,d_h) @ (B,H,d_h,T)$
加權求和	$A @ V$	注意力輸出	$(B,H,T,T) @ (B,H,T,d_h)$
輸出投影	$h @ W_{out}$	映射回詞彙表	$(B,T,d) @ (d,V)$

看最後一列：把 (B,T,d) 映射到 (B,T,V) 。若 $V = 32000$ 、 $d = 768$ 、 $B = 8$ 、 $T = 512$ ，這一步的輸出張量有 $8 \times 512 \times 32000 = 1.3$ 億個浮點數，用 float32 存要 524 MB。這就是為什麼在計算損失時，輸出層往往是記憶體的大戶——第 15 章與第 18 章會處理這個問題。

3.2.3 廣播：方便但危險

廣播 (broadcasting) 允許形狀不同的張量做逐元素運算，規則是從最後一維往前對齊，每一維要嘛相等、要嘛其中一個是 1。

A 的形狀	B 的形狀	結果	說明
(4, 3)	(3,)	(4, 3)	B 被複製 4 次，常見於加 bias
(2, 1, 5)	(1, 4, 5)	(2, 4, 5)	兩邊各自擴張，很常用
(B, T, d)	(d,)	(B, T, d)	對每個位置加同一個向量
(B, H, T, T)	(1, 1, T, T)	(B, H, T, T)	因果遮罩套用到所有批次與頭
(4, 3)	(4,)	錯誤	最後一維 3 與 4 不相容

廣播的危險在於：它可能把一個你不想要的運算變成合法運算。經典的例子是形狀 $(T,)$ 與 $(T, 1)$ 的差別——前者加上 $(T,)$ 得到 $(T,)$ ，後者加上 $(T,)$ 卻得到 (T, T) 。在計算損失時若不小心弄錯，你會得到一

個看起來正常但完全錯誤的數字。

防呆習慣

任何時候你不確定廣播會怎麼發生，就明確地用 `reshape` 或 `unsqueeze` 把維度寫出來，不要依賴自動推斷。多寫三個字元，省下三小時除錯。

3.2.4 愛因斯坦記法：把形狀寫在程式裡

`einsum` 讓你用字母標記每一維，運算規則直接寫在字串裡。它的最大價值不是簡潔，而是自我說明——讀程式的人不必推理就知道每一維是什麼。

einsum 的四個實用範例

```
import torch

# 1. 內積：兩個向量
torch.einsum("i,i->", a, b)           # a:(d,) b:(d,) -> 純量

# 2. 批次線性投影
torch.einsum("btd,de->bte", x, W)     # (B,T,d) @ (d,e) -> (B,T,e)

# 3. 注意力分數（不必手動 transpose）
torch.einsum("bhqd,bhkd->bhqk", q, k) # -> (B,H,T_q,T_k)

# 4. 注意力加權求和
torch.einsum("bhqk,bhkd->bhqd", attn, v) # -> (B,H,T_q,d_h)
```

請看第三個例子：`q` 的形狀是 (b,h,q,d) 、`k` 的形狀是 (b,h,k,d) ，輸出是 (b,h,q,k) 。字母 `d` 在輸出中消失了，代表它被求和掉——這正是內積。而 `q` 與 `k` 兩個字母都保留，形成一個 $T \times T$ 的矩陣。整個注意力分數的計算，用一行字串就說清楚了，而且不必寫 `transpose`。

本書的實作會同時提供 `einsum` 版本與傳統版本，讓你對照。建議初學時兩種都寫一次，確認結果相同——這也是一個很好的自我檢查方式。

3.2.5 算一次矩陣乘法要多少次浮點運算

這個估算你會用一輩子，因為第 17 章的尺度定律、第 18 章的 MFU 計算、第 21 章的算力預算全都建立在它上面。

$$(m, k) @ (k, n) \text{ 的浮點運算次數} \approx 2 \times m \times k \times n$$

為什麼是 2 倍？因為輸出的每一格需要 k 次乘法與 k 次加法，共 $2k$ 次運算，而輸出有 $m \times n$ 格。

用這個公式估算一個 Transformer 層的前向計算量。設 $B=1$ 、 $T=1024$ 、 $d=768$ 、 $d_{\text{ff}}=3072$ ：

運算	形狀	FLOPs	占比
Q、K、V 投影	(T,d) @ (d,d) × 3	約 3.6 G	約 20.5%
注意力分數 QK ^T	(T,d) @ (d,T)	約 1.6 G	約 9.1%
注意力加權 AV	(T,T) @ (T,d)	約 1.6 G	約 9.1%
輸出投影	(T,d) @ (d,d)	約 1.2 G	約 6.8%
前饋第一層	(T,d) @ (d,d_ff)	約 4.8 G	約 27.3%
前饋第二層	(T,d_ff) @ (d_ff,d)	約 4.8 G	約 27.3%

兩個值得記住的結論。第一，前饋網路占了約 55% 的計算量——大多數人以為注意力最貴，其實在中等序列長度下並不是。第二，注意力那兩項 (QK^T 與 AV) 的 FLOPs 正比於 T²，而其他項正比於 T。所以序列越長，注意力的占比就越高；當 T 遠大於 d 時，它才會成為主導項。

一個實用的粗估公式：一次前向傳播的 FLOPs 約為 2 × 參數量 × 詞元數；訓練（前向加反向）約為 6 × 參數量 × 詞元數。第 17 章會用這個公式把「我有多少張 GPU、跑多久」翻譯成「我可以訓練多大的模型、餵多少 token」。

3.3 機率、最大概似與交叉熵

3.3.1 從第 2 章接上來

第 2 章已經建立了自回歸分解：

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t / w_{<t})$$

現在的問題是：模型有一堆參數 θ ，我們該怎麼調整它們？答案是最大概似估計——選擇讓訓練資料出現機率最大的那組參數。

$$\theta^* = \operatorname{argmax}_{\theta} \prod_{t=1}^T P_{\theta}(w_t / w_{<t})$$

連乘不好處理，取對數變成連加；最大化不好處理，加負號變成最小化。於是得到本書從第 5 章到第 27 章都在最小化的那個東西：

$$L(\theta) = - (1/T) \sum_{t=1}^T \log P_{\theta}(w_t / w_{<t})$$

這個式子有兩個名字。從機率的角度看，它是負對數概似（negative log-likelihood）；從資訊理論

的角度看，它是交叉熵（cross-entropy）。兩者在這個情境下完全等價——這件事讓很多學生困惑，所以下一小節專門說明。

3.3.2 為什麼負對數概似等於交叉熵

交叉熵衡量的是：用分布 q 去編碼來自分布 p 的資料，平均需要多少位元。

$$H(p, q) = -\sum_x p(x) \log q(x)$$

在語言模型中，真實分布 p 是什麼？是一個 one-hot 分布——正確答案的那個詞元機率為 1，其他全為 0。把這個代入上式，連加中只有一項存活：

$$H(p, q) = -1 \cdot \log q(\text{正確詞元}) = -\log P_{\theta}(w_t / w_{\{<t\}})$$

所以在 one-hot 標籤的情況下，交叉熵就是負對數概似，一模一樣。這也解釋了為什麼 PyTorch 的損失函數叫 CrossEntropyLoss，而我們談的是最大概似——它們是同一件事的兩種說法。

順帶一提，這也解釋了困惑度的定義為什麼是 $\exp(\text{loss})$ ：

$$PPL = \exp(L) = \exp(- (1/T) \sum \log P(w_t / w_{\{<t\}}))$$

所以下次你看到訓練日誌上寫 $\text{loss} = 3.2$ ，你可以心算 $\exp(3.2) \approx 24.5$ ，這就是當前的困惑度。這個換算你會用一輩子。

3.3.3 softmax：把任意實數變成機率

模型的輸出是一組任意的實數（稱為 logits），要變成機率分布就要用 softmax：

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

它做三件事：用指數把所有值變成正數、保持大小順序、正規化使總和為 1。

用一個數值例子。假設某個位置的 logits 是 $[2.0, 1.0, 0.1]$ ：

步驟	計算	結果	說明
原始 logits	—	$[2.0, 1.0, 0.1]$	模型的原始輸出，可正可負
取指數	$\exp(z)$	$[7.39, 2.72, 1.11]$	全部變正，差距被放大
求總和	$\sum$	11.21	正規化的分母
相除	$\exp(z)/\sum$	$[0.659, 0.242, 0.099]$	總和為 1，成為機率分布

3.3.4 數值穩定：為什麼不能照公式寫

這是本章最實用的一節。如果你照著上面的公式直接寫程式，在真實模型上它會爆炸。

原因是 logits 可能很大。假設某個 logit 是 1000，那麼 $\exp(1000)$ 在 float32 中會溢位成 inf，接著 inf/inf 得到 nan，整個訓練就毀了。反過來，如果 logits 都很小（例如 -1000）， $\exp$ 會下溢成 0，然後 $0/0$ 也是 nan。

解法是一個小技巧：softmax 對所有 logits 同時減去一個常數是不變的。

$$\text{softmax}(z)_i = \exp(z_i - c) / \sum_j \exp(z_j - c) \quad \cdot \text{對任意常數 } c \text{ 成立}$$

證明只需要把分子分母的 $\exp(-c)$ 約掉。實務上取 $c = \max(z)$ ，這樣最大的指數變成 $\exp(0) = 1$ ，絕對不會溢位；而最小的可能下溢成 0，但那本來就代表機率極小，影響可以忽略。

數值穩定的 softmax 與 log-softmax

```
import numpy as np

def softmax_naive(z):
    """照公式寫，會溢位"""
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def softmax_stable(z):
    """減去最大值，數值安全"""
    z = z - z.max(axis=-1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=-1, keepdims=True)

def log_softmax_stable(z):
    """直接算 log，避免先 exp 再 log 的精度損失"""
    z = z - z.max(axis=-1, keepdims=True)
    return z - np.log(np.exp(z).sum(axis=-1, keepdims=True))

z = np.array([1000.0, 999.0, 998.0])
print(softmax_naive(z))    # [nan nan nan]
print(softmax_stable(z))   # [0.665 0.245 0.090]
```

3.3.5 log-sum-exp：同一個技巧的一般形式

上面那個技巧有一個名字叫 log-sum-exp trick，它的一般形式是：

$$\log \sum_j \exp(z_j) = c + \log \sum_j \exp(z_j - c) \quad \cdot \text{通常取 } c = \max(z)$$

這個式子在很多地方會出現：計算 log-softmax、計算序列的對數機率、在混合模型中合併機率、以及第 27 章的多樣本推理中合併多條路徑的分數。認得它，你會在很多論文的附錄裡少卡住幾次。

一個必須養成的習慣

本書所有實作都要求：只要涉及機率計算，一律在對數空間進行。不要先算出機率再取對數，而是直接算對數機率。理由是浮點數在接近 0 的地方精度極差，而語言模型的機率經常是 10 的負十次方等級。這個習慣會在第 5 章（困惑度）、第 14 章（解碼）、第 25 章（DPO 的偏好比較）反覆用到。

3.3.6 資訊理論的三個量：熵、交叉熵、KL 散度

前面談的交叉熵屬於資訊理論。這裡把三個相關的量一次講清楚，因為它們在第 25 章（DPO 的 KL 正則）與第 26 章（蒸餾）都會出現，屆時再學就來不及了。

量	定義	直覺	在本書中的位置
熵 $H(p)$	$-\sum p \log p$	這個分布本身有多不確定	評估資料的多樣性
交叉熵 $H(p,q)$	$-\sum p \log q$	用 q 去編碼 p 的資料要付多少代價	語言模型的損失函數
KL 散度 $D(p q)$	$\sum p \log(p/q) = H(p,q) - H(p)$	q 偏離 p 有多遠	DPO、蒸餾、對齊的正則項

三者的關係就寫在第三列：交叉熵等於熵加上 KL 散度。由於熵 $H(p)$ 只跟真實分布有關、與模型無關，所以最小化交叉熵等價於最小化 KL 散度。這解釋了一件事：訓練語言模型時，我們表面上在最小化交叉熵，實質上是在讓模型分布逼近真實的語言分布。

要注意 KL 散度不對稱： $D(p||q)$ 不等於 $D(q||p)$ 。這個不對稱性有實際後果——用哪一個方向會導致模型「傾向覆蓋所有模式」或「傾向聚焦單一模式」，在第 26 章的蒸餾中會影響學生模型的行為。現在只要記得這件事存在。

一個具體的數值例子。假設真實分布 $p = [0.7, 0.2, 0.1]$ ，模型分布 $q = [0.5, 0.3, 0.2]$ ：

量	計算	結果	解讀
$H(p)$	$-(0.7 \cdot \ln 0.7 + 0.2 \cdot \ln 0.2 + 0.1 \cdot \ln 0.1)$	約 0.802	真實分布本身的不確定度
$H(p,q)$	$-(0.7 \cdot \ln 0.5 + 0.2 \cdot \ln 0.3 + 0.1 \cdot \ln 0.2)$	約 0.887	模型的損失值
$D(p q)$	$H(p,q) - H(p)$	約 0.085	模型與真實的差距

注意 $H(p) = 0.802$ 是一個下界——即使模型完美 ($q = p$)，損失也只能降到 0.802，不可能到 0。這解釋了為什麼真實訓練中 loss 不會趨近於零：語言本身就有不確定性，「今天天氣很 _」後面本來就可以接好、熱、糟、悶。一個 loss 真的降到接近 0 的語言模型，多半是把訓練資料背起來了——那是記憶化，不是學習（第 16 章會教你怎麼偵測）。

3.4 導數、梯度與反向傳播

3.4.1 導數的工程直覺

對工程師而言，導數只需要一句話就能理解：這個參數動一點點，損失會變多少？

$$\partial L / \partial w \approx (L(w + \varepsilon) - L(w)) / \varepsilon$$

如果 $\partial L / \partial w$ 是正的，代表把 w 調大會讓損失變大，所以我們應該把 w 調小。這就是梯度下降的全部直覺：

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

η 是學習率，決定每次走多大一步。太小走太慢，太大會衝過頭甚至發散。第 15 章會用一整節處理學習率的選擇。

當參數不只一個時，把所有偏導數收集成一個向量，就是梯度。梯度指向損失上升最快的方向，所以我們往它的反方向走。

3.4.2 鏈式法則：把長式子拆成短式子

神經網路是一連串函數的合成。假設 $L = f(g(h(x)))$ ，要算 L 對 x 的導數，鏈式法則說：

$$\partial L / \partial x = (\partial L / \partial f) \cdot (\partial f / \partial g) \cdot (\partial g / \partial h) \cdot (\partial h / \partial x)$$

每一項都是一個簡單函數的導數，而它們相乘就得到整體的導數。這正是「反向傳播」這個名字的由來——梯度從損失端出發，沿著計算圖往回乘。

注意這個連乘結構直接解釋了第 2 章講過的梯度消失：如果每一項都小於 1，乘上幾十層之後就趨近於零。這也解釋了為什麼殘差連接有效——它在連乘中加入了一條「乘以 1」的捷徑。

3.4.3 計算圖與自動微分

PyTorch 在你做每一次運算時，會偷偷記錄下來，形成一張計算圖。當你呼叫 `loss.backward()` 時，它沿著這張圖反向走一遍，用鏈式法則把每個參數的梯度算出來。

為什麼這比手動求導快？考慮一個有 N 個參數的模型。若逐一用有限差分估計每個參數的梯度，需要

N 次前向傳播。而反向傳播只需要一次前向加一次反向，成本大約是兩次前向。當 N 是十億時，這個差距是十億倍。

自動微分的最小示範

```
import torch

w = torch.tensor([2.0], requires_grad=True)
x = torch.tensor([3.0])

y = w * x          # y = 6
z = y ** 2         # z = 36
z.backward()       # 沿計算圖反向

# 手算驗證：dz/dw = 2*y*x = 2*6*3 = 36
print(w.grad)      # tensor([36.])
```

3.4.4 交叉熵對 logits 的梯度：一個漂亮的結果

這是本章唯一一個我希望你親手推導的公式，因為結果出乎意料地簡潔，而且它會讓你對訓練過程產生真正的直覺。

設 logits 為 z 、softmax 輸出為 p 、正確答案的索引為 y 。交叉熵損失為 $L = -\log p_y$ 。可以證明：

$$\partial L / \partial z_i = p_i - 1[i = y]$$

也就是說：梯度等於「模型預測的機率」減去「正確答案的 one-hot」。

這個結果的直覺極為清楚。假設正確答案是第 3 個詞元，而模型給它的機率是 0.2：

詞元	模型機率 p	one-hot	梯度 = $p - \text{onehot}$
詞元 1	0.5	0	+0.5 → 要把這個 logit 調小
詞元 2	0.3	0	+0.3 → 要把這個 logit 調小
詞元 3 (正確)	0.2	1	-0.8 → 要把這個 logit 調大

模型在錯誤答案上給了越多機率，那些答案受到的下壓力就越大；正確答案的機率越低，它受到的上推力就越強。整個訓練過程就是這樣一步一步把機率質量從錯誤答案搬到正確答案上。

順帶一提，這個簡潔的結果也是為什麼實務上要把 softmax 與交叉熵合併成一個函數（PyTorch 的 CrossEntropyLoss 就是這樣做的）。分開算不但慢，還會有額外的數值誤差。你在第 13 章實作時千萬不要自己先做 softmax 再算 log。

3.4.5 梯度檢查：確認你的推導是對的

手寫反向傳播很容易出錯，而且錯了通常不會報錯——它只會讓模型學得比較差，你可能永遠不會發現。梯度檢查是唯一可靠的驗證方式：用有限差分算出數值梯度，跟你的解析梯度比對。

$$\text{數值梯度} \approx (L(w + \epsilon) - L(w - \epsilon)) / (2\epsilon)$$

注意這裡用的是中央差分而不是前向差分，誤差階數從 $O(\epsilon)$ 降到 $O(\epsilon^2)$ ，精度好很多。 ϵ 通常取 $1e-5$ 左右——太大則近似誤差大，太小則浮點誤差大。

ch03/grad_check.py (節錄)

```
def grad_check(f, w, analytic_grad, eps=1e-5, tol=1e-6):
    """f: 接受 w 回傳純量損失; analytic_grad: 你算出的梯度"""
    num_grad = np.zeros_like(w)
    it = np.nditer(w, flags=["multi_index"])
    while not it.finished:
        i = it.multi_index
        old = w[i]
        w[i] = old + eps; f_plus = f(w)
        w[i] = old - eps; f_minus = f(w)
        w[i] = old
        num_grad[i] = (f_plus - f_minus) / (2 * eps)
        it.iternext()

    # 用相對誤差，而非絕對誤差
    denom = np.maximum(np.abs(num_grad) + np.abs(analytic_grad), 1e-8)
    rel_err = np.abs(num_grad - analytic_grad) / denom
    worst = rel_err.max()
    print(f"最大相對誤差 {worst:.2e} {'通過' if worst < tol else '失敗'}")
    return worst < tol
```

判讀梯度檢查的結果

相對誤差小於 $1e-7$ 表示幾乎確定正確； $1e-7$ 到 $1e-4$ 之間通常可接受，尤其在有 ReLU 這類不可微點的網路中；大於 $1e-2$ 幾乎確定有 bug。另外要注意：如果你的損失函數有 dropout 或其他隨機成分，必須先把它關掉再做梯度檢查，否則兩次前向傳播算的根本不是同一個函數。

3.4.6 常用運算的反向公式速查

手寫反向傳播時，你需要知道每種運算的梯度怎麼傳。下表列出大模型中最常出現的幾種，第 11 到 13 章實作時可以直接查。

前向運算	形狀變化	對輸入的梯度	注意事項
$y = x @ W$	$(N,d) @ (d,e) \rightarrow (N,e)$	$dx = dy @ W^T$; $dW = x^T @ dy$	兩個梯度的形狀要各自對得上
$y = x + b$	廣播加 bias	$dx = dy$; $db = dy.sum(axis=0)$	廣播的反向是求和，這是最常錯的一項

前向運算	形狀變化	對輸入的梯度	注意事項
$y = \text{ReLU}(x)$	形狀不變	$dx = dy \cdot 1[x > 0]$	注意是 $x > 0$ 而非 $y > 0$
$y = \tanh(x)$	形狀不變	$dx = dy \cdot (1 - y^2)$	可以用輸出算，省一次計算
$y = \text{softmax}(x)$	形狀不變	見 3.4.4 節（與交叉熵合併）	不要單獨算 softmax 的 Jacobian
$y = x.\text{reshape}(\dots)$	形狀改變	$dx = dy.\text{reshape}(x.\text{shape})$	reshape 的反向就是 reshape 回去
$y = x.\text{transpose}(a,b)$	維度交換	$dx = dy.\text{transpose}(a,b)$	轉置的反向是同樣的轉置
$y = x.\text{sum}(\text{axis}=k)$	第 k 維消失	$dx = dy$ 沿第 k 維廣播回去	求和的反向是廣播，與第二列互為對偶

請注意第二列與最後一列的對稱性：廣播的反向是求和，求和的反向是廣播。這不是巧合，而是線性代數中轉置關係的體現。認出這個模式之後，你會發現手寫反向傳播比想像中規律。

第三列的「 $x > 0$ 而非 $y > 0$ 」是一個經典陷阱。ReLU 的輸出 y 在 $x \leq 0$ 時是 0，所以用 $y > 0$ 判斷在數值上碰巧也對；但如果你在某處對 y 做了 in-place 修改，判斷就會錯。養成用輸入判斷的習慣比較安全。

3.5 最佳化器：從 SGD 到 AdamW

有了梯度，接下來的問題是：怎麼用它更新參數？這個問題的答案在過去十年演化了好幾代，而今天訓練大模型幾乎清一色使用 AdamW。這一節說明為什麼。

3.5.1 SGD：最單純的那一個

$$w \leftarrow w - \eta \cdot g \quad , \text{其中 } g = \partial L / \partial w$$

沿著梯度反方向走一步。簡單、記憶體占用最低（除了參數本身不需要額外儲存），但有兩個問題。

第一，它對所有參數用同一個學習率。但在神經網路中，不同層、不同參數的梯度尺度可能差好幾個數量級——嵌入層的梯度通常很稀疏很小，而輸出層的梯度很大。用同一個 η 必然顧此失彼。

第二，它在狹長的損失谷地中會來回震盪。想像一個像山谷的損失曲面：沿著谷底方向坡度很緩，橫越谷的方向很陡。梯度主要指向橫越方向，於是模型在兩側壁面之間來回彈跳，往谷底前進得極慢。

3.5.2 Momentum：給它一點慣性

$$v \leftarrow \beta \cdot v + g \quad ; \quad w \leftarrow w - \eta \cdot v$$

把過去的梯度累積成一個速度向量 v ， β 通常取 0.9。直覺是把「當前梯度」換成「最近梯度的指數移動平均」。

這解決了震盪問題：來回震盪的分量方向不斷改變，累加後會互相抵消；而穩定指向谷底的分量會不斷累積，越走越快。用物理的比喻，就是把一個在坡上滑動的質點換成一顆有質量的球。

3.5.3 Adam：每個參數有自己的學習率

Adam 同時追蹤梯度的一階動量（平均）與二階動量（平方的平均）：

$$m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g \quad ; \quad v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g^2$$

然後做偏差修正（因為 m 與 v 初始化為 0，早期會偏小）：

$$\hat{m} = m / (1 - \beta_1^t) \quad ; \quad \hat{v} = v / (1 - \beta_2^t)$$

最後更新：

$$w \leftarrow w - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

關鍵在分母的 $\sqrt{\hat{v}}$ ：它是這個參數最近梯度的均方根。梯度一直很大的參數，分母大，實際步伐被縮小；梯度一直很小的參數，分母小，實際步伐被放大。等於每個參數都有了自己的自適應學習率。

β_1 通常取 0.9、 β_2 取 0.999（大模型有時用 0.95）、 ϵ 取 $1e-8$ 。 ϵ 的存在純粹是為了避免除以零，但它的大小在低精度訓練時會有影響——第 18 章談 BF16 時會再提。

3.5.4 AdamW：把權重衰減修對

權重衰減（weight decay）是一種正則化：讓參數傾向於保持小的數值，避免過擬合。傳統做法是在損失函數裡加一項 $\lambda|w|^2$ ，這在 SGD 上等價於在梯度裡加 λw 。

但在 Adam 上這個等價關係壞掉了。因為加進梯度的那個 λw 也會被 $\sqrt{\hat{v}}$ 除，導致衰減強度被自適應學習率扭曲——梯度大的參數反而衰減得少，這與正則化的本意相反。

AdamW 的修正很簡單：把權重衰減從梯度中拿出來，直接作用在參數上。

$$w \leftarrow w - \eta \cdot (\hat{m} / (\sqrt{\hat{v}} + \epsilon) + \lambda \cdot w)$$

這個看似微小的改動，在實務上對泛化能力有明顯影響，因此今天所有大模型訓練都用 AdamW 而非 Adam。這是一個很好的例子：一個數學上的細節錯誤，可以隱藏很多年才被發現與修正。

3.5.5 四者對照與記憶體代價

最佳化器	核心想法	每個參數的額外狀態	記憶體倍數	適用
SGD	直接沿梯度反方向	無	1× (僅參數)	記憶體極度受限、微調小模型
SGD + Momentum	累積速度向量	1 個 (v)	2×	視覺模型、部分微調
Adam	一階與二階動量自適應	2 個 (m, v)	3×	一般深度學習
AdamW	Adam 加上解耦的權重衰減	2 個 (m, v)	3×	大模型訓練的標準選擇

最右邊那兩欄在大模型訓練中極為重要。用 AdamW 訓練一個 7B 參數的模型，光是參數本身 (FP32) 就要 28 GB，加上梯度 28 GB，再加上 m 與 v 各 28 GB，總共 112 GB——遠超過單張 GPU 的記憶體。這就是為什麼第 19 章要講 ZeRO 分片、第 24 章要講 LoRA。

先做一個小練習，你在課後習題會做完整版：一個 1.5B 參數的模型，用 AdamW 在 FP32 下訓練，光是「參數 + 梯度 + 最佳化器狀態」需要多少記憶體？答案是 $1.5 \times 4 \times 4 = 24$ GB，還沒算 activation。這就是為什麼「我的顯卡有 24 GB，應該可以訓練 1.5B 吧」這個直覺是錯的。

3.5.6 學習率排程的預覽

學習率不是設定一次就不動的。標準做法是三個階段：

階段	做什麼	為什麼	典型比例
warmup	從接近 0 線性升到目標值	訓練初期梯度不穩，大步伐容易發散	總步數的 1–5%
穩定 / 衰減	維持高學習率或緩慢下降	主要學習發生在這個階段	總步數的 80–90%
最終衰減	降到接近 0	讓模型收斂到一個較平坦的極小值	總步數的 10–20%

常見的排程形狀包括 cosine (餘弦下降) 與 WSD (warmup-stable-decay)。第 15 章會完整處理排程設計與學習率探測，這裡只需要知道：如果你的訓練在前幾百步就發散，第一個要檢查的就是有

沒有 warmup。

3.5.7 初始化：訓練開始之前就決定了一半

最佳化器決定「怎麼走」，初始化決定「從哪裡出發」。這件事的重要性常被低估——一個壞的初始化會讓再好的最佳化器也救不回來。

核心的考量是：讓每一層的輸出變異數與輸入變異數大致相同。如果每層都把訊號放大 1.5 倍，二十層之後就放大了三千多倍；如果每層縮小到 0.7 倍，二十層之後只剩千分之一。兩種都會讓訓練失敗。

方法	權重的標準差	適用	理由
Xavier / Glorot	$\sqrt{2 / (\text{fan_in} + \text{fan_out}))}$	tanh、sigmoid	同時考慮前向與反向的變異數
He / Kaiming	$\sqrt{2 / \text{fan_in}}$	ReLU 及其變體	ReLU 砍掉一半訊號，需補回倍數
GPT 式縮放	$0.02 \cdot \text{殘差分支再除以 } \sqrt{2L}$	Transformer	層數越深，每個殘差分支貢獻應越小
嵌入層	通常 0.02 或較小	所有模型	嵌入不經過非線性，尺度需與後續匹配

第三列的 $\sqrt{2L}$ 縮放值得特別說明。在一個 L 層的 Transformer 中，每一層都會把自己的輸出加到殘差流上。如果每個分支的輸出變異數都是 1，那麼經過 L 層之後殘差流的變異數會累積到 L 左右——層數越深，訊號越失控。把每個殘差分支的輸出權重除以 $\sqrt{2L}$ ，可以讓總變異數維持在合理範圍。這是 GPT-2 論文中一個看似微小、實則關鍵的細節，第 12 章實作時你會用到。

一個實用的診斷方法：訓練開始前，先做一次前向傳播，印出每一層輸出的標準差。如果它們大致在同一個量級（例如都在 0.5 到 2 之間），初始化大概沒問題；如果逐層放大或縮小，就要回頭檢查。這只花你三十秒，卻能省下數小時的無效訓練。

3.5.8 批次大小、梯度雜訊與有效批次

最後一個與最佳化直接相關的概念。批次大小不只是「一次處理幾筆資料」，它決定了梯度估計的雜訊程度。

我們真正想要的是整個資料集上的梯度，但那太貴，所以用一個小批次去估計。估計的標準誤大致與批次大小的平方根成反比：

梯度雜訊 $\propto 1/\sqrt{B}$

批次從 32 增加到 128 (4 倍)，雜訊只降到一半 ($\sqrt{4} = 2$)。這個平方根關係有兩個實務後果。

情境	現象	處置	章節
批次太小	loss 曲線抖動劇烈，難以判斷是否真的在進步	增大批次或用梯度累積	第 15 章
批次太大	每步變貴但收斂不成比例地變快，算力浪費	找到臨界批次大小後不再增加	第 15、17 章
記憶體不足	想要大批次但 VRAM 放不下	梯度累積：算 N 次小批次，累加後才更新一次	第 15 章
多卡訓練	每張卡各算一部分，同步後平均	資料平行，等效於單卡的大批次	第 19 章

梯度累積的等價關係值得寫清楚，因為第 15 章的程式會直接用到：批次 8 加上累積 4 步，其有效批次為 32，梯度與直接用批次 32 幾乎相同（差別只在 BatchNorm 這類跨樣本統計的層，而 Transformer 通常不用 BatchNorm，所以完全等價）。這讓你能在小顯卡上模擬大批次訓練，代價只是時間。

另外一個常被忽略的細節：改變批次大小時，學習率通常也要跟著調。常見的經驗法則是線性縮放（批次加倍、學習率加倍）或平方根縮放。哪一個比較好取決於訓練規模，第 15 章會用實驗讓你自己量。

3.6 數值穩定性：浮點數的真相

3.6.1 浮點數不是實數

這一節可能是本章最違反直覺的部分。電腦裡的浮點數是實數的一個離散近似，它的性質與你在數學課學的實數不同。

兩個最重要的差異：第一，浮點加法不滿足結合律， $(a+b)+c$ 不一定等於 $a+(b+c)$ 。第二，浮點數的精度不是均勻的——在 0 附近非常密集，在大數附近非常稀疏。

三個會讓你不安的例子

```
>>> 0.1 + 0.2 == 0.3
False                                     # 十進位小數無法用二進位精確表示

>>> 1e16 + 1 == 1e16
```

```
True                                     # 1 相對於 1e16 太小，被吃掉了

>>> import numpy as np
>>> x = np.float32(1e-8)
>>> (1.0 + x) - 1.0
0.0                                     # 精度不足，資訊完全消失
```

第二個例子在大模型訓練中有直接後果：當你把上百萬個梯度加總時，如果先加的都是大數、後加的都是小數，那些小數可能完全被忽略。這就是為什麼函式庫會用 Kahan 求和或分段求和等技巧。

3.6.2 常見的浮點格式

格式	總位元	指數位元	尾數位元	特性	用途
FP32	32	8	23	範圍與精度都足夠，記憶體大	傳統訓練、 master weights
TF32	19 (存為 32)	8	10	範圍同 FP32，精度較低	NVIDIA GPU 的 矩陣乘法
FP16	16	5	10	範圍窄，容易溢位，需 loss scaling	早期混合精度
BF16	16	8	7	範圍同 FP32，精度較低但夠 用	現代大模型訓練 的主流
FP8	8	4 或 5	3 或 2	極省記憶體，需仔細校準	最新硬體的訓練 與推論

這張表最重要的一列是 BF16。它的關鍵在於指數位元與 FP32 相同（都是 8 位），所以數值範圍一樣大，不會溢位；犧牲的是尾數精度。而深度學習的經驗是：範圍比精度重要得多。這就是為什麼 BF16 取代了 FP16 成為主流。

3.6.3 訓練中的四種數值災難

現象	徵兆	成因	處置
梯度爆炸	loss 突然變成極大值或 nan；梯度範數暴增	學習率過大、初始化不當、資 料異常	梯度裁切、降學習率、加 warmup
梯度消失	loss 停滯不動；深層的梯度	深層網路、激勵函數飽和、缺	殘差連接、Pre-LN、換激勵函

現象	徵兆	成因	處置
	接近 0	殘差	數
數值溢位	inf 或 nan 出現在 forward	exp 未做穩定化、FP16 範圍不足	log-sum-exp、改用 BF16、loss scaling
精度損失	訓練曲線不平滑；結果無法重現	大量小數累加、混合精度處理不當	FP32 master weights、確認 reduce 的精度

第 15 章的程式 15C (訓練哨兵) 會實作一個即時監控器，自動偵測這四種狀況並在發生前告警。現在你只需要記得徵兆長什麼樣。

3.6.4 梯度裁切

最常用的防爆炸手段。做法是：算完梯度後，如果整體的範數超過閾值，就等比例縮小。

$$\text{if } \|g\| > c \quad \text{then} \quad g \leftarrow g \cdot c / \|g\|$$

注意這是按整體範數縮放，不是逐元素截斷——後者會改變梯度的方向，前者只改變步伐大小。c 通常取 1.0。

一個很實用的診斷技巧：即使你不打算裁切，也應該記錄每一步的梯度範數。它是訓練健康度最靈敏的指標——在 loss 出問題之前的幾十步，梯度範數往往就已經開始異常了。

3.6.5 混合精度：一個工程上的折衷

既然低精度省記憶體又快，為什麼不全部用 BF16？因為有些運算對精度敏感，全低精度會讓訓練品質下降甚至無法收斂。實務上的做法是混合精度：不同的運算用不同的精度。

項目	精度	理由	記憶體影響
前向與反向的矩陣乘法	BF16	這裡最耗時，低精度收益最大	大幅降低
activation	BF16	量最大，且對精度不敏感	大幅降低
master weights (主權重)	FP32	參數更新量常常極小，低精度會被吃掉	維持不變
最佳化器狀態 m、v	FP32	累積量，誤差會不斷放大	維持不變

項目	精度	理由	記憶體影響
loss 與 reduce 運算	FP32	大量數字相加，精度損失明顯	影響很小

第三列是關鍵。回想 3.6.1 節那個例子：1.0 加上 $1e-8$ 在 float32 中等於 1.0。訓練後期參數的更新量常常就是這個量級，如果參數用 BF16（精度更低）儲存，更新會完全消失，模型就停止學習了。所以即使計算用 BF16，也必須保留一份 FP32 的主權重來累積更新。

這也解釋了為什麼 3.5.5 節那張記憶體表要分開列出——實際訓練的記憶體不是簡單的「參數量乘以精度」，而是多份不同精度的副本加總。一個 7B 模型在混合精度下的典型配置是：BF16 參數 14 GB、BF16 梯度 14 GB、FP32 主權重 28 GB、FP32 的 m 與 v 各 28 GB，合計約 112 GB，跟純 FP32 差不多——省下的主要是 activation 與計算時間，不是最佳化器狀態。

一個常見的誤解

「用 BF16 訓練可以把記憶體需求砍半」這句話只對了一部分。它確實把 activation 與部分中間值砍半，但最佳化器狀態通常仍是 FP32，而那才是大宗。真正能大幅降低記憶體的是第 19 章的 ZeRO 分片與第 24 章的 LoRA——前者把狀態分散到多張卡，後者根本不訓練原本的參數。

3.7 高中生友善路徑：用數值實驗取代形式證明

這一節是給數學基礎較弱的讀者，以及所有想在教高中生時使用本書的老師。核心主張是：在工程領域，能程式驗證的事情，不一定要先會證明。

3.7.1 三個可以直接跑的替代路徑

原本需要的數學	可替代的數值實驗	學到的東西
證明 softmax 平移不變	用不同的 c 值跑一次，看結果是否相同	知道這件事成立，且知道怎麼驗證
推導交叉熵梯度	用有限差分算出來，跟公式對照	同時學會梯度檢查這項實用技能
證明矩陣乘法結合律	用隨機矩陣測 100 次，看誤差有多小	順便發現浮點數不完全滿足結合律
理解梯度消失	畫出 0.9 的 n 次方曲線	得到比證明更強的直覺
理解 Adam 的自適應	畫出兩個參數在不同最佳化器下的軌跡	看到震盪與收斂的實際樣子

3.7.2 什麼時候必須回頭補數學

但我也必須誠實：有三種情況你躲不掉。

1. 當你要讀論文並判斷它的主張是否成立時。論文不會為你畫圖，它假設你看得懂式子。
2. 當你的實驗結果與預期不符，而你必須從理論推測原因時。這時直覺不夠用，你需要能算。
3. 當你要設計一個新方法而非套用現成方法時。這是研究生與工程師的分水嶺。

所以我的建議是：先用數值實驗建立直覺、把系統做出來，然後在遇到上述三種情況時，帶著具體問題回頭補數學。這樣學到的東西會黏得住，因為它有掛鉤。

3.7.3 高中課程的建議切法

節次	高中處理方式	需要的先備	省略的部分
3.1 張量與形狀	完整教，這是最重要的	只需要理解陣列	無
3.2 矩陣運算	教內積與矩陣乘法的意義，用圖解	國中的座標概念	einsum 可略
3.3 機率與交叉熵	教到 softmax 的數值例子為止	機率的基本概念	最大概似的形式推導
3.4 梯度	用「往下坡走」的比喻與一維例子	斜率的概念	鏈式法則的形式證明
3.5 最佳化器	只看軌跡圖，不看公式	無	所有公式
3.6 數值穩定	完整教，這是很好的震撼教育	無	浮點格式細節可略

一個教學觀察

3.6.1 節那三個浮點數的例子，在高中課堂上效果驚人。學生從小被教「電腦算得很準」，突然發現 $0.1 + 0.2$ 不等於 0.3 ，那個震撼會讓他們認真起來。我建議把這一段放在課程的前段而不是後段。

3.7.4 臺灣工程教育現場的一個觀察

這一節是寫給教師與自學者的，也是本書把整整一章放在數學補強的理由。

臺灣的工程科系學生通常修過微積分與線性代數，理論上具備本章所需的一切。但實際教學經驗顯示，多數學生在面對大模型的程式碼時仍然卡住，原因不是不會微分，而是三個銜接上的斷層。

斷層	具體表現	本章的對策
符號與程式的落差	看得懂 $\partial L / \partial w$ ，但不知道它在程式中對應哪一行	每個公式配一段可執行程式
低維與高維的落差	會算二維向量的內積，看到 (B,H,T,d_h) 就迷路	形狀推理與斷言的訓練
理想與實作的落差	知道 softmax 的定義，不知道它會溢位	數值穩定性獨立成節

第三個斷層特別值得注意，因為傳統數學課完全不會教。在數學裡 $\exp(1000)$ 是一個很大的數；在電腦裡它是 inf ，而 inf 除以 inf 是 nan ，你的六小時訓練就毀了。這不是數學問題，是工程問題，而它只在工程課裡才會被教。

我的建議是：不要因為學生「修過微積分」就跳過本章。相反地，正因為他們修過，本章才能講得快——你不必教什麼是導數，只需要教導數在張量世界裡長什麼樣、在浮點數世界裡會出什麼錯。這兩件事加起來，大約就是本章的全部內容。

程式實作

本章三個實作全部在 A 級硬體（一般筆電）上完成，不需要 GPU。程式碼位於課程倉庫的 ch03/ 目錄。

程式 3A 只用 NumPy 實作數值穩定的 softmax 與交叉熵

目標：親手實作並與 PyTorch 逐位元比對。這個實作的價值不在於「寫出來」，而在於「證明你寫的跟工業級實作一致」——這是本書第一次做交叉驗證，往後每一章都會用到。

ch03/lab3a_softmax_ce.py (節錄)

```
import numpy as np
import torch
import torch.nn.functional as F

def log_softmax(z, axis=-1):
    """數值穩定版 log-softmax"""
    z_max = z.max(axis=axis, keepdims=True)
    z_shift = z - z_max
    lse = np.log(np.exp(z_shift).sum(axis=axis, keepdims=True))
    return z_shift - lse

def cross_entropy(logits, targets):
    """logits: (N, V) targets: (N,) 為整數索引"""
    N = logits.shape[0]
    logp = log_softmax(logits, axis=-1)          # (N, V)
    picked = logp[np.arange(N), targets]         # (N,)
    return -picked.mean()

def cross_entropy_grad(logits, targets):
    """回傳 dL/dlogits, 形狀 (N, V)"""
    N, V = logits.shape
    p = np.exp(log_softmax(logits, axis=-1))    # softmax
    onehot = np.zeros_like(p)
    onehot[np.arange(N), targets] = 1.0
    return (p - onehot) / N                    # 見 3.4.4 節

def test_against_pytorch(seed=0, N=8, V=50, scale=1.0):
    rng = np.random.default_rng(seed)
    logits = rng.normal(0, scale, size=(N, V))
    targets = rng.integers(0, V, size=(N,))

    mine = cross_entropy(logits, targets)
    tl = torch.tensor(logits, requires_grad=True)
    ref = F.cross_entropy(tl, torch.tensor(targets))
    ref.backward()

    assert abs(mine - ref.item()) < 1e-10, (mine, ref.item())
    g_mine = cross_entropy_grad(logits, targets)
    assert np.allclose(g_mine, tl.grad.numpy(), atol=1e-10)
    print(f"scale={scale:<6} loss={mine:.8f} 與 PyTorch 一致")

if __name__ == "__main__":
    for s in [1.0, 10.0, 100.0, 1000.0]:      # 逐步逼近溢位邊界
        test_against_pytorch(scale=s)
```

思考題與延伸挑戰：

- 把 `log_softmax` 改成不減 `z_max` 的版本，在 `scale = 1000` 時會發生什麼？請觀察並解釋。
- 實作一個 label smoothing 版本的交叉熵（把 one-hot 換成 0.9 與 $0.1/(V-1)$ 的混合），並推導它的梯度。這在第 23 章的微調中偶爾會用到。
- 量測 $N = 8$ 、 $V = 32000$ 時，你的 NumPy 版本與 PyTorch 版本的執行時間差距，並解釋原因。

程式 3B 梯度檢查與抓蟲

目標：課程倉庫提供一份刻意植入錯誤的反向傳播程式碼，你要用梯度檢查找出是哪一層、哪一個運算寫錯了。這是本書設計的第一個「除錯訓練」，之後每一篇都會有一個。

ch03/lab3b_buggy_mlp.py (提供給學生的版本，含一個錯誤)

```
class TinyMLP:
    """兩層 MLP: x -> Linear -> ReLU -> Linear -> CrossEntropy"""
    def __init__(self, d_in, d_hid, d_out, seed=0):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, 0.1, (d_in, d_hid))
        self.b1 = np.zeros(d_hid)
        self.W2 = rng.normal(0, 0.1, (d_hid, d_out))
        self.b2 = np.zeros(d_out)

    def forward(self, x):
        self.x = x
        self.z1 = x @ self.W1 + self.b1
        self.a1 = np.maximum(0, self.z1)          # ReLU
        self.z2 = self.a1 @ self.W2 + self.b2
        return self.z2

    def backward(self, dz2):
        self.dW2 = self.a1.T @ dz2
        self.db2 = dz2.sum(axis=0)
        da1 = dz2 @ self.W2.T
        dz1 = da1 * (self.z1 > 0)                # ← 這裡有一行是錯的
        self.dW1 = self.x.T @ dz1
        self.db1 = dz1.sum(axis=0)
        return dz1 @ self.W1.T
```

除錯流程（請照這個順序，這是專業做法）：

1. 先對每一個參數獨立做梯度檢查，找出哪一個參數的梯度不對。
2. 若 $W1$ 錯而 $W2$ 對，代表問題出在反向傳播的中段，而非最後一層。
3. 把中間變數（ $da1$ 、 $dz1$ ）也拿去做梯度檢查，逐步縮小範圍。
4. 找到錯誤後，寫一個回歸測試，確保這個錯誤不會再出現。

延伸挑戰：自己在程式中植入三種不同類型的錯誤（維度轉置錯、忘記除以 batch size、activation

導數用錯)，讓同學互相除錯。設計錯誤比找錯誤更能建立理解。

程式 3C 最佳化器競技場

目標：在同一個二維損失曲面上，視覺化四種最佳化器的軌跡。用二維是因為可以畫出來——所有直覺都來自看得見的東西。

ch03/lab3c_optimizer_arena.py (節錄)

```
import numpy as np
import matplotlib.pyplot as plt

def rosenbrock(w, a=1.0, b=100.0):
    """經典的狹長谷地函數，最小值在 (1, 1)"""
    x, y = w
    return (a - x) ** 2 + b * (y - x ** 2) ** 2

def rosenbrock_grad(w, a=1.0, b=100.0):
    x, y = w
    dx = -2 * (a - x) - 4 * b * x * (y - x ** 2)
    dy = 2 * b * (y - x ** 2)
    return np.array([dx, dy])

def run(opt_name, w0, steps=500, lr=1e-3):
    w = np.array(w0, dtype=np.float64)
    m = np.zeros(2); v = np.zeros(2); traj = [w.copy()]
    b1, b2, eps, lam = 0.9, 0.999, 1e-8, 0.0

    for t in range(1, steps + 1):
        g = rosenbrock_grad(w)
        if opt_name == "sgd":
            w -= lr * g
        elif opt_name == "momentum":
            m = 0.9 * m + g
            w -= lr * m
        elif opt_name in ("adam", "adamw"):
            m = b1 * m + (1 - b1) * g
            v = b2 * v + (1 - b2) * g ** 2
            mh = m / (1 - b1 ** t)
            vh = v / (1 - b2 ** t)
            step = mh / (np.sqrt(vh) + eps)
            if opt_name == "adamw":
                step = step + lam * w # 解耦權重衰減 (見 3.5.4)
            w -= lr * step
        traj.append(w.copy())
    return np.array(traj)

for name in ["sgd", "momentum", "adam", "adamw"]:
    tr = run(name, w0=[-1.5, 2.0])
    final = rosenbrock(tr[-1])
    print(f"{name:<10} 終點 ({tr[-1][0]:.4f}, {tr[-1][1]:.4f}) 損失 {final:.6f}")
    plt.plot(tr[:, 0], tr[:, 1], label=name, lw=1.2)
```

你應該觀察到的現象：

- SGD 在谷地中來回震盪，前進緩慢；把學習率調大會直接發散。這是 3.5.1 節那段描述的視覺化版本。

- Momentum 的軌跡比較平順，但在轉彎處會衝過頭——慣性的代價。
- Adam 前期收斂最快，但在接近極小值時可能出現細微震盪。
- 把 AdamW 的 λ 設為非零值，觀察軌跡如何被拉向原點——這就是正則化的幾何意義。

一個重要的提醒

二維玩具問題的結論不能直接套用到真實訓練。Rosenbrock 函數上 Adam 最快，不代表在你的 7B 模型上 Adam 也最好。這個實驗的目的是建立幾何直覺，不是選最佳化器。真實的選擇必須用第 16 章的實驗方法在你自己的任務上量測。

系統案例：Attention 張量尺寸追蹤器

本章的系統案例是製作一個互動式教學工具，讓使用者調整參數後，即時看到注意力機制中每一步的張量形狀與記憶體占用。這個工具你會在第 11 章教學生時用到，也可以放進你的專題展示中。

功能規格

功能	說明	教學價值
參數輸入	B、T、d、H、L 五個滑桿或輸入框	讓使用者親手感受每個參數的影響
形狀表	逐步列出每一步的張量形狀與運算	把 3.1.2 節那張表變成動態的
記憶體估算	每一步的張量大小 (MB)，並標出最大者	看見注意力矩陣為何是記憶體瓶頸
複雜度曲線	T 變化時，計算量與記憶體的成長曲線	視覺化平方複雜度
錯誤偵測	當 d 無法被 H 整除時給出明確訊息	示範好的錯誤訊息長什麼樣

核心計算

記憶體估算的公式 (以 BF16 每個數 2 bytes 計)：

張量	形狀	元素數	備註
輸入 x	(B, T, d)	$B \cdot T \cdot d$	相對很小
Q / K / V 各一	(B, H, T, d_h)	$B \cdot T \cdot d$ 各一	合計三倍輸入大小
注意力分數	(B, H, T, T)	$B \cdot H \cdot T^2$	主要瓶頸，隨 T 平方成長

張量	形狀	元素數	備註
softmax 後	(B, H, T, T)	$B \cdot H \cdot T^2$	若不做 in-place 會再多一份
輸出	(B, T, d)	$B \cdot T \cdot d$	與輸入同大小

請自己算一次： $B = 4$ 、 $H = 32$ 、 $T = 4096$ 時，單一層的注意力分數矩陣有多大？答案是 $4 \times 32 \times 4096^2 \times 2 \text{ bytes} \approx 4.3 \text{ GB}$ ——而這只是一層，且還沒算反向傳播需要保留的中間值。這個數字會讓你立刻理解為什麼 FlashAttention (第 18 章) 如此重要：它的核心貢獻正是避免把完整的 $T \times T$ 矩陣寫進記憶體。

交付要求

- 可執行的工具（網頁、Notebook 或命令列皆可），能正確處理至少五組參數組合。
- 一份對照驗證：用 PyTorch 實際跑一次，比對你的估算與 `torch.cuda.max_memory_allocated` 的差距，並解釋差異來源。
- 一段五分鐘的教學說明，能讓一位沒學過的同學理解「為什麼長上下文很貴」。

章末評量

一、概念題

1. 用自己的話說明形狀推理的三條規則，並各舉一個大模型中的實例。
2. 為什麼內積可以用來衡量兩個向量的相似度？餘弦相似度與內積的差別是什麼，什麼時候應該用哪一個？
3. 解釋為什麼負對數概似與交叉熵在語言模型的情境下是同一件事。這個等價關係需要什麼前提？
4. 為什麼實務上一律在對數空間計算機率？請舉一個具體會出錯的例子。
5. Adam 的分母 $\sqrt{v}$ 扮演什麼角色？為什麼說它讓每個參數有了自己的學習率？
6. AdamW 修正了 Adam 的什麼問題？為什麼這個問題在 SGD 上不存在？

二、shape tracing 十題

以下每題給定輸入形狀與運算，請寫出輸出形狀。設 $B=4$ 、 $T=128$ 、 $d=768$ 、 $H=12$ 、 $d_h=64$ 、 $V=32000$ 、 $d_{ff}=3072$ 。

1. x 形狀 (B, T, d) ， W 形狀 (d, d_{ff}) ，求 $x @ W$ 。
2. h 形狀 (B, T, d_{ff}) ， W_2 形狀 (d_{ff}, d) ，求 $h @ W_2$ 。
3. q 形狀 (B, T, d) ，reshape 成多頭後再 $\text{transpose}(1,2)$ ，求結果形狀。
4. q 形狀 (B, H, T, d_h) ， k 形狀 (B, H, T, d_h) ，求 $q @ k.\text{transpose}(-2,-1)$ 。
5. scores 形狀 (B, H, T, T) ，對最後一維做 softmax ，求輸出形狀。
6. attn 形狀 (B, H, T, T) ， v 形狀 (B, H, T, d_h) ，求 $\text{attn} @ v$ 。
7. out 形狀 (B, H, T, d_h) ， $\text{transpose}(1,2)$ 後 reshape 成三階，求形狀。
8. h 形狀 (B, T, d) ， lm_head 形狀 (d, V) ，求 logits 形狀。
9. logits 形狀 (B, T, V) ， labels 形狀 (B, T) ，計算交叉熵時需先 reshape 成什麼？
10. mask 形狀 (T, T) ，要與 $\text{scores}(B, H, T, T)$ 相加，需要如何調整 mask 的形狀？請寫出兩種寫法。

三、推導與計算題

1. 證明 softmax 對輸入同時減去任意常數 c 的結果不變。
2. 推導交叉熵對 logits 的梯度為 $p - \text{onehot}$ 。（提示：分成 $i = y$ 與 $i \neq y$ 兩種情況處理。）
3. 一個 3B 參數的模型，用 AdamW 在 FP32 下訓練。計算參數、梯度、 m 、 v 四者合計需要多少 GB。若改用 BF16 存參數與梯度、FP32 存最佳化器狀態，變成多少？
4. 某層的 logits 為 $[5.0, 3.0, 2.0, 8.0]$ ，正確答案為索引 0。計算 softmax 機率、交叉熵損失，以及

對每個 logit 的梯度。

5. 若某 RNN 每步梯度衰減 0.92，計算傳遞 30 步後剩下的比例。若改成有殘差連接的結構（可視為部分路徑衰減係數為 1），定性說明差異。

四、除錯題

1. 同學的 softmax 在測試小數值時正確，但套用到真實模型後輸出 nan。請說出最可能的原因與三行修正。
2. 同學的梯度檢查顯示 W2 的相對誤差是 $3e-8$ （通過），但 W1 是 0.47（失敗）。請說明錯誤最可能在哪一段程式碼，以及下一步該檢查什麼。
3. 同學把 mask 形狀寫成 (T_i) 而非 (T, T) ，程式沒有報錯就跑起來了。請說明廣播在這裡做了什麼，以及後果是什麼。
4. 一個訓練在前 50 步 loss 從 10.5 降到 4.2，第 51 步突然變成 nan。請列出四個可能原因與各自的檢查方式。

五、系統設計題

1. 你要為一個團隊撰寫「張量形狀規範」文件，規定所有程式碼必須遵守的形狀標記與斷言慣例。請寫出這份規範的五條規則，並各附一個正例與反例。
2. 你的 GPU 有 24 GB VRAM，想微調一個 7B 模型。請估算全參數微調（AdamW、BF16）需要多少記憶體，說明為什麼不可行，並列出三種讓它可行的方向（每一種都要說出代價）。

六、臺灣情境與倫理題

1. 本章的實作都在筆電上可完成，但第 15 章之後的實驗需要 GPU。請為你的學校設計一個「讓沒有 GPU 的學生也能完整修完這門課」的方案，涵蓋教學設計、資源分配與作業設計三個層面。
2. 有人主張「反正框架都幫你算好梯度了，不需要學反向傳播」。請提出至少三個反駁，以及一個這個主張有道理的地方。

評量提示（給自學者）

- shape tracing 第 9 題：交叉熵需要 (N, V) 與 (N_i) ，所以 logits 要 reshape 成 $(B \cdot T, V)$ 、labels 成 $(B \cdot T_i)$ 。這是第 13 章實作時最常見的錯誤點。
- shape tracing 第 10 題：兩種寫法分別是 `mask[None, None, :, :]` 與 `mask.unsqueeze(0).unsqueeze(0)`，結果都是 $(1, 1, T, T)$ ，再靠廣播對齊。
- 推導題第 3 題：FP32 下每個參數需 $4 \text{ bytes} \times 4$ （參數、梯度、 m 、 v ）= 16 bytes，故 $3B \times 16$

= 48 GB。混合精度下約為 $3B \times (2+2+4+4) = 36 \text{ GB}$ 。

- 推導題第 4 題：softmax 後約為 [0.0470, 0.0064, 0.0023, 0.9443]；損失為 $-\log(0.0470) \approx 3.057$ ；梯度為 [-0.9530, 0.0064, 0.0023, 0.9443]。
- 系統設計第 2 題： $7B \times 16 \text{ bytes} = 112 \text{ GB}$ ，遠超 24 GB。三種方向為 LoRA（第 24 章，犧牲部分品質）、量化訓練（第 24 章 QLoRA，犧牲精度）、以及梯度累積加 offload（犧牲速度）。

研究延伸與版本注意事項

- 線性代數與最佳化的補強：若需要系統性的補課，建議選擇以「機器學習中的數學」為導向的教材，而非傳統的純數學課本——後者的重點與你的需求相差很遠。
- Adam 原始論文 (Kingma & Ba, 2015) 與 AdamW (Loshchilov & Hutter, 2019)。後者篇幅短、論證清晰，非常適合作為第一篇精讀的論文。
- 自動微分：PyTorch 官方的 autograd 教學文件是最好的入門；若想理解實作，可閱讀 micrograd 這類極簡的教學用實作。
- 數值計算：任何一本科學計算教材關於浮點數與數值穩定性的章節都值得一讀。IEEE 754 標準的科普介紹也很有幫助。
- 課程資源：Stanford CS231n 關於反向傳播與最佳化的講義至今仍是最清楚的教材之一，雖然它是視覺領域的課程，但這部分完全通用。李宏毅教授課程中關於最佳化的中文講解適合作為第一次接觸。
- 本章的所有公式都不會過時，但函式庫的 API 會變。程式碼中的 PyTorch 版本請以課程倉庫的 requirements 為準，並在每學期開課前確認。

常見問題

問：我完全沒學過微積分，能修這門課嗎？答：可以，但你必須接受一件事：有些地方你會先「知道結果」而不是「知道為什麼」。3.7 節提供的路徑是可行的，我教過的高中生用這條路走完了整本書。關鍵在於你要真的去跑那些程式，而不是只讀文字。

問：既然 PyTorch 會自動算梯度，為什麼還要學反向傳播？答：三個實際理由。第一，當梯度出問題（nan、消失、爆炸）時，你必須知道它是怎麼流動的才能診斷。第二，第 24 章的 LoRA、第 27 章的 DPO 都需要你理解梯度在哪裡被切斷、哪裡被保留。第三，記憶體優化（第 18 章的 activation checkpointing）本質上就是在拿計算換梯度儲存，不懂反向傳播就不知道在換什麼。

問：einsum 一定要學嗎？答：不一定要用，但一定要看得懂——因為很多論文的參考實作用它。建

議至少能讀懂注意力那幾行。

問：為什麼大家都用 AdamW，還需要知道 SGD 嗎？答：需要。在記憶體極度受限時（例如在單張消費級顯卡上微調），SGD 能省下三分之二的最佳化器狀態記憶體，這可能是「跑得動」與「跑不動」的差別。另外，理解 SGD 的問題才能理解 Adam 為何被設計成那樣。

教師使用建議

本章排在第 3 週，建議 2 小時講授加 1 小時實驗。這是全書數學密度最高的一章，也是學生最容易掉隊的一章，建議放慢速度。

時段	內容	互動設計	注意事項
前 30 分鐘	3.1 至 3.2 節，形狀推理與矩陣運算	現場出五題 shape tracing 讓學生舉手作答	這一段務必確認全班跟上再往下
中間 30 分鐘	3.3 節，機率、交叉熵與 softmax	用 [2.0, 1.0, 0.1] 帶全班手算一次 softmax	不要跳過數值例子
後 30 分鐘	3.4 節，梯度與反向傳播	用 3.4.4 節那張梯度表講解訓練直覺	交叉熵梯度是本章的高潮，值得多花時間
最後 30 分鐘	3.5 至 3.6 節，最佳化器與浮點數	現場示範 $0.1 + 0.2 \neq 0.3$	浮點數那段留在最後可提振精神
實驗課 1 小時	程式 3A 與 3B	3B 的抓蟲改成分組競賽，先找到的組加分	3C 可作為課後作業

差異化建議：對數學基礎好的學生，指定推導題第 2 題（交叉熵梯度）與研究延伸中的 AdamW 論文；對基礎較弱的學生，只要求完成 shape tracing 十題與程式 3A，並允許用數值實驗代替推導。兩組的期末要求可以不同，但 shape tracing 是所有人都必須通過的門檻——因為第 11 與 13 章完全建立在它上面。

本章小結

1. 形狀推理是本章最重要的技能：矩陣乘法消掉中間維、逐元素運算形狀不變、約簡運算讓指定維消失。在關鍵步驟寫下形狀斷言是強制要求。
2. 內積衡量方向相似度，是注意力機制的基礎；矩陣乘法一次完成大量內積，是 GPU 效率的來源，

能寫成矩陣乘法就不要寫迴圈。

3. 廣播方便但危險，不確定時就明確 `reshape`；`einsum` 把維度標記寫進程式碼，可讀性遠優於連續的 `transpose`。
4. 自回歸語言模型的目標由最大概似導出，在 `one-hot` 標籤下與交叉熵完全等價；困惑度是損失取指數。
5. `softmax` 必須減去最大值才不會溢位；所有機率計算一律在對數空間進行，`log-sum-exp` 是這個技巧的一般形式。
6. 梯度指向損失上升最快的方向；反向傳播用鏈式法則在計算圖上一次算出所有參數的梯度，成本約為兩次前向傳播。
7. 交叉熵對 logits 的梯度等於 `p` 減 `one-hot`，這個簡潔結果直接說明了訓練如何把機率質量從錯誤答案搬到正確答案。
8. 梯度檢查用中央差分驗證解析梯度，相對誤差小於 $1e-7$ 表示正確；做檢查前必須關閉 `dropout` 等隨機成分。
9. `AdamW` 是大模型訓練的標準選擇，代價是每個參數需要兩個額外狀態，記憶體約為參數的三倍——這是第 19、24 章各種節省技術的動機。
10. 浮點數不是實數：加法不滿足結合律、精度不均勻。`BF16` 因為指數範圍與 `FP32` 相同而成為主流。梯度裁切與梯度範數監控是訓練健康度的第一道防線。

成果檢核

完成本章後你必須繳交：

- 數學速查表：一頁 A4，包含本書統一符號表、形狀推理三規則、`softmax` 與交叉熵的穩定寫法、四種最佳化器的更新式、四種數值災難的徵兆與處置。這張表你會用整本書。
- `shape tracing` 十題的完整答案，並附上一段可執行的程式驗證你的答案（用 `assert`）。
- 程式 3A 的執行結果：四種 `scale` 下與 `PyTorch` 的比對，以及不穩定版本在 `scale=1000` 時的失敗紀錄。
- 程式 3B 的除錯報告：說明你如何定位錯誤、錯誤是什麼、以及你寫的回歸測試。
- 程式 3C 的軌跡圖與一段分析：四種最佳化器的行為差異，以及你認為這個玩具實驗的結論有哪些不能推廣到真實訓練。
- 選作：Attention 張量尺寸追蹤器，以及與 `PyTorch` 實測記憶體的對照驗證。

數學速查表請保留，第 11 章（注意力實作）、第 13 章（`MiniFormosaGPT`）、第 15 章（訓練穩定

性) 與第 18 章 (記憶體優化) 都會直接用到它。

附：數學速查表範本

以下是成果檢核要求的速查表結構，你可以直接依此製作。建議印出來貼在螢幕旁邊，第 11、13、15、18 章都會用到。

區塊	應包含的內容	用途
符號表	B 、 T 、 V 、 d 、 H 、 d_h 、 L 、 d_{ff} 的定義與典型值	讀任何論文或程式碼的基礎
形狀三規則	矩陣乘法、逐元素、約簡三種運算的形狀變化	shape tracing 的依據
注意力形狀鏈	從 (B,T,d) 到 (B,T,d) 的完整八步	第 11 章實作時直接對照
穩定寫法	softmax 減最大值、log-softmax、log-sum-exp	任何涉及機率的計算
損失與指標	交叉熵、困惑度、KL 散度的定義與換算	讀訓練日誌
梯度速查	八種常見運算的反向公式	手寫反向傳播與除錯
最佳化器	SGD、Momentum、Adam、AdamW 更新式與記憶體倍數	選擇與估算
記憶體公式	元素數乘位元組數、混合精度各項配置	判斷實驗跑不跑得動
FLOPs 估算	2mkn、前向 2ND、訓練 6ND	算力預算與 MFU
數值災難	四種現象的徵兆與處置	訓練出問題時的第一份檢查清單

這張表的價值不在於它包含多少知識，而在於它是你自己整理的。我強烈建議你手寫或自己排版，不要直接複製本書的表格——整理的過程本身就是學習，而且你會發現有些地方你以為懂了其實沒有。

第 4 章

Python、PyTorch 與可重現的大模型實驗環境

Python, PyTorch, and Reproducible LLM Experiment Environments

難度：核心 | 建議授課：第 4 週 | 硬體需求：A 級（有 GPU 者可加測 BF16 吞吐量）

叫獸開講

研究生最常說的一句話是「我上禮拜跑出來明明是這個數字」。可重現性不是潔癖，它是你唯一能證明自己沒搞錯的方法。這一章不教你任何模型，只教你怎麼把工作台整理好——因為接下來三十六章，你會在這張工作台上度過幾百個小時。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 熟練使用大模型開發所需的 Python 語法子集，包括型別註記、資料類別、迭代器與例外處理。
2. 正確使用 PyTorch 的 tensor、autograd、Module、Dataset 與 DataLoader，並說明各自的職責。
3. 判斷什麼工作應該放在 Notebook、什麼應該寫成腳本、什麼應該寫成套件與測試。
4. 偵測可用的運算裝置，估算 VRAM 上限，並在資源不足時降級執行。
5. 建立可鎖定版本的虛擬環境或容器，處理 Windows 與 Linux 的常見差異。
6. 正確設定隨機種子與 determinism，並理解為什麼完全的位元級重現往往做不到。
7. 設計結構化的設定檔、日誌與實驗追蹤，讓每次執行都可以被還原。
8. 安全地管理程式碼、資料版本、模型權重與機密金鑰，不把 API 金鑰推上 GitHub。
9. 交付一份可一鍵驗證的課程環境，附版本清單、硬體報告、測試結果與故障排除紀錄。

這一章為什麼值得認真對待

本章是全書唯一一章不教任何大模型概念的章節，也是最容易被學生跳過的章節。但根據我多年的觀察，專題做到後半段崩潰的組別，八成的根因都落在這一章：環境沒鎖版本、實驗沒記錄、資料沒版本、金鑰外洩、或是「換一台電腦就跑不起來」。花三小時把這一章做扎實，會在往後三十六章省下三十小時。

4.1 大模型開發需要的 Python 子集

Python 很大，但大模型開發實際上只用到其中一小部分。這一節不重教 Python，而是挑出那些「不會就會卡住」的語法，並說明它們在本書後續章節的用途。

4.1.1 型別註記：給人看的，也給工具看的

Python 不強制型別，但寫上註記有三個好處：編輯器能自動補全、靜態檢查工具能提早抓錯、以及最重要的——三個月後的你看得懂這個函式要什麼。

型別註記在大模型程式碼中的典型用法

```
from typing import Optional, Sequence
import torch
from torch import Tensor

def causal_attention(
    q: Tensor,                # (B, H, T, d_h)
    k: Tensor,                # (B, H, T, d_h)
    v: Tensor,                # (B, H, T, d_h)
    mask: Optional[Tensor] = None, # (1, 1, T, T) 或 None
    dropout_p: float = 0.0,
) -> Tensor:                # 回傳 (B, H, T, d_h)
    ...
```

請注意註解中的形狀標記。這是本書強制的慣例：每一個張量參數都必須在旁邊寫上形狀。這比型別註記本身更重要，因為 Python 的型別系統無法表達「這是一個四階張量且第三維是序列長度」。

4.1.2 資料類別與設定物件

模型設定有一堆參數（層數、維度、頭數、詞彙表大小.....），用一堆分散的變數傳來傳去是災難的開始。用 `dataclass` 把它們包起來，程式會乾淨很多。

模型設定的標準寫法

```
from dataclasses import dataclass, asdict

@dataclass
class ModelConfig:
    vocab_size: int = 32000
    d_model: int = 768
    n_layers: int = 12
    n_heads: int = 12
    d_ff: int = 3072
    max_seq_len: int = 1024
    dropout: float = 0.0

    def __post_init__(self):
        # 在建立時就檢查，不要等到 forward 才爆炸
        assert self.d_model % self.n_heads == 0, \
            f"d_model={self.d_model} 無法被 n_heads={self.n_heads} 整除"
        assert self.d_ff >= self.d_model, "d_ff 通常應大於等於 d_model"

    @property
    def d_head(self) -> int:
        return self.d_model // self.n_heads

    def n_params(self) -> int:
        """粗估參數量，用於資源規劃（精確版見第 12 章）"""
        emb = self.vocab_size * self.d_model
```

```

attn = 4 * self.d_model ** 2
ffn = 2 * self.d_model * self.d_ff
return emb + self.n_layers * (attn + ffn)

```

這個小小的 `_post_init_` 會在第 12 與 13 章救你很多次——它讓錯誤在建立設定的那一行就爆炸，而不是在訓練跑了十分鐘之後才出現一個難以理解的形狀錯誤。

4.1.3 迭代器、生成器與大檔案處理

第 9 章你要處理數 GB 的語料，不可能整份讀進記憶體。生成器是標準解法。

逐行處理大型語料的正確方式

```

from pathlib import Path
from typing import Iterator
import json

def read_jsonl(path: Path) -> Iterator[dict]:
    """一次只讀一行，記憶體占用與檔案大小無關"""
    with open(path, encoding="utf-8") as f:
        for line_no, line in enumerate(f, 1):
            line = line.strip()
            if not line:
                continue
            try:
                yield json.loads(line)
            except json.JSONDecodeError as e:
                # 不要讓一行壞資料炸掉整個管線
                print(f"[警告] {path.name}:{line_no} 解析失敗：{e}")
                continue

# 用法：可以串接多層處理而不占記憶體
docs = read_jsonl(Path("data/corpus.jsonl"))
clean = (d for d in docs if len(d["text"]) > 50)
for i, doc in enumerate(clean):
    if i >= 10: break
    print(doc["text"][:40])

```

4.1.4 例外處理：讓失敗留下線索

學生最常見的兩個壞習慣：一是完全不處理例外，讓程式在跑了六小時後崩潰；二是用一個空的 `except` 把所有錯誤吞掉，讓問題永遠找不到。

壞習慣	為什麼糟糕	正確做法	章節
<code>except: pass</code>	把 <code>KeyboardInterrupt</code> 與程式錯誤一起吞掉	指定具體的例外類型，並記錄下來	第 9 章
完全不處理	一筆壞資料毀掉整個管線	在資料層級容錯，在邏輯層級快速失敗	第 9 章

壞習慣	為什麼糟糕	正確做法	章節
只 print 不記錄	關掉終端機就什麼都沒了	用 logging 寫入檔案	本章 4.5 節
吞掉後繼續	結果是錯的但你不知道	記錄跳過的筆數，最後報告出來	第 9、16 章

一條實用的原則：資料層要容錯（一筆壞資料就跳過並記錄），邏輯層要快速失敗（設定錯誤、形狀錯誤應該立刻拋出）。把這兩者搞反是很多長時間訓練崩潰的原因。

4.2 PyTorch 五大元件

PyTorch 的核心可以歸納成五個東西。理解它們各自的職責與邊界，比記住 API 重要得多。

元件	職責	常見誤用	本書位置
Tensor	多維陣列與裝置管理	忘記 .to(device) 導致裝置不一致	第 3、11 章
autograd	記錄計算圖並反向求導	在不需要梯度的地方沒關掉，浪費記憶體	第 3、13 章
nn.Module	參數管理與模型組裝	把參數存成一般屬性，導致沒被註冊	第 11–13 章
Dataset	定義單一樣本怎麼取得	在 __getitem__ 裡做重運算，拖慢訓練	第 13 章
DataLoader	批次、打亂、平行載入	num_workers 設定不當造成瓶頸或死鎖	第 13、15 章

4.2.1 Tensor 與裝置

裝置管理的三個要點

```
import torch

# 1. 統一決定裝置，不要在程式各處硬寫 "cuda"
def get_device() -> torch.device:
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available(): # Apple Silicon
        return torch.device("mps")
    return torch.device("cpu")

device = get_device()
```

```
# 2. 建立時就指定裝置，而非先建在 CPU 再搬過去
x = torch.randn(8, 512, 768, device=device)      # 好
# x = torch.randn(8, 512, 768).to(device)         # 多一次複製

# 3. 取數值回 Python 時務必 detach 並轉 CPU
loss_value = loss.detach().cpu().item()           # 好
# losses.append(loss)                             # 壞：整張計算圖被留住，記憶體爆炸
```

第三點是初學者最常見的記憶體洩漏來源。如果你把帶梯度的 tensor 存進 list，整張計算圖都不會被釋放，訓練跑到第幾百步就會 OOM。永遠記得：要記錄數值就用 `.item()`，不要存 tensor。

4.2.2 autograd 的開關

什麼時候該關掉梯度

```
# 推論時：關掉梯度，省記憶體也變快
model.eval()
with torch.no_grad():
    logits = model(x)

# 更精簡的寫法 (PyTorch 2.x)
@torch.inference_mode()
def evaluate(model, loader):
    model.eval()
    total, n = 0.0, 0
    for x, y in loader:
        loss = compute_loss(model(x), y)
        total += loss.item() * y.numel(); n += y.numel()
    return total / n

# 凍結部分參數 (第 24 章 LoRA 會大量用到)
for p in model.embedding.parameters():
    p.requires_grad = False
```

`model.eval()` 與 `torch.no_grad()` 是兩件不同的事，經常被混淆。前者改變模型的行為（關掉 dropout、讓 BatchNorm 用執行統計）；後者關掉梯度記錄。評估時兩者都需要，缺一不可。忘記 `model.eval()` 會讓你的驗證損失無故偏高，而且很難察覺。

4.2.3 nn.Module：參數是怎麼被找到的

正確與錯誤的參數註冊

```
import torch.nn as nn

class GoodBlock(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.linear = nn.Linear(d, d)
        self.weight = nn.Parameter(torch.zeros(d))
        self.register_buffer("mask",
                              torch.tril(torch.ones(128, 128)))

class BadBlock(nn.Module):
```

```
def __init__(self, d):
    super().__init__()
    self.w = torch.zeros(d)          # 壞：不會出現在 parameters()
    self.layers = [nn.Linear(d, d)   # 壞：普通 list 不會被註冊
                    for _ in range(4)] # 應用 nn.ModuleList
```

BadBlock 的問題在於：程式跑得起來，但那些參數不會被最佳化器看到，也不會被存進 checkpoint。訓練會「成功」，但模型永遠學不好，而且你會找很久。診斷方法很簡單，養成習慣在建立模型後印一次參數量：

建模後的例行檢查

```
def model_summary(model: nn.Module):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"總參數 {total:,} 可訓練 {trainable:,} 凍結 {total - trainable:,}")
    for name, p in model.named_parameters():
        print(f" {name:<40} {tuple(p.shape)} {p.numel():,}")
```

4.2.4 Dataset 與 DataLoader

語言模型資料集的最小實作

```
from torch.utils.data import Dataset, DataLoader

class TokenChunkDataset(Dataset):
    """把一長串 token id 切成固定長度的訓練樣本"""
    def __init__(self, ids: torch.Tensor, block_size: int):
        self.ids = ids          # 一維 LongTensor
        self.block = block_size

    def __len__(self):
        return max(0, len(self.ids) - self.block)

    def __getitem__(self, i):
        x = self.ids[i : i + self.block]
        y = self.ids[i + 1 : i + 1 + self.block]  # 往右移一格
        return x, y

loader = DataLoader(
    ds, batch_size=8, shuffle=True,
    num_workers=4,          # Windows 上請先試 0，見 4.4 節
    pin_memory=True,        # GPU 訓練時加速資料搬移
    drop_last=True,         # 讓每個批次大小一致，避免形狀變動
)
```

注意 `__getitem__` 中的 `y` 是 `x` 往右移一格——這就是第 2 章那個「預測下一個詞元」目標的具體實作。

第 13 章你會直接用這個類別訓練 MiniFormosaGPT。

num_workers 的實務建議

這個參數決定用幾個子行程平行載入資料。設 0 表示在主行程載入（最慢但最不容易出問題）。在 Linux 上

通常設為 CPU 核心數的一半到全部；在 Windows 上因為行程建立方式不同，設大於 0 時必須把訓練程式碼放在 `if __name__ == "__main__":` 底下，否則會無限遞迴建立行程。這是 Windows 使用者最常遇到的第一個坑。

4.3 Notebook、腳本與套件：三種形態的分工

學生常犯的錯誤是把所有東西都寫在 Notebook 裡，然後在專題後期發現無法重現、無法測試、無法交接。這一節給出明確的分界。

形態	適合做什麼	不適合做什麼	本書中的例子
Notebook	探索資料、視覺化、試驗想法、教學演示	長時間訓練、可重複執行的流程、被別人引用的程式	第 2、10 章的視覺化
腳本	一次性的資料處理、單次實驗、可排程的工作	被多處引用的核心邏輯	第 9 章的清理管線
套件	模型定義、tokenizer、訓練迴圈、評測工具	一次性的探索	第 11–13 章的模型程式碼
測試	驗證核心邏輯的正確性，防止回歸	取代人工檢查輸出品質	全書所有實作

4.3.1 建議的專案結構

本書統一使用的目錄結構

```

llm-course/
├── pyproject.toml      # 專案定義與依賴
├── requirements.lock   # 鎖定的完整版本清單
├── README.md          # 如何重現：五行以內能跑起來
├── configs/           # 設定檔（YAML），不寫死在程式裡
│   ├── model_tiny.yaml
│   └── train_default.yaml
├── src/formosa/       # 核心套件（可被 import 與測試）
│   ├── __init__.py
│   ├── config.py
│   ├── tokenizer.py   # 第 7 章
│   ├── model.py       # 第 11–13 章
│   ├── data.py        # 第 9、13 章
│   ├── train.py       # 第 15 章
│   └── evaluate.py    # 第 36 章
├── scripts/          # 命令列進入點
│   ├── prepare_data.py
│   ├── train.py
│   └── generate.py
├── notebooks/         # 探索與教學，不放核心邏輯
└── tests/             # pytest
    ├── test_shapes.py
    └── test_tokenizer.py

```



```
└─ data/           # 不進版控（見 .gitignore）
└─ outputs/        # 訓練產出，不進版控
└─ .env.example    # 金鑰範本，真正的 .env 不進版控
```

4.3.2 最重要的三個測試

完整的測試套件不必一步到位，但以下三種測試在大模型開發中投資報酬率極高，建議從第一天就寫。

tests/test_shapes.py (本書所有模型實作的必備測試)

```
import pytest, torch
from formosa.config import ModelConfig
from formosa.model import MiniFormosaGPT

@pytest.fixture
def tiny_cfg():
    return ModelConfig(vocab_size=100, d_model=32, n_layers=2,
                      n_heads=4, d_ff=64, max_seq_len=16)

def test_forward_shape(tiny_cfg):
    """測試一：形狀正確"""
    m = MiniFormosaGPT(tiny_cfg)
    x = torch.randint(0, 100, (2, 16))
    logits = m(x)
    assert logits.shape == (2, 16, 100)

def test_causality(tiny_cfg):
    """測試二：因果性—改變未來的 token 不應影響過去的輸出"""
    m = MiniFormosaGPT(tiny_cfg).eval()
    x = torch.randint(0, 100, (1, 16))
    with torch.no_grad():
        out1 = m(x)
        x2 = x.clone(); x2[0, -1] = (x2[0, -1] + 1) % 100
        out2 = m(x2)
    assert torch.allclose(out1[0, :-1], out2[0, :-1], atol=1e-5)

def test_overfit_one_batch(tiny_cfg):
    """測試三：能不能背下一個批次—最有效的健全性檢查"""
    m = MiniFormosaGPT(tiny_cfg)
    opt = torch.optim.AdamW(m.parameters(), lr=1e-2)
    x = torch.randint(0, 100, (2, 16)); y = torch.randint(0, 100, (2, 16))
    for _ in range(200):
        loss = torch.nn.functional.cross_entropy(
            m(x).reshape(-1, 100), y.reshape(-1))
        opt.zero_grad(); loss.backward(); opt.step()
    assert loss.item() < 0.1, f"無法過擬合單一批次，loss={loss.item():.3f}"
```

第三個測試值得特別強調。「能不能把一個批次背起來」是判斷模型實作有沒有根本錯誤的最快方法：一個實作正確的模型，一定能把幾十個樣本完全記住（loss 趨近於 0）。如果做不到，代表梯度沒有正確流動、某層被凍結了、或是損失計算錯了。這個測試在第 13 章會是你的第一道防線。

測試的投資報酬率

很多學生覺得寫測試浪費時間。但請算一筆帳：test_forward_shape 花你三分鐘寫，而一個形狀錯誤如果你訓練六小時後才發現，你損失六小時。這個報酬率是 120 倍。而且測試只寫一次，往後每次改程式都在保

護你。

4.4 硬體偵測、資源估算與降級執行

4.4.1 先知道自己有什麼

scripts/hardware_report.py (本章成果檢核要用到)

```
import platform, sys, torch, psutil

def report():
    print("=" * 56)
    print(f"作業系統 {platform.system()} {platform.release()}")
    print(f"Python {sys.version.split()[0]}")
    print(f"PyTorch {torch.__version__}")
    print(f"CPU 核心 {psutil.cpu_count(logical=False)} 實體 / "
          f"{psutil.cpu_count()} 邏輯")
    print(f"系統記憶體 {psutil.virtual_memory().total / 1e9:.1f} GB")

    if torch.cuda.is_available():
        for i in range(torch.cuda.device_count()):
            p = torch.cuda.get_device_properties(i)
            print(f"GPU {i} {p.name}")
            print(f"VRAM {p.total_memory / 1e9:.1f} GB")
            print(f"運算能力 {p.major}.{p.minor}")
            print(f"CUDA 版本 {torch.version.cuda}")
            print(f"BF16 支援 {torch.cuda.is_bf16_supported()}")
    elif torch.backends.mps.is_available():
        print("加速裝置 Apple MPS (部分運算不支援，可能自動退回 CPU)")
    else:
        print("加速裝置 無，將使用 CPU (本書 A 級路線)")
    print("=" * 56)
```

4.4.2 從硬體反推可訓練的模型大小

第 3 章已經給了記憶體公式，這裡把它變成一個可以直接查的決策表。以下假設使用 AdamW、混合精度、序列長度 1024、批次 4。

VRAM	全參數訓練上限	LoRA 微調上限	建議做法
無 GPU (CPU)	約 10M 參數	不適用	第 13 章 tiny 設定，或用量化模型做推論
8 GB	約 100–200M	約 3B (4-bit)	走 QLoRA 路線，第 24 章
16 GB	約 300–500M	約 7B (4-bit)	本書 B 級主線
24 GB	約 700M–1B	約 13B (4-bit)	舒適的單卡開發環境
80 GB	約 3–7B	約 70B (4-bit)	第 19 章之前的單卡上限

VRAM	全參數訓練上限	LoRA 微調上限	建議做法
多卡叢集	依分片策略而定	—	第 19 章 FSDP / ZeRO

請注意這張表是「數量級的指引」而非保證。實際上限受序列長度、批次大小、activation checkpointing、注意力實作與框架版本影響甚大。真正該做的是自己量——第 18 章會教你用 profiler 找出記憶體去向。

4.4.3 降級執行：讓同一份程式碼在四級硬體上都能跑

本書的教學前提是「沒有 GPU 的學生也要能修完」。做到這件事的方法不是準備兩套程式碼，而是讓同一份程式碼依硬體自動降級。

src/formosa/scaling.py — 硬體感知的設定選擇

```
from dataclasses import replace

PROFILES = {
    "A": dict(d_model=128, n_layers=4, n_heads=4, max_seq_len=128,
              batch_size=8, grad_accum=1),
    "B": dict(d_model=512, n_layers=8, n_heads=8, max_seq_len=512,
              batch_size=8, grad_accum=4),
    "C": dict(d_model=1024, n_layers=16, n_heads=16, max_seq_len=1024,
              batch_size=16, grad_accum=8),
}

def pick_profile() -> str:
    import torch
    if not torch.cuda.is_available():
        return "A"
    vram = torch.cuda.get_device_properties(0).total_memory / 1e9
    if torch.cuda.device_count() >= 4 or vram >= 40:
        return "C"
    return "B" if vram >= 12 else "A"

def auto_config(base_cfg, override: str | None = None):
    level = override or pick_profile()
    cfg = replace(base_cfg, **{k: v for k, v in PROFILES[level].items()
                              if hasattr(base_cfg, k)})
    print(f"[環境] 採用 {level} 級設定：d={cfg.d_model} L={cfg.n_layers} "
          f"T={cfg.max_seq_len}")
    return cfg, level
```

這個設計有一個重要的教學價值：學生在 A 級筆電上跑出來的結果，雖然模型小、品質差，但整條流程是完整的、正確的。當他之後拿到 GPU 時，只需要改一個參數就能放大。這比「先學一個簡化版、之後再學真正的版本」要好得多。

一個關於教學公平的說明

把「沒有 GPU 也能學」當成設計約束，不只是為了照顧資源不足的學生，也是為了讓所有人都被迫理解資

源限制。一個從來沒有為記憶體煩惱過的學生，到了業界處理真實成本問題時會很吃力。限制本身就是教材。

4.5 環境、依賴與跨平台

4.5.1 為什麼「我這邊可以跑」不是答案

一個大模型專案通常依賴數十個套件，而這些套件彼此之間有版本相容性要求。同一份程式碼在你的電腦上跑得好好的，在助教的電腦上可能因為某個間接依賴的小版本差異而行為不同——這在深度學習中特別嚴重，因為數值運算的細節可能隨版本改變。

層級	要鎖定什麼	工具	本書要求
Python 版本	主版本與次版本	pyenv、conda、容器	必須寫在 README
直接依賴	你 import 的套件	pyproject.toml / requirements.txt	必須指定版本範圍
間接依賴	依賴的依賴	lock 檔 (pip freeze 或 uv lock)	必須產生並提交
系統層	CUDA、驅動、編譯器	容器或文件記錄	至少寫進硬體報告
資料版本	語料的哪一版	雜湊值或版本標籤	第 8 章要求

最小可行的環境鎖定流程

```
# 1. 建立隔離環境
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

# 2. 安裝直接依賴（版本範圍寫在 requirements.txt）
pip install -r requirements.txt

# 3. 產生完整鎖定清單（含所有間接依賴）
pip freeze > requirements.lock

# 4. 別人重現時用 lock 檔
pip install -r requirements.lock
```

requirements.txt 寫「我需要什麼」，requirements.lock 寫「當時實際裝了什麼」。兩者都要提交進版控。前者給人看，後者給機器用。

4.5.2 Windows 與 Linux 的五個常見差異

臺灣的教學現場，學生的筆電多半是 Windows，而伺服器多半是 Linux。這個落差會製造一些可預期的麻煩。

項目	差異	處置	影響
路徑分隔符號	Windows 用反斜線，Linux 用斜線	一律使用 <code>pathlib.Path</code> ，不要手動拼字串	高
多行程	Windows 用 <code>spawn</code> 而非 <code>fork</code>	<code>DataLoader</code> 的 <code>num_workers>0</code> 時要加 <code>__main__</code> 保護	高
檔案編碼	Windows 預設可能不是 UTF-8	所有 <code>open()</code> 明確寫 <code>encoding="utf-8"</code>	極高（中文必踩）
換行符號	CRLF 與 LF 差異	設定 <code>.gitattributes</code> ，避免 diff 全紅	中
大小寫敏感	Windows 檔名不分大小寫	統一用小寫命名，避免搬到 Linux 才發現找不到檔案	中

第三列必須特別強調：在 Windows 上不寫 `encoding="utf-8"` 開啟中文檔案，很可能得到亂碼或直接拋出 `UnicodeDecodeError`。這是本書讀者最可能遇到的第一個錯誤，而且第 6 章會告訴你為什麼——因為繁體中文的編碼問題比英文複雜得多。

跨平台安全的檔案處理

```
from pathlib import Path

# 好：pathlib 自動處理分隔符號
data_dir = Path("data") / "corpus" / "train.jsonl"

# 好：明確指定編碼與換行
with open(data_dir, encoding="utf-8", newline="") as f:
    ...

# 壞：在 Windows 上會出問題
# path = "data/corpus/train.jsonl"
# with open(path) as f: # 編碼看系統預設，中文可能爆炸
```

4.5.3 容器：一勞永逸但有門檻

容器把作業系統層以上的一切都固定下來，是可重現性的終極解法。但它有學習成本，而且在共用 GPU 伺服器上不一定被允許。

情境	建議	理由	難度
個人筆電探索	虛擬環境即可	容器的額外複雜度不划算	低
實驗室共用伺服器	虛擬環境 + lock 檔	多半已有共用的 CUDA 環境	低
跨機器重現實驗	容器	系統層差異是最難除錯的	中
交付給業界或部署	容器	這是業界標準做法	中
論文投稿附程式碼	容器 + lock 檔	審稿人要能一鍵重現	中

本書的立場是：課程作業用虛擬環境加 lock 檔即可；總整專題（第 40 章）建議提供容器，因為那份成果會被別人拿去跑。

4.6 隨機性、種子與可重現的真相

4.6.1 隨機性從哪裡來

一次訓練中的隨機來源比你想像的多。要控制它們，得先知道它們在哪裡。

來源	影響什麼	控制方式	是否可完全控制
參數初始化	模型的起點	<code>torch.manual_seed</code>	可以
資料打亂順序	每個批次的組成	DataLoader 的 generator	可以
dropout 遮罩	每次前向的丟棄位置	同上，受全域種子影響	可以
資料增強 / 取樣	樣本內容	各自的 rng 物件	可以
cuDNN 演算法選擇	數值結果的最低位	deterministic 模式	可以但會變慢
多執行緒歸約順序	浮點加法的順序	部分無法控制	不一定
分散式訓練的通訊順序	梯度歸約的順序	幾乎無法控制	否

本書統一使用的 seed 設定函式

```
import os, random
import numpy as np
import torch

def set_seed(seed: int = 42, deterministic: bool = False):
```

```

"""設定所有隨機源。deterministic=True 會變慢，僅在除錯時開啟"""
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
os.environ["PYTHONHASHSEED"] = str(seed)

if deterministic:
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    # 某些運算沒有確定性實作，會直接報錯而非默默不確定
    torch.use_deterministic_algorithms(True, warn_only=True)
    os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
else:
    torch.backends.cudnn.benchmark = True # 讓 cuDNN 自動選最快演算法

```

4.6.2 為什麼位元級重現往往做不到

這是一個必須誠實面對的限制。即使你設好所有種子，換一張不同型號的 GPU、換一個 CUDA 版本、或改變批次大小，結果就可能不同。原因回到第 3 章講的浮點數：加法不滿足結合律，而 GPU 上的歸約順序取決於硬體與演算法選擇。

所以本書對「可重現」的定義是分層的：

層級	定義	達成條件	本書要求
位元級重現	每個數字完全相同	同硬體、同版本、deterministic 模式	僅除錯時要求
統計級重現	多次執行的結果分布一致	固定種子集合，報告平均與變異	第 16 章的標準
結論級重現	別人能得到相同的結論	完整的設定、資料與流程紀錄	本書最低要求

第三層才是科學上真正重要的。一篇論文的價值不在於你能複製它的第七位小數，而在於別人照著做能得到同樣的結論。這也是為什麼第 16 章會要求你跑多個 seed 並報告變異——單次結果不能作為結論，這一點在第 16 章會用統計方法徹底處理。

一個常見的誤解

「我設了 seed 但結果還是不一樣」不代表你做錯了。先確認是不是換了硬體、改了批次大小、或用了非確定性的運算。如果差異只在小數點後幾位且不影響結論，那是正常的；如果差異大到改變結論，那才是問題——通常代表你的實驗對隨機性太敏感，需要更多樣本或更穩定的設定。

4.7 設定檔、日誌與實驗追蹤

4.7.1 把設定從程式碼裡拿出來

硬寫在程式裡的超參數是可重現性的頭號殺手。你改了學習率跑了一次，忘了改回來，三天後看到結果卻不知道當時用的是哪個值。

configs/train_default.yaml

```
# 每次實驗只改這個檔，程式碼不動
run_name: miniformosa_tiny_v1
seed: 42
profile: auto          # auto / A / B / C

model:
  vocab_size: 32000
  d_model: 512
  n_layers: 8
  n_heads: 8
  d_ff: 2048
  max_seq_len: 512
  dropout: 0.0

data:
  train_path: data/tw_corpus/train.bin
  valid_path: data/tw_corpus/valid.bin
  tokenizer: artifacts/tokenizer_32k.json
  data_version: "2026-08-15-a"    # 見第 8 章

train:
  batch_size: 8
  grad_accum: 4                # 有效批次 = 32
  max_steps: 20000
  lr: 3.0e-4
  warmup_steps: 500
  weight_decay: 0.1
  grad_clip: 1.0
  precision: bf16
  eval_every: 500
  save_every: 2000
```

4.7.2 日誌：不只是 print

src/formosa/logging_setup.py

```
import logging, sys, json, time
from pathlib import Path

def setup_logging(out_dir: Path, run_name: str):
    out_dir.mkdir(parents=True, exist_ok=True)
    fmt = "%(asctime)s %(levelname)-7s %(name)-18s %(message)s"
    logging.basicConfig(
        level=logging.INFO,
        format=fmt,
        handlers=[
            logging.StreamHandler(sys.stdout),
            logging.FileHandler(out_dir / f"{run_name}.log", encoding="utf-8"),
        ],
    )
    return logging.getLogger(run_name)
```



```

class JsonlMetricLogger:
    """指標寫成 JSONL，方便事後用 pandas 分析與繪圖"""
    def __init__(self, path: Path):
        path.parent.mkdir(parents=True, exist_ok=True)
        self.f = open(path, "a", encoding="utf-8")

    def log(self, step: int, **metrics):
        rec = {"step": step, "wall_time": time.time(), **metrics}
        self.f.write(json.dumps(rec, ensure_ascii=False) + "\n")
        self.f.flush()          # 立刻寫入，訓練中斷也不會遺失

    def close(self):
        self.f.close()

```

注意 flush() 那一行。訓練跑了六小時之後被 OOM 殺掉，如果日誌還在緩衝區裡，你什麼都拿不到。

犧牲一點效能換取「隨時被殺都有紀錄」是划算的。

4.7.3 環境指紋：讓每次執行都能被還原

這是本章最重要的一個工具。每次執行訓練時，自動記錄下當時的完整環境，存成一份指紋檔。

src/formosa/fingerprint.py

```

import subprocess, platform, sys, json, hashlib
from pathlib import Path
import torch

def git_info() -> dict:
    def run(cmd):
        try:
            return subprocess.check_output(cmd, shell=True,
                                           text=True).strip()
        except Exception:
            return "unknown"
    return {
        "commit": run("git rev-parse HEAD"),
        "branch": run("git rev-parse --abbrev-ref HEAD"),
        "dirty": run("git status --porcelain") != "", # 有未提交的修改?
    }

def file_hash(path: Path, chunk=1 << 20) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while block := f.read(chunk):
            h.update(block)
    return h.hexdigest()[:16]

def fingerprint(cfg: dict, data_files: list[Path]) -> dict:
    fp = {
        "python": sys.version.split()[0],
        "torch": torch.__version__,
        "cuda": torch.version.cuda,
        "platform": f"{platform.system()} {platform.release()}",
        "gpu": (torch.cuda.get_device_name(0)
                if torch.cuda.is_available() else None),
        "git": git_info(),
        "config": cfg,
        "data": {p.name: file_hash(p) for p in data_files},
    }

```

```
}
return fp
```

git 的 dirty 欄位特別重要：它告訴你這次執行時是否有未提交的修改。如果是 True，那麼 commit 雜湊值不足以還原當時的程式碼——這是一個必須被記錄的警訊。本書要求：正式的實驗（要寫進報告的那些）必須在 dirty 為 False 的狀態下執行。

4.7.4 實驗追蹤的最小可行方案

你不需要一開始就用複雜的實驗追蹤平台。一個 SQLite 或一個 JSONL 目錄就能滿足課程需求，而且沒有外部依賴、不需要網路、不會有資料外流問題。

方案	優點	缺點	建議情境
JSONL 檔案	零依賴、可用 pandas 分析、可進版控	沒有介面、需自己畫圖	課程作業、單人專題
SQLite	可查詢、單一檔案、零設定	仍需自己做視覺化	多次實驗的比較
本地追蹤工具	有網頁介面、圖表自動生成	多一層依賴	實驗室共用
雲端追蹤平台	協作方便、功能完整	資料外流風險、需網路、可能收費	需先確認資料敏感度

一個資料治理的提醒

如果你的實驗涉及敏感資料（學生資料、病歷、製程參數），使用雲端實驗追蹤平台時要特別小心——有些工具預設會上傳樣本資料或設定檔內容。在第 8 章之前，養成一個習慣：任何會把資料送出你機房的工具，都要先確認它送了什麼。

4.8 版本控制、資料、權重與機密

4.8.1 什麼該進 Git，什麼不該

項目	進版控？	理由	替代做法
原始碼、設定檔、測試	是	這是專案的本體	—
lock 檔、README	是	重現所需	—

項目	進版控？	理由	替代做法
小型範例資料 (<1 MB)	是	讓測試能跑	—
訓練語料	否	太大且可能有授權限制	記錄來源與雜湊值
模型權重	否	太大，且 Git 不適合二進位檔	獨立儲存並記錄版本
訓練輸出、日誌	否	每次都不同，會汙染歷史	存在 outputs/ 並排除
API 金鑰、密碼	絕對不行	一旦推上去就視為外洩	.env 檔 + 環境變數
個資、學生資料	絕對不行	違反資料治理原則	見第 8 章

.gitignore 的必備內容

```
# 環境與快取
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/

# 機密（最重要的兩行）
.env
*.key

# 資料與產出
data/
outputs/
checkpoints/
*.bin
*.safetensors

# 但保留範例與說明
!data/samples/
!.env.example
```

4.8.2 金鑰管理：三條不可妥協的規則

1. 金鑰永遠不寫在程式碼裡，一律從環境變數讀取。程式碼會被分享、會被截圖、會被推上 GitHub，環境變數不會。
2. 提供 .env.example 而非 .env。前者只有欄位名稱沒有值，可以安全地進版控，讓別人知道需要設定哪些變數。
3. 一旦懷疑外洩，立刻撤銷並重新產生，不要只是「把那個 commit 刪掉」——Git 的歷史很難真正清除，而且可能已經被爬走。

正確的金鑰讀取方式

```
import os
from dotenv import load_dotenv    # python-dotenv

load_dotenv()                    # 讀取專案根目錄的 .env

def get_api_key(name: str) -> str:
    key = os.environ.get(name)
    if not key:
        raise RuntimeError(
            f"環境變數 {name} 未設定。請複製 .env.example 為 .env 並填入。"
        )
    return key

# 用法
api_key = get_api_key("LLM_API_KEY")

# 絕對不要這樣寫：
# api_key = "sk-xxxxxxxxxxxxxxxxxxxxxx"
```

一個真實發生過的教訓

有學生把含 API 金鑰的 Notebook 推上公開倉庫，兩小時內金鑰就被自動爬蟲抓走並開始產生費用。GitHub 有金鑰掃描機制會通知你，但通知到的時候通常已經來不及。這就是為什麼規則三是「立刻撤銷」而不是「趕快刪掉」。

4.8.3 資料與模型的版本

程式碼有 Git，但資料與模型權重沒有。這在大模型專案中是個真問題：同一份程式碼配上不同版本的語料，會得到完全不同的模型。

課程層級的最小可行做法是：

- 每份資料檔算出 SHA-256 雜湊值的前 16 碼，寫進設定檔與環境指紋。
- 資料版本用日期加序號標記（例如 2026-08-15-a），並在 Data Card（第 8 章）中說明這一版做了什麼變更。
- 模型 checkpoint 的檔名包含 run_name、步數與設定雜湊，例如 miniformosa_tiny_v1_step20000_cfg3a7f.pt。
- checkpoint 內部除了權重，也要存入設定與環境指紋——這樣光憑一個檔案就能知道它是怎麼來的。

checkpoint 的標準內容

```
def save_checkpoint(path, model, optimizer, step, cfg, fp):
    torch.save({
        "model_state": model.state_dict(),
        "optim_state": optimizer.state_dict(),
        "step": step,
        "config": cfg,                # 沒有這個就不知道怎麼重建模型
        "fingerprint": fp,           # 沒有這個就不知道它是怎麼訓練的
```

```
"format_version": 1,      # 未來格式改變時的相容性依據
}, path)
```

4.9 教室現場：共用資源與作業規範

這一節是給教師與助教的，但學生也應該讀——理解制度為什麼這樣設計，會讓你更願意遵守。

4.9.1 共用 GPU 的三個問題與對策

問題	典型情況	對策	難度
資源被獨占	一位學生跑滿八張卡三天	排程系統或時段配額，設定單人上限	中
記憶體被吃光	有人忘記釋放，別人全部 OOM	限制單一行程的 VRAM，設定逾時清理	中
磁碟爆滿	checkpoint 沒清理，塞滿共用空間	每人配額 + 自動清理舊 checkpoint	低

一個在課程中運作良好的簡單制度：把 GPU 分成「開發時段」與「訓練時段」。開發時段（白天）限制單次執行不超過 30 分鐘，讓所有人都能除錯；訓練時段（夜間與週末）開放長時間工作，但需事先登記。這個制度不需要複雜的排程軟體，一張共用試算表就能運作。

4.9.2 作業繳交的統一規範

本書所有章节的實作作業都採用同一套規範，從第 4 章開始執行：

項目	要求	檢查方式	配分權重
可重現	README 五行以內能跑起來	助教在乾淨環境實測	高
環境指紋	每次執行自動產生並附上	檢查 fingerprint.json 存在且 dirty=false	高
測試	核心邏輯有 pytest 且全數通過	執行 pytest	中
日誌	訓練過程有完整紀錄	檢查 log 與 metrics.jsonl	中
無機密外洩	倉庫中無金鑰、無個資	自動掃描 + 人工抽查	一票否決

項目	要求	檢查方式	配分權重
分析	不只交程式，要有結果解讀	閱讀報告	高

最後一系列的「一票否決」是認真的。一份技術上完美但把 API 金鑰或學生個資推上公開倉庫的作業，不會及格。這不是嚴苛，這是專業底線——在業界，這種事會導致資安事件通報。

程式實作

本章三個實作都在 A 級硬體上完成，並且會成為你往後三十六章的基礎設施。請認真做，因為第 13 章之後你會一直在用它們。

程式 4A 可重現的訓練模板

目標：建立一個包含 seed、config、logging、checkpoint、fingerprint 與 pytest 的訓練骨架，同時支援 CPU 與 GPU。這份模板會在第 13、15、22、23 章被直接沿用。

scripts/train.py (骨架，模型部分留待第 13 章填入)

```
import argparse, json, time
from pathlib import Path
import torch, yaml

from formosa.config import ModelConfig, TrainConfig
from formosa.logging_setup import setup_logging, JsonlMetricLogger
from formosa.fingerprint import fingerprint
from formosa.scaling import auto_config
from formosa.utils import set_seed, get_device

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", type=Path, required=True)
    ap.add_argument("--profile", default=None, help="A / B / C，預設自動偵測")
    ap.add_argument("--resume", type=Path, default=None)
    args = ap.parse_args()

    raw = yaml.safe_load(args.config.read_text(encoding="utf-8"))
    mcfg = ModelConfig(**raw["model"])
    tcfg = TrainConfig(**raw["train"])
    mcfg, level = auto_config(mcfg, args.profile)

    out = Path("outputs") / raw["run_name"]
    log = setup_logging(out, raw["run_name"])
    metrics = JsonlMetricLogger(out / "metrics.jsonl")

    set_seed(raw["seed"])
    device = get_device()
    log.info(f"裝置 {device} 硬體層級 {level} 種子 {raw['seed']}")

    # 環境指紋：先寫檔再開始訓練，這樣中途失敗也留得下來
    fp = fingerprint(raw, [Path(raw["data"])[ "train_path" ]])
    (out / "fingerprint.json").write_text(
        json.dumps(fp, ensure_ascii=False, indent=2), encoding="utf-8")
    if fp["git"]["dirty"]:
        log.warning("偵測到未提交的修改，此次結果不可作為正式實驗紀錄")

    model, opt, start_step = build_or_resume(mcfg, tcfg, args.resume, device)
    log.info(f"參數量 {sum(p.numel() for p in model.parameters()):,}")

    t0 = time.perf_counter()
    for step in range(start_step, tcfg.max_steps):
```

```

    loss = train_one_step(model, opt, tcfg, device)
    if step % 50 == 0:
        metrics.log(step, loss=loss,
                    lr=opt.param_groups[0]["lr"],
                    elapsed=round(time.perf_counter() - t0, 1))
    if step % tcfg.save_every == 0 and step > 0:
        save_checkpoint(out / f"step{step:06d}.pt",
                        model, opt, step, raw, fp)
        log.info(f"已存檔 step={step}")

    metrics.close()

if __name__ == "__main__":    # Windows 的 DataLoader 需要這個保護
    main()

```

驗收條件 (本實作的評分依據) :

- 在乾淨環境依 README 執行，五行以內能跑起來並產生 outputs/ 內容。
- 同一個 seed 連跑兩次，前 100 步的 loss 序列一致 (CPU 上應完全相同，GPU 上允許小數點後五位的差異)。
- 中途以 Ctrl+C 中斷後，用 --resume 續訓，loss 曲線應該接得上，不會跳回起點。
- pytest 全數通過，且至少包含第 4.3.2 節的三個測試。
- 故意在 config 中把 d_model 設成不能被 n_heads 整除，程式應在啟動三秒內給出清楚的錯誤訊息。

程式 4B 矩陣乘法基準測試

目標：量測你的硬體在不同尺寸、不同精度下的實際吞吐量，並與理論峰值比較。這個數字會在第 17 章 (算力預算) 與第 18 章 (MFU) 被直接引用。

scripts/bench_matmul.py (節錄)

```

import time, json, torch
from pathlib import Path

def bench(n: int, dtype, device, iters: int = 20, warmup: int = 5):
    a = torch.randn(n, n, dtype=dtype, device=device)
    b = torch.randn(n, n, dtype=dtype, device=device)

    for _ in range(warmup):          # 暖機必須捨棄 (第 2 章的紀律)
        _ = a @ b
    if device.type == "cuda":
        torch.cuda.synchronize()

    t0 = time.perf_counter()
    for _ in range(iters):
        c = a @ b

```



```

if device.type == "cuda":
    torch.cuda.synchronize()    # 不加這行量到的是發指令的時間
dt = (time.perf_counter() - t0) / iters

flops = 2 * n ** 3              # 見第 3.2.5 節
return {
    "n": n,
    "dtype": str(dtype).replace("torch.", ""),
    "seconds": round(dt, 6),
    "TFLOPS": round(flops / dt / 1e12, 2),
    "mem_GB": round(3 * n * n * a.element_size() / 1e9, 3),
}

def main():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    dtypes = [torch.float32]
    if device.type == "cuda":
        dtypes += [torch.bfloat16, torch.float16]

    rows = []
    for n in [512, 1024, 2048, 4096]:
        for dt in dtypes:
            try:
                r = bench(n, dt, device)
                rows.append(r)
                print(f"n={r['n']}<5} {r['dtype']<9} "
                      f"{r['TFLOPS']>8.2f} TFLOPS {r['mem_GB']>6.3f} GB")
            except torch.cuda.OutOfMemoryError:
                print(f"n={n} {dt} 記憶體不足，跳過")
                torch.cuda.empty_cache()
    Path("outputs/bench_matmul.json").write_text(
        json.dumps(rows, indent=2), encoding="utf-8")

```

你應該觀察到並解釋的現象：

- 矩陣越大，達到的 TFLOPS 通常越高。為什麼？因為小矩陣的時間被 kernel launch 開銷與記憶體搬移主導，算力沒被填滿。這是第 18 章「計算受限 vs 頻寬受限」的第一次接觸。
- BF16 或 FP16 的 TFLOPS 通常遠高於 FP32。查出你的 GPU 規格書上的理論峰值，計算你實際達到的比例。這個比例就是 MFU 的雛形。
- 在 CPU 上執行時，TFLOPS 會低上兩三個數量級。把這個數字記下來——它解釋了為什麼第 15 章之後的訓練實驗需要 GPU。

程式 4C 實驗追蹤器最小實作

目標：用 SQLite 建立一個能記錄、查詢、比較實驗的工具。零外部依賴，可離線使用，不會有資料外流疑慮。

src/formosa/tracker.py (節錄)

```

import sqlite3, json, time
from pathlib import Path

```

```

SCHEMA = """
CREATE TABLE IF NOT EXISTS runs (
    run_id      TEXT PRIMARY KEY,
    created_at  REAL,
    config      TEXT,          -- JSON
    fingerprint TEXT,          -- JSON
    status      TEXT           -- running / done / failed
);
CREATE TABLE IF NOT EXISTS metrics (
    run_id TEXT, step INTEGER, name TEXT, value REAL,
    PRIMARY KEY (run_id, step, name)
);
"""

class Tracker:
    def __init__(self, db: Path = Path("outputs/experiments.db")):
        db.parent.mkdir(parents=True, exist_ok=True)
        self.con = sqlite3.connect(db)
        self.con.executescript(SCHEMA)

    def start(self, run_id, config, fp):
        self.con.execute(
            "INSERT OR REPLACE INTO runs VALUES (?, ?, ?, ?, ?)",
            (run_id, time.time(), json.dumps(config, ensure_ascii=False),
             json.dumps(fp, ensure_ascii=False), "running"))
        self.con.commit()

    def log(self, run_id, step, **kv):
        self.con.executemany(
            "INSERT OR REPLACE INTO metrics VALUES (?, ?, ?, ?)",
            [(run_id, step, k, float(v)) for k, v in kv.items()])
        self.con.commit()

    def best(self, metric="valid_loss", limit=10):
        """列出各次實驗在該指標上的最佳值，用於快速比較"""
        return self.con.execute("""
            SELECT run_id, MIN(value) AS best, MAX(step) AS steps
            FROM metrics WHERE name = ?
            GROUP BY run_id ORDER BY best ASC LIMIT ?
        """, (metric, limit)).fetchall()

```

延伸挑戰：

- 加入一個 diff 功能，輸入兩個 run_id，自動列出它們的設定有哪些欄位不同。這在你跑了三十次實驗之後會非常有用。
- 加入一個檢查：如果新實驗的設定與某個既有實驗完全相同，就警告使用者「你可能在重複已經做過的實驗」。
- 寫一個簡單的報表產生器，輸入指標名稱後輸出一張比較圖與一份 Markdown 表格，可直接貼進報告。

系統案例：三級教學環境的統一介面設計

本章的系統案例是為一個班級設計完整的運算環境方案，涵蓋 A 級筆電、B 級單卡工作站與 C 級多卡叢集，並確保同一份程式碼在三種環境下都能執行。

設計需求

層級	典型設備	要能完成的實驗	每人可用時數
A 級	學生自備筆電，無獨立顯卡	tokenizer、注意力、tiny 模型、RAG、API 應用	不限
B 級	實驗室工作站，單張 GPU	small 模型預訓練、QLoRA 微調、量化推論	每週 6 小時
C 級	多卡伺服器	分散式訓練、尺度實驗、專題訓練	需登記，專題階段開放

必須交付的四份文件

1. 環境安裝指南：三種層級各一份，從零到跑通第一個範例，每一步都要有預期輸出，讓學生能自行判斷是否成功。
2. 統一程式介面規格：定義所有實作必須支援的命令列參數（`--config`、`--profile`、`--resume` 等），以及降級規則。
3. 資源使用制度：時段劃分、配額、登記方式、超時處理、以及爭議如何解決。
4. 故障排除手冊：至少收錄十五個常見錯誤與處置，包括 CUDA 版本不符、OOM、Windows 編碼錯誤、`num_workers` 死鎖、磁碟滿等。

評分重點

- 同一份程式碼是否真的能在三種環境跑起來（助教會實測）。
- 降級規則是否合理：A 級跑出來的結果雖然品質差，但流程必須完整正確。
- 故障排除手冊的實用性：錯誤訊息要能被搜尋到，處置步驟要具體。
- 制度設計是否考慮了公平性：沒有 GPU 的學生會不會因此在成績上吃虧？

這份成果不只是作業。如果你未來要在系上開設類似課程，或在公司裡導入大模型開發，這四份文件就是你的起點。

章末評量

一、概念題

1. 說明 Notebook、腳本、套件與測試四種形態各自適合什麼工作，並解釋為什麼核心邏輯不應該寫在 Notebook 裡。
2. `model.eval()` 與 `torch.no_grad()` 分別做了什麼？評估時為什麼兩者都需要？只用其中一個會有什麼後果？
3. 為什麼把帶梯度的 `tensor` 存進 `list` 會造成記憶體洩漏？正確的做法是什麼？
4. 解釋「位元級重現」「統計級重現」「結論級重現」的差異，並說明為什麼科學上最重要的是第三種。
5. `requirements.txt` 與 `requirements.lock` 的差別是什麼？為什麼兩者都要提交？
6. 為什麼「資料層要容錯、邏輯層要快速失敗」？把這兩者搞反會發生什麼？

二、計算與估算題

1. 你的 GPU 有 16 GB VRAM。使用 AdamW 混合精度訓練，模型參數量 400M、序列長度 1024、批次 8。請估算參數、梯度、最佳化器狀態各占多少，並判斷還剩多少給 activation。（提示：第 3 章）
2. 承上題，若改用 LoRA 只訓練 1% 的參數，最佳化器狀態的記憶體變成多少？可訓練的模型能放大到多少參數量？
3. 程式 4B 測出你的裝置在 BF16 下達到 X TFLOPS，而規格書標示理論峰值為 Y TFLOPS。寫出 MFU 的計算式，並說明為什麼實際值通常遠低於理論值。
4. 一份訓練跑 20000 步，每 50 步記錄一次指標，每筆指標約 200 bytes。計算 `metrics.jsonl` 的大小。若改成每步都記錄呢？請據此討論記錄頻率的取捨。

三、除錯題

1. 同學的程式在自己電腦上正常，助教執行時出現 `UnicodeDecodeError`。請指出最可能的原因與修正方式，並說明為什麼這個問題在中文專案上特別常見。
2. 同學的 `DataLoader` 設定 `num_workers=4`，在 Windows 上執行時程式無限重複建立行程直到當機。請說明原因與修正。
3. 同學設定了 `seed`，但兩次執行的 `loss` 曲線仍然不同。請列出五個可能原因，並依「最可能」到「最不可能」排序。

4. 一個模型訓練時 loss 完全不下降，停在初始值附近。請依序列出你的五個檢查步驟（提示：其中一個與 `nn.Module` 的參數註冊有關）。
5. 同學的訓練跑到第 3000 步時 OOM，但前 2999 步都正常。請列出三個可能原因。

四、系統設計題

1. 為你的專題設計一份完整的可重現性方案，涵蓋程式碼、環境、資料、模型權重與實驗紀錄五個面向。每個面向都要說明：用什麼工具、記錄什麼、以及「半年後別人要重現時，缺了它會怎樣」。
2. 你的實驗室有兩張 GPU 要給十五位學生使用。請設計一套資源分配制度，需考慮公平性、除錯需求、長時間訓練需求、以及專題衝刺期的特殊安排。並說明你的制度在什麼情況下會失效。

五、臺灣情境與倫理題

1. 臺灣的教學現場學生多用 Windows 筆電，而伺服器多為 Linux。請整理一份「Windows 學生的前五個坑」清單，每項附上症狀、原因與解法，可作為你的課程助教手冊。
2. 你在整理專題倉庫時，發現三個月前的某個 commit 裡有一位同學不小心提交的 API 金鑰，該金鑰目前仍然有效。請說明你的處理步驟與順序，以及為什麼「刪掉那個 commit」不是足夠的處置。
3. 有人主張「課程作業不需要那麼嚴格的資料治理，反正只是練習」。請提出三個反駁，以及一個這個主張有道理的地方。

評量提示（給自學者）

- 計算題第 1 題：BF16 參數 0.8 GB、BF16 梯度 0.8 GB、FP32 主權重 1.6 GB、m 與 v 各 1.6 GB，合計約 6.4 GB，剩下約 9.6 GB 給 activation 與框架開銷。
- 計算題第 2 題：最佳化器狀態從 3.2 GB 降到約 0.032 GB。但要注意底模的權重仍需載入（雖然可用 4-bit 量化大幅壓縮），這才是 QLoRA 能微調 7B 的關鍵。
- 除錯題第 3 題排序建議：換了硬體或批次大小 > 有非確定性運算（如某些 scatter 操作）> DataLoader 的 worker 有自己的隨機狀態 > 沒有設定 PYTHONHASHSEED > cuDNN benchmark 模式選了不同演算法。
- 除錯題第 4 題：檢查參數是否被正確註冊（`nn.ModuleList` vs `list`）> 檢查 optimizer 是否拿到了 `model.parameters()` > 檢查是否忘了 `opt.step()` 或 `loss.backward()` > 檢查學習率是否為 0 或極小 > 檢查 loss 是否對正確的目標計算。
- 除錯題第 5 題：驗證時沒有 `no_grad` 導致額外記憶體累積 > 把 loss tensor 存進 list > 序列長度

不固定，某個批次特別長。

研究延伸與版本注意事項

- PyTorch 官方文件中關於 autograd 機制、Dataset 與 DataLoader、以及 reproducibility 的三份說明，是本章最直接的延伸閱讀，建議在做程式 4A 時對照閱讀。
- 可重現性研究：機器學習領域近年對「重現危機」有大量討論，相關的檢查清單（例如各大會議的 reproducibility checklist）值得參考，第 16 章會再深入。
- 軟體工程實務：測試驅動開發與持續整合的基本概念，對大模型專案同樣適用。不需要讀完整本書，理解「為什麼要測試」與「什麼該測」即可。
- 容器與環境管理：若你的專題需要交付給外部單位，建議至少學會撰寫一份基本的容器設定檔。
- 本章提到的所有工具版本都會變動。requirements 請以課程倉庫為準，並在每學期開課前重新產生 lock 檔並測試。

常見問題

問：我只是做作業，需要搞這麼複雜嗎？答：本章的東西加起來大約三小時，而且只做一次。相對地，我每學期都會看到專題組別在最後兩週因為「換一台電腦跑不起來」「不知道那個最好的結果是用什麼設定跑的」而損失數十小時。這筆帳很好算。

問：Notebook 真的不能用嗎？答：可以用，而且應該用——用在它擅長的地方：探索資料、畫圖、驗證想法、教學演示。本章反對的是「把訓練迴圈與模型定義寫在 Notebook 裡」，因為那些程式碼需要被測試、被引用、被重複執行。實務上的做法是：在 Notebook 裡 import 你的套件，而不是在 Notebook 裡定義它。

問：我沒有 GPU，這一章的 4B 還有意義嗎？答：有，而且很重要。你會量到 CPU 的實際吞吐量，這個數字讓你能估算「這個實驗在我的筆電上要跑多久」，進而決定該用多大的設定。知道自己的極限在哪，比擁有一張 GPU 更重要。

問：實驗追蹤一定要自己寫嗎？現成工具不是更好？答：現成工具功能更完整，課程也不禁止使用。要求你寫一次的理由有二：一是你會理解這些工具在做什麼，選型時才有判斷力；二是本章的版本完全離線，在處理敏感資料的專題中反而是優勢。

教師使用建議

本章排在第 4 週，建議 2 小時講授加 1 小時實驗。這是全書唯一一章「講授比例應該低、實作比例應

該高」的章節——學生必須在課堂上真的把環境建起來。

時段	內容	互動設計	注意事項
前 20 分鐘	4.1 至 4.2 節，Python 與 PyTorch 要點	快速帶過，重點放在三個常見誤用	不要重教 Python 基礎
中間 30 分鐘	4.3 至 4.4 節，專案結構與硬體	現場執行 hardware_report，收集全班的硬體分布	這份分布會決定後續課程的設定
後 35 分鐘	4.5 至 4.7 節，環境、種子與追蹤	示範「改一個版本號導致結果不同」	4.6.2 節的誠實說明很重要
最後 35 分鐘	4.8 至 4.9 節，版控與規範	現場示範金鑰外洩的後果（用測試金鑰）	規範必須在此週定案並公告
實驗課 1 小時	程式 4A 建置	兩人一組互相在對方電腦上跑一次	互測是最有效的可重現性檢驗

一個強烈建議的做法：在課堂上收集全班的 hardware_report 輸出，做成一張硬體分布表。這張表會告訴你這學期的實作設定該定在哪個層級，也讓學生看到彼此的資源差異——這是一個自然的時機來說明本書為何設計四級硬體路徑。

高中授課的調整：4.1 至 4.4 節完整教（這些是通用的工程素養），4.5 至 4.8 節簡化為「三個必做」——設 seed、用設定檔、不要把金鑰放進程式碼。4.9 節可略。程式 4A 可提供半成品讓學生填空，不必從零建置。

本章小結

1. 大模型開發只需要 Python 的一小部分：型別註記與形狀註解、dataclass 設定物件、生成器處理大檔案、以及正確的例外策略（資料層容錯、邏輯層快速失敗）。
2. PyTorch 的五大元件各有職責：Tensor 管裝置、autograd 管梯度、Module 管參數、Dataset 定義取樣、DataLoader 管批次與平行載入。三個最常見的誤用是裝置不一致、把帶梯度的 tensor 存進 list、以及參數沒被正確註冊。
3. Notebook 用於探索與教學，核心邏輯必須寫成可測試的套件。三個投資報酬率最高的測試是形狀測試、因果性測試與單批次過擬合測試。
4. 先偵測硬體再決定設定：同一份程式碼應能依 VRAM 自動降級，讓沒有 GPU 的學生也能完整走

完流程。

5. 環境必須分層鎖定：Python 版本、直接依賴、間接依賴（lock 檔）、系統層與資料版本。
Windows 與 Linux 的五個差異中，檔案編碼對中文專案影響最大。
6. 隨機性有七個來源，設定 seed 只能控制其中一部分；位元級重現往往做不到，本書要求的是統計級與結論級重現。
7. 設定必須外部化成 YAML；日誌要寫檔並立刻 flush；環境指紋（含 git commit 與 dirty 旗標、資料雜湊）是讓實驗可還原的關鍵。
8. 版本控制的界線很明確：原始碼與設定進版控，資料、權重與輸出不進；金鑰與個資絕對不進，一旦外洩必須立刻撤銷而非刪除 commit。
9. checkpoint 應包含權重、最佳化器狀態、步數、設定與環境指紋——光憑一個檔案就能知道它是怎麼來的。
10. 共用資源需要制度而非默契：時段劃分、配額與登記，加上一份具體的故障排除手冊。

成果檢核

完成本章後你必須繳交一套「可一鍵驗證的課程環境」，內容包括：

- 專案骨架：依 4.3.1 節的結構建立，含 pyproject.toml、requirements.lock、README（五行以內能跑起來）。
- 硬體報告：hardware_report.py 的完整輸出，並標明你屬於哪一個硬體層級。
- 訓練模板（程式 4A）：通過全部五項驗收條件，特別是「同 seed 兩次執行結果一致」與「中斷後可續訓」。
- 基準測試（程式 4B）：矩陣乘法的吞吐量表格與一段分析，說明你的裝置在什麼尺寸下才能發揮效能。
- 實驗追蹤器（程式 4C）：能記錄與查詢，並附上至少三筆測試用的實驗紀錄。
- 故障排除紀錄：你在建置過程中實際遇到的問題與解法，至少三則。這份紀錄會成為班級共用手冊的素材。
- 安全檢查：確認倉庫中沒有金鑰、沒有個資、.gitignore 完整。

這套環境會被你使用整整三十六章。第 7 章的 tokenizer、第 13 章的 MiniFormosaGPT、第 15 章的長時間訓練、第 22 章的持續預訓練、第 40 章的總整專題，全部建立在你這一週交出來的東西上。現在多花一小時，往後省下的遠不只十倍。

索引

依詞條首字排序，數字為出現的章次。英文詞條排在中文詞條之前。索引依本冊（第 1–4 章）內容編製；跨篇的完整索引見全書合訂本。

英文詞條

Adam 3、4	MFU 3、4
AdamW 3、4	MoE 1、2
Agent 1、2	Momentum 3
autograd 3、4	n-gram 2
baseline 1	nn.Module 4
BERT 2	provenance 1
BF16 3、4	PyTorch 3、4
checkpoint 3、4	QLoRA 1、3、4
Data Card 1、4	RAG 1、2、4
DataLoader 4	RLHF 1
Dataset 4	RMSNorm 2
determinism 4	RNN 1、2、3
DPO 1、3	RoPE 2
dropout 3、4	SGD 3
einsum 3	SLO 1
FlashAttention 2、3	softmax 2、3
FLOPs 3	SwiGLU 2、3
FP32 3、4	T5 2
FP8 3	TAIDE 1
FSDP 4	token 稅 1
GPT 1、2、3、4	tokenizer 1、2、4
GRU 2	Transformer 1、2、3
KV cache 2	VRAM 3、4
log-sum-exp 2、3	warmup 3
LoRA 1、3、4	Word2Vec 2
LSTM 2	ZeRO 3、4

中文詞條

上下文長度 1	推理模型 1、2
分散式訓練 1、3、4	梯度 2、3、4

分詞器 1	梯度消失 2、3
反向傳播 3	梯度累積 3
尺度定律 1、2、3	梯度裁切 2、3
幻覺 1、2	梯度爆炸 2、3
去識別 1	混合精度 2、3、4
可重現 1、4	嵌入 1、2、3
交叉熵 2、3	智慧體 1
因果遮罩 1、3	殘差連接 2、3
污染 1、2	評測 1、2、4
自注意力 2	詞元 1、2、3、4
自動微分 3	量化 1、2、3、4
位置編碼 2	開放權重 1、2
困惑度 1、2、3	微調 1、2、3、4
批次大小 2、3、4	資料平行 3
注意力 1、2、3、4	對齊 1、2、3
前饋網路 2、3	蒸餾 1、3
思考鏈 1	算力 1、2、3、4
指令微調 1、2	臺語 1
紅隊 1	臺灣用語 1
個資 1、2、3、4	廣播 3
浮點 1、2、3、4	學習率 2、3、4
消融 1	隨機種子 2、4
參數量 1、2、3、4	檢索 1、2、3
基礎模型 1、2	環境指紋 1、4
張量 1、3、4	繁體中文 1、2、4
授權 1、2、4	

大模型導論 Introduction to Large Language Models

第 1 篇 大模型視野、數學與工程基礎 (第 1–4 章)

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

課程與勘誤 : @prof (<https://page.line.me/prof>)

臺灣自編大學教科書系列 | 2026 年 8 月 第一版